

**Technical Reference Manual for Castalia**

Revised July 23<sup>rd</sup>, 2026

University of Nebraska–Lincoln  
Department of Electrical & Computer Engineering

## Contents

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>System Configuration</b>                              | <b>8</b>  |
| 1.1      | Chip Configuration . . . . .                             | 9         |
| <b>2</b> | <b>Packaging and Pins Configuration</b>                  | <b>12</b> |
| <b>3</b> | <b>Address Space</b>                                     | <b>17</b> |
| <b>4</b> | <b>Multi-Core Architecture</b>                           | <b>18</b> |
| 4.1      | Harts and Roles . . . . .                                | 18        |
| 4.2      | The Shared Window . . . . .                              | 19        |
| 4.2.1    | Arbitration and Stalling . . . . .                       | 19        |
| 4.3      | Boot and Tile Loading . . . . .                          | 19        |
| 4.4      | Synchronization . . . . .                                | 21        |
| <b>5</b> | <b>Startup and Reset Behavior</b>                        | <b>23</b> |
| <b>6</b> | <b>Interrupts</b>                                        | <b>24</b> |
| 6.1      | Interrupt Vector Table . . . . .                         | 24        |
| 6.2      | Interrupt Entry, Exit, and Recursion . . . . .           | 25        |
| 6.3      | Sleep Mode and Interrupt Wake-Up . . . . .               | 25        |
| <b>7</b> | <b>CPU Instructions and Assembly</b>                     | <b>26</b> |
| 7.1      | CPU Registers . . . . .                                  | 26        |
| 7.2      | Load/Store Memory Instructions . . . . .                 | 27        |
| 7.3      | Shift Instructions . . . . .                             | 27        |
| 7.4      | Arithmetic Instructions . . . . .                        | 28        |
| 7.5      | Bitwise Instructions . . . . .                           | 28        |
| 7.6      | Comparison Instructions . . . . .                        | 29        |
| 7.7      | Branch Instructions . . . . .                            | 29        |
| 7.8      | Jump and Link Instructions . . . . .                     | 29        |
| 7.9      | Atomic Memory Operations (A Extension) . . . . .         | 30        |
| 7.10     | Bit Manipulation Instructions (Zb* Extensions) . . . . . | 31        |
| 7.10.1   | Address Generation (Zba) . . . . .                       | 31        |
| 7.10.2   | Basic Bit Manipulation (Zbb) . . . . .                   | 31        |
| 7.10.3   | Carryless Multiplication (Zbc) . . . . .                 | 32        |
| 7.10.4   | Single-Bit Manipulation (Zbs) . . . . .                  | 32        |
| 7.11     | Pseudo-Instructions . . . . .                            | 33        |
| <b>8</b> | <b>Forth Interpreter</b>                                 | <b>35</b> |
| 8.1      | Memory Access Functions . . . . .                        | 35        |
| 8.2      | Print and Input Functions . . . . .                      | 36        |
| 8.3      | Arithmetic Functions . . . . .                           | 37        |
| 8.4      | Bitwise Logic Functions . . . . .                        | 39        |
| 8.5      | Comparison Functions . . . . .                           | 41        |
| 8.6      | Boolean Functions . . . . .                              | 41        |
| 8.7      | Stack Manipulation Functions . . . . .                   | 42        |
| 8.8      | SPI Flash R/W and Programming Functions . . . . .        | 44        |

|           |                                                            |           |
|-----------|------------------------------------------------------------|-----------|
| 8.9       | Miscellaneous Functions . . . . .                          | 44        |
| 8.10      | Creating New Forth Functions . . . . .                     | 46        |
| 8.11      | Conditionals . . . . .                                     | 46        |
| 8.12      | Loops . . . . .                                            | 46        |
| <b>9</b>  | <b>Software Development and Build Process</b>              | <b>47</b> |
| 9.1       | The RISC-V Toolchain and its Extensions . . . . .          | 47        |
| 9.2       | Getting the RISC-V Toolchain on Linux . . . . .            | 47        |
| 9.2.1     | Option 1: Prebuilt xPack Toolchain (Recommended) . . . . . | 48        |
| 9.2.2     | Option 2: Building from Source . . . . .                   | 48        |
| 9.3       | Getting the RISC-V Toolchain on Windows . . . . .          | 49        |
| 9.4       | Compilation . . . . .                                      | 49        |
| 9.5       | Linking . . . . .                                          | 51        |
| 9.6       | Intel Hex File Generation . . . . .                        | 51        |
| 9.7       | A Note on Using C++ . . . . .                              | 52        |
| <b>10</b> | <b>Uploading Program to SPI Flash</b>                      | <b>53</b> |
| 10.1      | SPI Flash Boot Process . . . . .                           | 53        |
| 10.2      | Upload Process . . . . .                                   | 53        |
| <b>11</b> | <b>GPIOx Peripheral</b>                                    | <b>55</b> |
| 11.1      | Primary/GPIO Mode Functionality . . . . .                  | 55        |
| 11.2      | Register Access Convenience Features . . . . .             | 55        |
| 11.3      | Alternate Function (Peripheral) Mode . . . . .             | 56        |
| 11.3.1    | Relocatable Peripheral Inputs . . . . .                    | 57        |
| 11.4      | Pin Interrupts . . . . .                                   | 57        |
| 11.5      | Register Description . . . . .                             | 58        |
| 11.5.1    | PIN Register . . . . .                                     | 58        |
| 11.5.2    | POUT Register . . . . .                                    | 58        |
| 11.5.3    | POUTS Register . . . . .                                   | 58        |
| 11.5.4    | POUTC Register . . . . .                                   | 58        |
| 11.5.5    | POUTT Register . . . . .                                   | 59        |
| 11.5.6    | PDIR Register . . . . .                                    | 59        |
| 11.5.7    | PIF Register . . . . .                                     | 59        |
| 11.5.8    | PIES Register . . . . .                                    | 59        |
| 11.5.9    | PIE Register . . . . .                                     | 60        |
| 11.5.10   | PSEL Register . . . . .                                    | 60        |
| 11.5.11   | PREN Register . . . . .                                    | 60        |
| 11.5.12   | PAFS Register . . . . .                                    | 61        |
| <b>12</b> | <b>SPIx Peripheral</b>                                     | <b>62</b> |
| 12.1      | SPI Pins . . . . .                                         | 62        |
| 12.2      | Shared Access on Castalia . . . . .                        | 63        |
| 12.3      | Generalized SPI Transfer Process . . . . .                 | 64        |
| 12.4      | Bit and Byte Ordering . . . . .                            | 65        |
| 12.5      | SPI Baud Clock . . . . .                                   | 65        |
| 12.6      | Transmit Buffer Register Queue . . . . .                   | 65        |
| 12.7      | Flags and Interrupts . . . . .                             | 66        |

|           |                                                             |           |
|-----------|-------------------------------------------------------------|-----------|
| 12.8      | Master Mode Transfer Procedure . . . . .                    | 66        |
| 12.9      | Slave Mode Transfer Procedure . . . . .                     | 67        |
| 12.10     | External SPI Flash Extended Memory Access . . . . .         | 68        |
| 12.10.1   | Flash Address Translation . . . . .                         | 68        |
| 12.10.2   | Flash Initialization Procedure . . . . .                    | 68        |
| 12.11     | SPI0 Boot Behavior . . . . .                                | 69        |
| 12.12     | Register Description . . . . .                              | 70        |
| 12.12.1   | SPICR Register . . . . .                                    | 70        |
| 12.12.2   | SPISR Register . . . . .                                    | 71        |
| 12.12.3   | SPITX Register . . . . .                                    | 72        |
| 12.12.4   | SPIRX Register . . . . .                                    | 72        |
| 12.12.5   | SPIFOS Register . . . . .                                   | 73        |
| <b>13</b> | <b>UARTx Peripheral</b>                                     | <b>75</b> |
| 13.1      | UART Pins . . . . .                                         | 75        |
| 13.2      | Shared Access on Castalia . . . . .                         | 76        |
| 13.3      | Generalized UART Transmission Process . . . . .             | 76        |
| 13.4      | UART Baud Generation . . . . .                              | 76        |
| 13.5      | Parity Bit . . . . .                                        | 77        |
| 13.6      | Transmit Buffer Register Queue . . . . .                    | 77        |
| 13.7      | Flags and Interrupts . . . . .                              | 77        |
| 13.8      | Transmit Procedure . . . . .                                | 78        |
| 13.9      | Receive Procedure . . . . .                                 | 79        |
| 13.10     | UART0 Boot Behavior . . . . .                               | 79        |
| 13.11     | Register Description . . . . .                              | 80        |
| 13.11.1   | UARTCR Register . . . . .                                   | 80        |
| 13.11.2   | UARTSR Register . . . . .                                   | 80        |
| 13.11.3   | UARTBR Register . . . . .                                   | 81        |
| 13.11.4   | UARTRX Register . . . . .                                   | 82        |
| 13.11.5   | UARTTX Register . . . . .                                   | 82        |
| <b>14</b> | <b>TIMERx Peripheral</b>                                    | <b>83</b> |
| 14.1      | Timer Pins . . . . .                                        | 83        |
| 14.2      | Clock Sources and Divider . . . . .                         | 83        |
| 14.3      | Starting, Halting, Setting, and Reading the Timer . . . . . | 84        |
| 14.4      | Overflow and Rollover Behavior . . . . .                    | 84        |
| 14.5      | Output Compare Units and Pulse Width Modulation . . . . .   | 85        |
| 14.6      | Input Capture Units . . . . .                               | 86        |
| 14.7      | Flags and Interrupts . . . . .                              | 86        |
| 14.8      | Register Description . . . . .                              | 87        |
| 14.8.1    | TIMCR Register . . . . .                                    | 87        |
| 14.8.2    | TIMSR Register . . . . .                                    | 89        |
| 14.8.3    | TIMVAL Register . . . . .                                   | 90        |
| 14.8.4    | TIMCMP0 Register . . . . .                                  | 90        |
| 14.8.5    | TIMCMP1 Register . . . . .                                  | 91        |
| 14.8.6    | TIMCMP2 Register . . . . .                                  | 91        |
| 14.8.7    | TIMCAP0 Register . . . . .                                  | 92        |
| 14.8.8    | TIMCAP1 Register . . . . .                                  | 93        |

|                                                   |            |
|---------------------------------------------------|------------|
| <b>15 SYSTEM Peripheral</b>                       | <b>94</b>  |
| 15.1 System Clocks and Clock Management . . . . . | 94         |
| 15.2 Power Saving Features . . . . .              | 94         |
| 15.3 CPU Sleep Mode . . . . .                     | 95         |
| 15.4 CRC Module . . . . .                         | 96         |
| 15.5 Watchdog Timer . . . . .                     | 96         |
| 15.6 Register Description . . . . .               | 98         |
| 15.6.1 SYSCLKCR Register . . . . .                | 98         |
| 15.6.2 CLKDIVCR Register . . . . .                | 99         |
| 15.6.3 BLOCKPWR Register . . . . .                | 99         |
| 15.6.4 CRCDATA Register . . . . .                 | 100        |
| 15.6.5 CRCSTATE Register . . . . .                | 100        |
| 15.6.6 WDTPASS Register . . . . .                 | 101        |
| 15.6.7 WDTCR Register . . . . .                   | 101        |
| 15.6.8 WDTSR Register . . . . .                   | 102        |
| 15.6.9 WDTVAL Register . . . . .                  | 103        |
| 15.6.10 DC00BIAS Register . . . . .               | 104        |
| 15.6.11 DC01BIAS Register . . . . .               | 104        |
| <b>16 NPU Peripheral</b>                          | <b>105</b> |
| 16.1 Register Description . . . . .               | 108        |
| 16.1.1 NPUCR Register . . . . .                   | 108        |
| 16.1.2 NPUIVSAR Register . . . . .                | 109        |
| 16.1.3 NPUWVSAR Register . . . . .                | 110        |
| 16.1.4 NPUOVSAR Register . . . . .                | 110        |
| 16.1.5 NPUSR Register . . . . .                   | 111        |
| 16.1.6 NPUCFG1 Register . . . . .                 | 112        |
| 16.1.7 NPUCFG2 Register . . . . .                 | 112        |
| <b>17 PWRCTRL Peripheral</b>                      | <b>114</b> |
| 17.1 Usage . . . . .                              | 114        |
| 17.2 Register Description . . . . .               | 115        |
| 17.2.1 PWRCR Register . . . . .                   | 115        |
| 17.2.2 PWRSR Register . . . . .                   | 115        |
| <b>18 I2Cx Peripheral</b>                         | <b>117</b> |
| 18.1 Register Description . . . . .               | 118        |
| 18.1.1 I2CCR Register . . . . .                   | 118        |
| 18.1.2 I2CFCR Register . . . . .                  | 120        |
| 18.1.3 I2CSR Register . . . . .                   | 121        |
| 18.1.4 I2CMTX Register . . . . .                  | 124        |
| 18.1.5 I2CMRX Register . . . . .                  | 124        |
| 18.1.6 I2CSTX Register . . . . .                  | 124        |
| 18.1.7 I2CSRX Register . . . . .                  | 124        |
| 18.1.8 I2CAR Register . . . . .                   | 125        |
| 18.1.9 I2CAMR Register . . . . .                  | 125        |

|                                 |            |
|---------------------------------|------------|
| <b>19 CLINT Peripheral</b>      | <b>126</b> |
| 19.1 Software Interrupts (IPIs) | 126        |
| 19.2 Timer Interrupts           | 126        |
| 19.3 Addressing Note            | 126        |
| 19.4 Register Description       | 127        |
| 19.4.1 MSIP0 Register           | 127        |
| 19.4.2 MSIP1 Register           | 127        |
| 19.4.3 MSIP2 Register           | 128        |
| 19.4.4 MSIP3 Register           | 129        |
| 19.4.5 MTIMEL Register          | 129        |
| 19.4.6 MTIMEH Register          | 130        |
| 19.4.7 MTIMECMP0L Register      | 130        |
| 19.4.8 MTIMECMP0H Register      | 131        |
| 19.4.9 MTIMECMP1L Register      | 132        |
| 19.4.10 MTIMECMP1H Register     | 132        |
| 19.4.11 MTIMECMP2L Register     | 133        |
| 19.4.12 MTIMECMP2H Register     | 134        |
| 19.4.13 MTIMECMP3L Register     | 134        |
| 19.4.14 MTIMECMP3H Register     | 135        |
| <b>20 MUTEX Peripheral</b>      | <b>136</b> |
| 20.1 Usage                      | 136        |
| 20.2 Rules                      | 136        |
| 20.3 Register Description       | 137        |
| 20.3.1 MUTEX0 Register          | 137        |
| 20.3.2 MUTEX1 Register          | 137        |
| 20.3.3 MUTEX2 Register          | 138        |
| 20.3.4 MUTEX3 Register          | 139        |
| 20.3.5 MUTEX4 Register          | 139        |
| 20.3.6 MUTEX5 Register          | 140        |
| 20.3.7 MUTEX6 Register          | 141        |
| 20.3.8 MUTEX7 Register          | 142        |
| 20.3.9 MUTEX8 Register          | 142        |
| 20.3.10 MUTEX9 Register         | 143        |
| 20.3.11 MUTEX10 Register        | 144        |
| 20.3.12 MUTEX11 Register        | 145        |
| 20.3.13 MUTEX12 Register        | 145        |
| 20.3.14 MUTEX13 Register        | 146        |
| 20.3.15 MUTEX14 Register        | 147        |
| 20.3.16 MUTEX15 Register        | 148        |
| <b>21 IRQROUTER Peripheral</b>  | <b>149</b> |
| 21.1 Delivery Model             | 149        |
| 21.2 Register Description       | 151        |
| 21.2.1 H0ENL Register           | 151        |
| 21.2.2 H0ENM Register           | 151        |
| 21.2.3 H0ENU Register           | 152        |
| 21.2.4 H0ENX Register           | 152        |

|           |                                           |            |
|-----------|-------------------------------------------|------------|
| 21.2.5    | H1ENL Register . . . . .                  | 153        |
| 21.2.6    | H1ENM Register . . . . .                  | 154        |
| 21.2.7    | H1ENU Register . . . . .                  | 154        |
| 21.2.8    | H1ENX Register . . . . .                  | 155        |
| 21.2.9    | H2ENL Register . . . . .                  | 155        |
| 21.2.10   | H2ENM Register . . . . .                  | 156        |
| 21.2.11   | H2ENU Register . . . . .                  | 156        |
| 21.2.12   | H2ENX Register . . . . .                  | 157        |
| 21.2.13   | H3ENL Register . . . . .                  | 158        |
| 21.2.14   | H3ENM Register . . . . .                  | 158        |
| 21.2.15   | H3ENU Register . . . . .                  | 159        |
| 21.2.16   | H3ENX Register . . . . .                  | 159        |
| 21.2.17   | CLAIM Register . . . . .                  | 160        |
| 21.2.18   | PENDL Register . . . . .                  | 161        |
| 21.2.19   | PENDM Register . . . . .                  | 161        |
| 21.2.20   | PENDU Register . . . . .                  | 162        |
| 21.2.21   | PENDX Register . . . . .                  | 162        |
| 21.2.22   | INSVCL Register . . . . .                 | 163        |
| 21.2.23   | INSVCM Register . . . . .                 | 164        |
| 21.2.24   | INSVCU Register . . . . .                 | 164        |
| 21.2.25   | INSVCX Register . . . . .                 | 165        |
| <b>22</b> | <b>Register and Peripheral Memory Map</b> | <b>166</b> |
| 22.1      | GPIO0 Peripheral . . . . .                | 166        |
| 22.2      | GPIO1 Peripheral . . . . .                | 166        |
| 22.3      | SPI0 Peripheral . . . . .                 | 166        |
| 22.4      | SPI1 Peripheral . . . . .                 | 167        |
| 22.5      | UART0 Peripheral . . . . .                | 167        |
| 22.6      | UART1 Peripheral . . . . .                | 167        |
| 22.7      | TIMER0 Peripheral . . . . .               | 167        |
| 22.8      | TIMER1 Peripheral . . . . .               | 168        |
| 22.9      | GPIO2 Peripheral . . . . .                | 168        |
| 22.10     | SYSTEM Peripheral . . . . .               | 168        |
| 22.11     | NPU Peripheral . . . . .                  | 169        |
| 22.12     | PWRCTRL Peripheral . . . . .              | 169        |
| 22.13     | GPIO3 Peripheral . . . . .                | 169        |
| 22.14     | I2C0 Peripheral . . . . .                 | 169        |
| 22.15     | I2C1 Peripheral . . . . .                 | 170        |
| 22.16     | CLINT Peripheral . . . . .                | 170        |
| 22.17     | MUTEX Peripheral . . . . .                | 170        |
| 22.18     | GPIO4 Peripheral . . . . .                | 171        |
| 22.19     | GPIO5 Peripheral . . . . .                | 171        |
| 22.20     | IRQROUTER Peripheral . . . . .            | 172        |
| <b>23</b> | <b>Errata</b>                             | <b>173</b> |

# 1 System Configuration

The microcontroller (MCU) described in this document is a four-hart (four-core) multiprocessor built from four identical VestaRV CPU cores, a custom 32-bit RISC-V processor. The microcontroller is composed of the four VestaRV harts, a ROM that contains boot software and a Forth interpreter, private RAM memory cells per hart, an arbitrated shared memory window for inter-hart communication, and several peripherals. The multi-core architecture is described in Section 4. The modular architecture enables customization of the peripheral set, memory configuration, and system features to match specific application requirements. Figure 1 shows the top-level organization of this configuration.

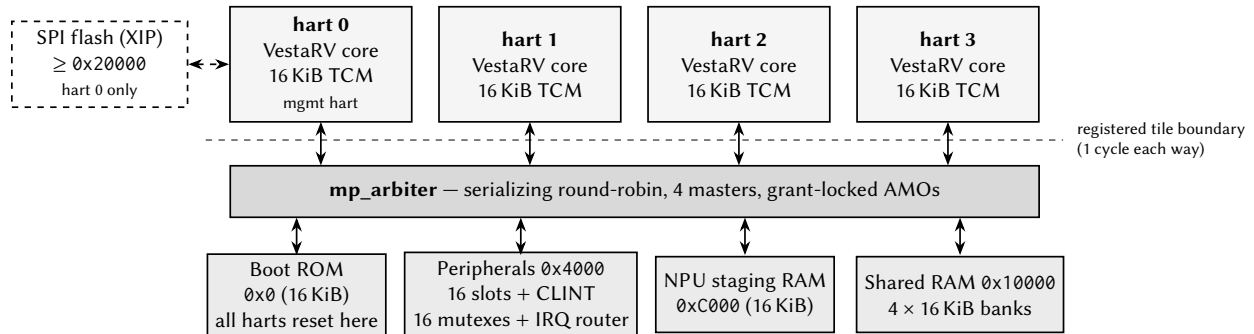


Figure 1: Castalia top-level block diagram (generated from this configuration: 4 harts, 64 KiB shared RAM). Every hart tile couples an unmodified VestaRV core with its private TCM; everything else is reached through the serializing round-robin arbiter.

The following list details the MCU configuration:

- Chip name: Castalia
- Compatible with RISC-V compiler RV32I[M][A][C][Zba][Zbb][Zbs][Zbc]
- 32-bit memory bus
- Contains 32 general purpose dual-port CPU registers
- Supports the RISC-V compressed instruction set and allows the use of the [C] compiler extension. Allowing the compiler to use the [C] extension reduces executable code size, but also takes the processor longer to fetch 32-bit long instructions that are aligned on a 2 but not 4 byte boundary from memory.
- Includes a multi-stage bit shifter in the ALU to save power and chip area for bit shift operations at the expense of speed
- Includes a fast 32-bit signed integer hardware multiplier in the ALU and allows the use of the [M] compiler extension
- Includes a medium speed 32-bit signed integer hardware divider and remainder calculator in the ALU
- Supports the RISC-V atomic instruction set (LR/SC and AMOs) and allows the use of the [A] compiler extension; on multi-hart configurations atomics are globally coherent across the shared window



- Supports the RISC-V bit-manipulation extensions Zba (address generation), Zbb (basic bit manipulation), Zbs (single-bit operations), and Zbc (carry-less multiplication)
- The read-only misa CSR (0x301) advertises the enabled instruction-set extensions at run time
- Supports maskable hardware interrupts generated by peripheral signals
- 4× VestaRV harts (cores), each with private RAM; hart ID readable via the `mhartid` CSR
- An arbitrated shared memory window with 80 KiB of shared RAM, a CLINT (inter-processor and per-hart timer interrupts), 16 hardware mutexes, and a per-hart peripheral interrupt router
- 6× 8-pin general purpose I/O (GPIO) ports with edge-triggered interrupts and per-pin multiplexed alternate functions (GPIO + up to 8 alternate functions per pin)
- 2× SPI interfaces (SPI0 provides memory-mapped access to external flash memory)
- 2× UART interfaces with hardware parity support
- 2× 32-bit timers with pulse-width modulation outputs and input capture units
- A CRC calculation engine (CRC16\_CDMA2000)
- 2× internal digitally controllable oscillators
- 2× external clock pins for clock generation and accurate timing
- A windowed watchdog timer
- A neural processing unit (NPU) co-processor for hardware acceleration of machine learning tasks
- Per-tile MTCMOS power gating with hardware gate/wake sequencing (cold-boot wake)
- 2× I<sup>2</sup>C interfaces (both master and slave mode)
- Claim/complete peripheral interrupt delivery (PLIC-style) with any-vector-to-any-hart routing

## 1.1 Chip Configuration

This document, the memory-map headers, the linker scripts, and the drop-in RTL are all generated from one chip configuration by `make chip`. The configuration is a small JSON document (every key optional; missing keys keep the Castalia defaults) passed as `make chip CONFIG=config.json`; an interactive front-end for producing it ships with the generator (`docs/chip_configurator.html`). Table 1 lists every configuration knob together with the value used to build *this* document, and Table 2 lists the geometry derived from those knobs. The resolved configuration is also written to `config/ChipConfig.resolved.json` at every build, and `make verify [CONFIG=config.json]` *proves* a configuration: it stages the generated RTL into a behavioral simulation environment and runs the multi-core boot/ISA smoke suite against it. The core peripheral set and the pad ring are not configuration knobs — they are fixed template content, and the pad ring (Section 2) is derived from the generator’s package model — but the second-instance peripherals (UART1, SPI1, TIMER1, I2C1) and the NPU are individually droppable through the `peripherals` section: a dropped instance’s register window reads zero, its interrupt vectors are reserved (the numbering is frozen), and its pins revert to plain GPIO.

Table 1: Chip configuration knobs (make chip CONFIG=config.json) and the values of this build

| Configuration key        | This build     | Meaning / valid values                                                                       |
|--------------------------|----------------|----------------------------------------------------------------------------------------------|
| chipName                 | Castalia       | non-empty string — renames the chip in the TRM/headers (docs-only; CHIP_NAME env still wins) |
| numHarts                 | 4              | int 1..32 — hart/tile count (4 = Castalia golden master, 18 = Argus sim-proven)              |
| numMutexes               | 16             | int 1..1024 — HW mutex bank size (16 = Castalia, 32 = Argus)                                 |
| registerFileDualPort     | true           | bool — dual-port register file (ASIC) vs single-port (Spartan-6 FPGA)                        |
| isa.mul                  | true           | bool — M multiply                                                                            |
| isa.fastMul              | true           | bool — docs-only on vesta (multiplier is already single-cycle)                               |
| isa.div                  | true           | bool — M divide                                                                              |
| isa atomics              | true           | bool — A extension (LR/SC + AMO)                                                             |
| isa.compressed           | true           | bool — C extension                                                                           |
| isa.bitmanip             | true           | bool — Zba/Zbb/Zbs                                                                           |
| isa.counters             | false          | bool — Zicntr mcycle/minstret                                                                |
| isa.counters64           | false          | bool — 64-bit counter high halves (needs isa.counters)                                       |
| memory.romSize           | 16384 (16 KiB) | int bytes, 1 KiB multiple $\leq 0x4000$ (region 0x0-0x3FFF)                                  |
| memory.tcmSizePerHart    | 16384 (16 KiB) | int bytes, 1 KiB multiple $\leq 0x4000$ (region 0x8000-0xBFFF)                               |
| memory.sharedBulkRamSize | 65536 (64 KiB) | int bytes, multiple of 0x4000 (one sram1p16k bank) from 0x10000                              |
| memory.npuStagingRamSize | 16384 (16 KiB) | int bytes, 1 KiB multiple $\leq 0x4000$ (region 0xC000-0xFFFF)                               |
| peripherals.npu          | true           | bool — False drops the NPU entirely (slot 10 + the 0xC000 staging window read zero)          |

Table 2: Derived geometry of this configuration (computed, not configurable)

| Derived value               | This build           | Notes                                                                                                       |
|-----------------------------|----------------------|-------------------------------------------------------------------------------------------------------------|
| ISA string (march)          | rv32imac_zba_zbb_zbs | Advertised in the read-only misa CSR                                                                        |
| Shared-window address width | 15 bits (words)      | Arbiter/tile word-address width; the window is $0x0 - 2^{w+2} - 1$                                          |
| Shared RAM banks            | $4 \times 16$ KiB    | One SRAM macro per bank behind the arbiter                                                                  |
| Extended flash base         | 0x20000              | First address decoded to the SPI-flash XIP path (hart 0 only); strictly the complement of the shared window |
| Interrupt vectors           | 121                  | Vectors 83/84 are the CLINT software/timer interrupts                                                       |

*Table 2 continued on next page*

*Table 2 continued from previous page*

| <b>Derived value</b>     | <b>This build</b>     | <b>Notes</b>                                                              |
|--------------------------|-----------------------|---------------------------------------------------------------------------|
| CLINT MTIME              | 0x5010                | Layout is hart-count-derived: MSIPx at 0x5000 + 4h, MTIMECMPx from 0x5020 |
| Boot-loader mailbox rows | 0x10500 + 0x10*hartid | Tile-loading rows {SRC, LEN, ENTRY} consumed by the boot ROM              |
| Stack pointer at reset   | 0xC000                | Top of each hart's private TCM, growing down                              |
| Peripheral count         | 20                    | Instantiated peripherals (including CLINT/MUTEX/IRQROUTER)                |

## 2 Packaging and Pins Configuration

**Preliminary:** the package and pinout in this section are inherited from the single-core Myshkin MCU and have not yet been finalized for Castalia. Pin assignments, package type, and pin count are subject to change.

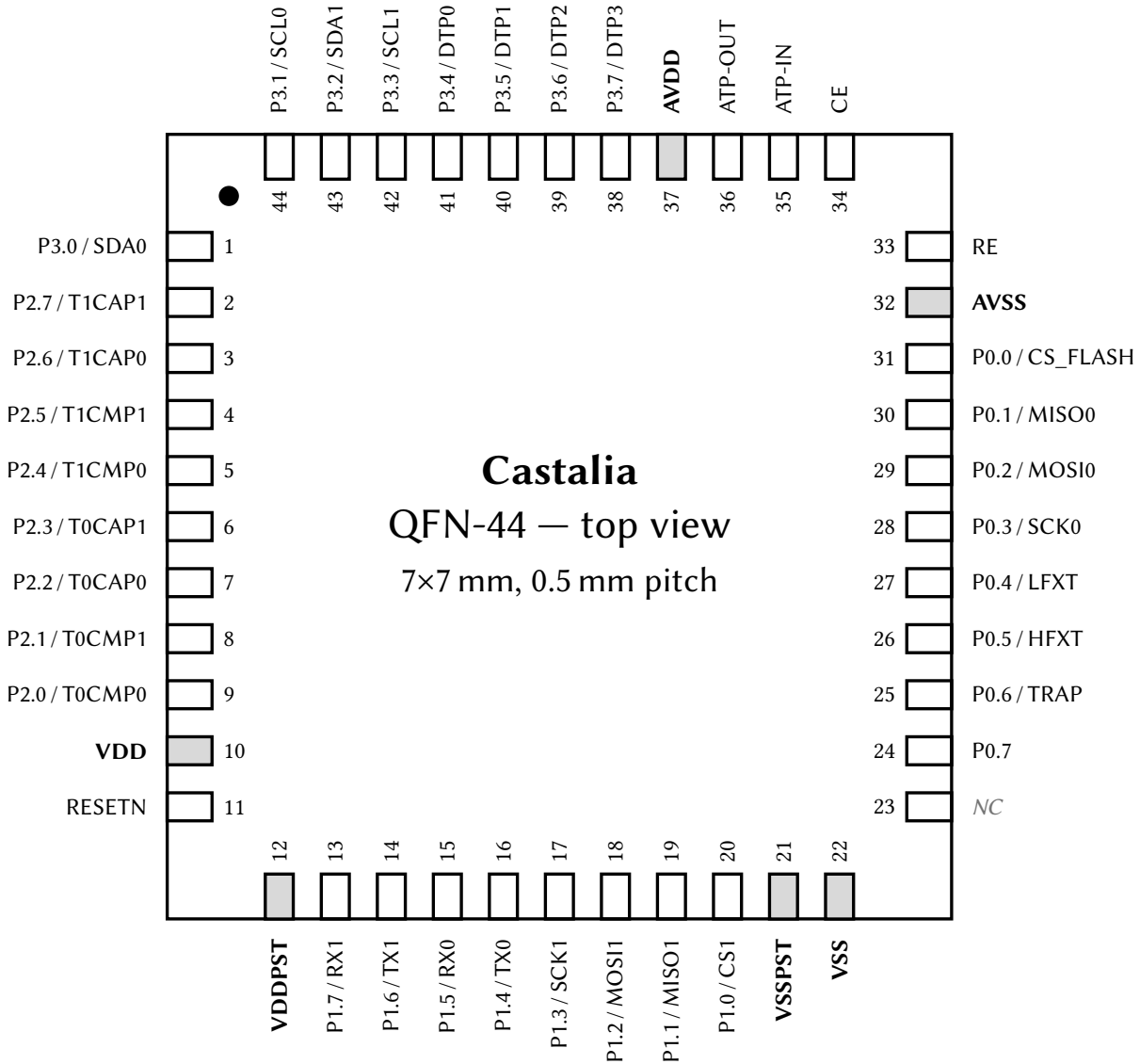


Figure 2: QFN-44 pinout, top view (7 × 7 mm, 0.5 mm pitch). Each pin is labeled with its pad name and, for GPIO pins, its AF0 (reset-selected) alternate function; power-rail pins are shaded and bold. The full per-pin alternate-function matrix is in Table 5.

The chip package is a QFN-44 with dimensions 7 × 7 mm and pitch 0.5 mm. The package footprint (as seen from the top down) is shown in Figure 2. The pins on the package are listed in Table 3. The pin ordering, side assignment, and power domains in both the figure and the table are derived from the generator’s package model and are also exported machine-readably to config/PadRing.json at every make chip build.

Table 3: Package Pins

| #  | Pin     | I/O          | Power Domain         | Power Pins    |
|----|---------|--------------|----------------------|---------------|
| 1  | P3.0    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 2  | P2.7    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 3  | P2.6    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 4  | P2.5    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 5  | P2.4    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 6  | P2.3    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 7  | P2.2    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 8  | P2.1    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 9  | P2.0    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 10 | VDD     | Power Input  | Digital Core (1.0 V) | VDD/VSS       |
| 11 | RESETN  | Input        | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 12 | VDDPST  | Power Input  | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 13 | P1.7    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 14 | P1.6    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 15 | P1.5    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 16 | P1.4    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 17 | P1.3    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 18 | P1.2    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 19 | P1.1    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 20 | P1.0    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 21 | VSSPST  | Power Input  | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 22 | VSS     | Power Input  | Digital Core (1.0 V) | VDD/VSS       |
| 23 | NC      | –            | –                    | –             |
| 24 | P0.7    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 25 | P0.6    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 26 | P0.5    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 27 | P0.4    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 28 | P0.3    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 29 | P0.2    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 30 | P0.1    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 31 | P0.0    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 32 | AVSS    | Power Input  | Analog (3.3 V)       | AVDD/AVSS     |
| 33 | RE      | Input/Output | Analog (3.3 V)       | AVDD/AVSS     |
| 34 | CE      | Input/Output | Analog (3.3 V)       | AVDD/AVSS     |
| 35 | ATP-IN  | Input        | Analog (3.3 V)       | AVDD/AVSS     |
| 36 | ATP-OUT | Output       | Analog (3.3 V)       | AVDD/AVSS     |
| 37 | AVDD    | Power Input  | Analog (3.3 V)       | AVDD/AVSS     |
| 38 | P3.7    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 39 | P3.6    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 40 | P3.5    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 41 | P3.4    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 42 | P3.3    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 43 | P3.2    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |
| 44 | P3.1    | Input/Output | Digital I/O (3.3 V)  | VDDPST/VSSPST |

Many of the digital I/O pins are GPIO pins. These pins can be in either primary/GPIO mode, or secondary/peripheral mode (see Section 11 on the GPIO peripherals for more details). A list of all the GPIO pins, along with their primary and secondary functions and reset values, is shown in Table 4.

Table 4: GPIO Pins

| Pin  | Primary Function | Alternate Function (AF0) | Additional Alternate Functions (PxAFS)                                          |
|------|------------------|--------------------------|---------------------------------------------------------------------------------|
| P0.0 | GPIO0            | CS_FLASH                 | AF1: TX0, AF2: TX1, AF3: T0CMP0, AF4: T0CMP1, AF5: T1CMP0, AF6: T1CMP1, AF7: S  |
| P0.1 | GPIO1            | MISO0                    | AF1: TX1, AF2: T0CMP0, AF3: T0CMP1, AF4: T1CMP0, AF5: T1CMP1, AF6: SCK1, AF7: S |
| P0.2 | GPIO2            | MOSI0                    | AF1: T0CMP0, AF2: T0CMP1, AF3: T1CMP0, AF4: T1CMP1, AF5: SCK1, AF6: MOSI1, AF   |
| P0.3 | GPIO3            | SCK0                     | AF1: T0CMP1, AF2: T1CMP0, AF3: T1CMP1, AF4: SCK1, AF5: MOSI1, AF6: TX0, AF7: T  |
| P0.4 | GPIO4            | LFXT                     | AF1: T1CMP0, AF2: T1CMP1, AF3: SCK1, AF4: MOSI1, AF5: TX0, AF6: TX1, AF7: T0CM  |
| P0.5 | GPIO5            | HFXT                     | AF1: T1CMP1, AF2: SCK1, AF3: MOSI1, AF4: TX0, AF5: TX1, AF6: T0CMP0, AF7: T0CM  |
| P0.6 | GPIO6            | TRAP                     | AF1: SCK1, AF2: MOSI1, AF3: TX0, AF4: TX1, AF5: T0CMP0, AF6: T0CMP1, AF7: T1CM  |
| P0.7 | BOOT             | –                        | AF1: MOSI1, AF2: TX0, AF3: TX1, AF4: T0CMP0, AF5: T0CMP1, AF6: T1CMP0, AF7: T   |
| P1.0 | GPIO8            | CS1                      | AF1: T0CMP0, AF2: TX1, AF3: T0CMP1, AF4: T1CMP0, AF5: T1CMP1, AF6: SCK1, AF7: S |
| P1.1 | GPIO9            | MISO1                    | AF1: T0CMP1, AF2: T0CMP0, AF3: T1CMP0, AF4: T1CMP1, AF5: SCK1, AF6: MOSI1, AF   |
| P1.2 | GPIO10           | MOSI1                    | AF1: T1CMP0, AF2: T1CMP1, AF3: SCK1, AF4: TX0, AF5: TX1, AF6: T0CMP0, AF7: T0   |
| P1.3 | GPIO11           | SCK1                     | AF1: T1CMP1, AF2: MOSI1, AF3: TX0, AF4: TX1, AF5: T0CMP0, AF6: T0CMP1, AF7: T   |
| P1.4 | GPIO12           | TX0                      | AF1: SDA1, AF2: SCK1, AF3: MOSI1, AF4: TX1, AF5: T0CMP0, AF6: T0CMP1, AF7: T1C  |
| P1.5 | GPIO13           | RX0                      | AF1: SCL1, AF2: T1CMP1, AF3: SCK1, AF4: MOSI1, AF5: TX0, AF6: TX1, AF7: T0CMP   |
| P1.6 | GPIO14           | TX1                      | AF1: SDA0, AF2: TX0, AF3: T0CMP0, AF4: T0CMP1, AF5: T1CMP0, AF6: T1CMP1, AF7: S |
| P1.7 | GPIO15           | RX1                      | AF1: SCL0, AF2: MOSI1, AF3: TX0, AF4: TX1, AF5: T0CMP0, AF6: T0CMP1, AF7: T1CM  |
| P2.0 | GPIO16           | T0CMP0                   | AF1: TX1, AF2: SCK1, AF3: MOSI1, AF4: TX0, AF5: T0CMP1, AF6: T1CMP0, AF7: T1CM  |
| P2.1 | GPIO17           | T0CMP1                   | AF1: RX1, AF2: T1CMP0, AF3: T1CMP1, AF4: SCK1, AF5: MOSI1, AF6: TX0, AF7: TX1   |
| P2.2 | GPIO18           | T0CAP0                   | AF1: SDA1, AF2: T0CMP0, AF3: T0CMP1, AF4: T1CMP0, AF5: T1CMP1, AF6: SCK1, AF    |
| P2.3 | GPIO19           | T0CAP1                   | AF1: SCL1, AF2: T0CMP1, AF3: T1CMP0, AF4: T1CMP1, AF5: SCK1, AF6: MOSI1, AF7: S |
| P2.4 | GPIO20           | T1CMP0                   | AF1: TX0, AF2: T0CMP1, AF3: T1CMP1, AF4: SCK1, AF5: MOSI1, AF6: TX1, AF7: T0CM  |
| P2.5 | GPIO21           | T1CMP1                   | AF1: RX0, AF2: TX0, AF3: TX1, AF4: T0CMP0, AF5: T0CMP1, AF6: T1CMP0, AF7: SCK   |
| P2.6 | GPIO22           | T1CAP0                   | AF1: SDA0, AF2: SCK1, AF3: MOSI1, AF4: TX0, AF5: TX1, AF6: T0CMP0, AF7: T0CMP   |
| P2.7 | GPIO23           | T1CAP1                   | AF1: SCL0, AF2: MOSI1, AF3: TX0, AF4: TX1, AF5: T0CMP0, AF6: T0CMP1, AF7: T1CM  |
| P3.0 | GPIO24           | SDA0                     | AF1: T0CAP0, AF2: TX0, AF3: TX1, AF4: T0CMP0, AF5: T0CMP1, AF6: T1CMP0, AF7: T  |
| P3.1 | GPIO25           | SCL0                     | AF1: T0CAP1, AF2: TX1, AF3: T0CMP0, AF4: T0CMP1, AF5: T1CMP0, AF6: T1CMP1, AF   |
| P3.2 | GPIO26           | SDA1                     | AF1: T1CAP0, AF2: T0CMP0, AF3: T0CMP1, AF4: T1CMP0, AF5: T1CMP1, AF6: SCK1, A   |
| P3.3 | GPIO27           | SCL1                     | AF1: T1CAP1, AF2: T0CMP1, AF3: T1CMP0, AF4: T1CMP1, AF5: SCK1, AF6: MOSI1, AF   |
| P3.4 | GPIO28           | DTP0                     | AF1: T0CMP0, AF2: TX0, AF3: TX1, AF4: T0CMP1, AF5: T1CMP0, AF6: T1CMP1, AF7: S  |
| P3.5 | GPIO29           | DTP1                     | AF1: T0CMP1, AF2: RX0, AF3: T0CMP0, AF4: T1CMP0, AF5: T1CMP1, AF6: SCK1, AF7: S |
| P3.6 | GPIO30           | DTP2                     | AF1: T1CMP0, AF2: T0CMP0, AF3: T0CMP1, AF4: T1CMP1, AF5: SCK1, AF6: MOSI1, AF   |
| P3.7 | GPIO31           | DTP3                     | AF1: T1CMP1, AF2: T0CMP1, AF3: T1CMP0, AF4: SCK1, AF5: MOSI1, AF6: TX0, AF7: T  |
| P4.0 | GPIO32           | –                        | –                                                                               |
| P4.1 | GPIO33           | –                        | –                                                                               |
| P4.2 | GPIO34           | –                        | –                                                                               |
| P4.3 | GPIO35           | –                        | –                                                                               |
| P4.4 | GPIO36           | –                        | –                                                                               |
| P4.5 | GPIO37           | –                        | –                                                                               |
| P4.6 | GPIO38           | –                        | –                                                                               |

*Table 4 continued on next page*

Table 4 continued from previous page

| Pin  | Primary Function | Alternate Function (AF0) | Additional Alternate Functions (PxAFS) |
|------|------------------|--------------------------|----------------------------------------|
| P4.7 | GPI039           | –                        | –                                      |
| P5.0 | GPI040           | –                        | –                                      |
| P5.1 | GPI041           | –                        | –                                      |
| P5.2 | GPI042           | –                        | –                                      |
| P5.3 | GPI043           | –                        | –                                      |
| P5.4 | GPI044           | –                        | –                                      |
| P5.5 | GPI045           | –                        | –                                      |
| P5.6 | GPI046           | –                        | –                                      |
| P5.7 | GPI047           | –                        | –                                      |

When a pin is in alternate function (peripheral) mode (its PxSEL bit = 1), its 3-bit PxAFS field selects which of up to eight alternate functions (AF0–AF7) drives the pad. Table 5 lists, for every GPIO pin, the exact function selected by each PxAFS value. Plane 0 is the pin's AF0 (legacy secondary) function and is the reset selection; planes marked **Hi-Z** have no function assigned for that pin and configure the pad as a high-impedance input. The superscript on each function denotes its direction (<sup>in</sup> input, <sup>out</sup> output, <sup>io</sup> bidirectional), and a <sup>†</sup> marks each pin's reset plane (its PxAFS reset value). While a pin is in GPIO (primary) mode the selected plane has no effect on the pad, but it still routes relocatable peripheral *inputs* (see the GPIOx chapter, Section 11).

Table 5: GPIO Alternate Function Selection (PxAFS)

| Pin  | PxAFS field value (selected AF plane) |                       |                       |                       |                       |                       |                       |                       |
|------|---------------------------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|
|      | 0 (AF0)                               | 1                     | 2                     | 3                     | 4                     | 5                     | 6                     | 7                     |
| P0.0 | CS_FLASH <sup>out†</sup>              | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   |
| P0.1 | MISO0 <sup>io†</sup>                  | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  |
| P0.2 | MOSI0 <sup>out†</sup>                 | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    |
| P0.3 | SCK0 <sup>out†</sup>                  | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    |
| P0.4 | LFX1 <sup>io†</sup>                   | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> |
| P0.5 | HFX1 <sup>io†</sup>                   | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> |
| P0.6 | TRAP <sup>out†</sup>                  | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> |
| P0.7 | Hi-Z <sup>†</sup>                     | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> |
| P1.0 | CS1 <sup>in†</sup>                    | T0CMP0 <sup>out</sup> | TX1 <sup>out</sup>    | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  |
| P1.1 | MISO1 <sup>io†</sup>                  | T0CMP1 <sup>out</sup> | T0CMP0 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    |
| P1.2 | MOSI1 <sup>io†</sup>                  | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> |
| P1.3 | SCK1 <sup>io†</sup>                   | T1CMP1 <sup>out</sup> | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> |
| P1.4 | TX0 <sup>out†</sup>                   | SDA1 <sup>io</sup>    | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> |
| P1.5 | RX0 <sup>io†</sup>                    | SCL1 <sup>io</sup>    | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> |
| P1.6 | TX1 <sup>out†</sup>                   | SDA0 <sup>io</sup>    | TX0 <sup>out</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   |
| P1.7 | RX1 <sup>io†</sup>                    | SCL0 <sup>io</sup>    | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> |
| P2.0 | T0CMP0 <sup>out†</sup>                | TX1 <sup>out</sup>    | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> |
| P2.1 | T0CMP1 <sup>out†</sup>                | RX1 <sup>io</sup>     | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    |
| P2.2 | T0CAP0 <sup>in†</sup>                 | SDA1 <sup>io</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  |
| P2.3 | T0CAP1 <sup>in†</sup>                 | SCL1 <sup>io</sup>    | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    |
| P2.4 | T1CMP0 <sup>out†</sup>                | TX0 <sup>out</sup>    | T0CMP1 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> |
| P2.5 | T1CMP1 <sup>out†</sup>                | RX0 <sup>io</sup>     | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | SCK1 <sup>out</sup>   |
| P2.6 | T1CAP0 <sup>in†</sup>                 | SDA0 <sup>io</sup>    | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> |
| P2.7 | T1CAP1 <sup>in†</sup>                 | SCL0 <sup>io</sup>    | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> |
| P3.0 | SDA0 <sup>io†</sup>                   | T0CAP0 <sup>in</sup>  | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> |

Table 5 continued on next page

Table 5 continued from previous page

| Pin  | PxAFS field value (selected AF plane) |                       |                       |                       |                       |                       |                       |                      |
|------|---------------------------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|----------------------|
|      | 0 (AF0)                               | 1                     | 2                     | 3                     | 4                     | 5                     | 6                     | 7                    |
| P3.1 | SCL0 <sup>io†</sup>                   | T0CAP1 <sup>in</sup>  | TX1 <sup>out</sup>    | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>  |
| P3.2 | SDA1 <sup>io†</sup>                   | T1CAP0 <sup>in</sup>  | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup> |
| P3.3 | SCL1 <sup>io†</sup>                   | T1CAP1 <sup>in</sup>  | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>   |
| P3.4 | DTP0 <sup>io†</sup>                   | T0CMP0 <sup>out</sup> | TX0 <sup>out</sup>    | TX1 <sup>out</sup>    | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>  |
| P3.5 | DTP1 <sup>io†</sup>                   | T0CMP1 <sup>out</sup> | RX0 <sup>io</sup>     | T0CMP0 <sup>out</sup> | T1CMP0 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup> |
| P3.6 | DTP2 <sup>io†</sup>                   | T1CMP0 <sup>out</sup> | T0CMP0 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP1 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | MISO1 <sup>io</sup>  |
| P3.7 | DTP3 <sup>io†</sup>                   | T1CMP1 <sup>out</sup> | T0CMP1 <sup>out</sup> | T1CMP0 <sup>out</sup> | SCK1 <sup>out</sup>   | MOSI1 <sup>out</sup>  | TX0 <sup>out</sup>    | TX1 <sup>out</sup>   |
| P4.0 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |
| P4.1 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |
| P4.2 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |
| P4.3 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |
| P4.4 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |
| P4.5 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |
| P4.6 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |
| P4.7 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |
| P5.0 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |
| P5.1 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |
| P5.2 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |
| P5.3 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |
| P5.4 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |
| P5.5 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |
| P5.6 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |
| P5.7 | Hi-Z <sup>†</sup>                     | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                  | Hi-Z                 |



### 3 Address Space

|           |                                                  |                                          |
|-----------|--------------------------------------------------|------------------------------------------|
| 0x00000   | ROM<br>Size = 16 KiB                             | RAM (Total size = 16 KiB)                |
| 0x03FFF   |                                                  |                                          |
| 0x04000   | Peripherals<br>Size = 4 KiB                      |                                          |
| 0x04FFF   |                                                  |                                          |
| 0x05000   | <i>Unused</i>                                    | Shared window<br>(all harts, arbitrated) |
| 0x07FFF   |                                                  |                                          |
| 0x08000   | SRAM02<br>Size = 16 KiB                          |                                          |
| 0x0BFFF   |                                                  |                                          |
| 0x0C000   | <i>Reserved</i>                                  |                                          |
| 0x04FFF   |                                                  |                                          |
| 0x05000   | CLINT<br>Size = 4 KiB                            |                                          |
| 0x05FFF   |                                                  |                                          |
| 0x06000   | Mutex bank<br>Size = 4 KiB                       |                                          |
| 0x06FFF   |                                                  |                                          |
| 0x07000   | IRQ router<br>Size = 4 KiB                       |                                          |
| 0x07FFF   |                                                  |                                          |
| 0x08000   | <i>Reserved</i>                                  |                                          |
| 0x0BFFF   |                                                  |                                          |
| 0x0C000   | NPU staging RAM<br>Size = 16 KiB                 |                                          |
| 0x0FFFF   |                                                  |                                          |
| 0x10000   | Shared RAM<br>Size = 64 KiB                      |                                          |
| 0x1FFFF   |                                                  |                                          |
| 0x0020000 | <i>Unused</i>                                    |                                          |
| 0x0FFFFFF |                                                  |                                          |
| 0x1000000 | SPI Flash Native<br>Memory Access<br>(read only) |                                          |
| 0x1FFFFFF |                                                  |                                          |

The 32-bit memory bus on the MCU is in the von Neumann architecture. All addressable memory elements (i.e. the ROM, RAM, and memory mapped registers) are all located on a single bus. A pointer can access any of these locations in memory.

The ROM cell contains boot code, which is read-only. Each SRAM cell can be read or written to, and may contain both program and variable information. The chip does not contain programmable non-volatile memory (e.g. flash memory), so the program must be entirely stored in RAM along with all runtime memory. Each SRAM and ROM cell can be individually powered off or on using the SYSTEM peripheral. When powered off, the cells use less power and cannot be read or written to. SRAM cells that are powered off do not retain their previous data, and the data becomes undefined.

The interrupt vector table (IVT) begins at address 0x8000 and contains jump instructions to the interrupt service routines (ISRs). When an interrupt occurs, the CPU directly vectors to the corresponding entry in the IVT, which then jumps to the appropriate ISR. Each IVT entry is 4 bytes and typically contains a RISC-V jump instruction (e.g., `j handler`). The IVT can be initialized and configured in software, allowing flexible interrupt routing and handler assignment at runtime.

The address space above is the view from hart 0. Harts 1–3 see their own private RAM at the same addresses and share the window at 0x5000–0x1FFFF with all other harts; see Section 4.

## 4 Multi-Core Architecture

Castalia is a four-hart (four-core) multiprocessor built from four identical VestaRV cores. Each hart is a complete, single-issue, in-order RV32IMAC processor with its own private *tightly-coupled memory* (TCM), so the common case — instruction fetch and private data access — never contends with the other harts and runs at full core clock. Everything else in the low address space is *shared*: a single boot ROM, all peripherals, the inter-processor interrupt hardware (CLINT), a hardware mutex bank, an interrupt router, the NPU staging RAM, and 64 KiB of bulk RAM, all reached through one serializing round-robin arbiter. The main features of the multi-core architecture are listed below:

- 4 identical VestaRV harts, each identified by the read-only `mhartid` CSR (address `0xF14`)
- One shared boot ROM at `0x0`: all four harts reset to PC `0x0` and dispatch on `mhartid` (see Section 4.3)
- 16 KiB of private TCM per hart at `0x8000–0xBFFF` (the same local address in every hart), holding each hart’s vector table, code, data, and stack
- All peripherals shared by all four harts at their legacy `0x4000`-page addresses, with the arbiter guaranteeing register-access atomicity
- 64 KiB of shared bulk RAM at `0x10000–0x1FFFF` for locks, mailboxes, staged tile programs, and inter-hart data
- CLINT with per-hart software interrupts (inter-processor interrupts) and a shared 64-bit `mtime` with per-hart compare registers
- 16 single-instruction hardware mutexes (MUTEX bank)
- Claim/complete peripheral interrupt delivery (IRQROUTER): any peripheral interrupt can be routed to any hart over one external-interrupt wire per hart, with exactly-once claiming
- LR/SC and AMO atomic instructions supported to shared RAM, including sub-word shared stores (byte lanes)
- Instruction fetch from shared memory is supported (the boot ROM is fetched this way by construction); parked harts sleep via the custom `sleep` instruction and are woken by a software interrupt — no busy-wait power burn

### 4.1 Harts and Roles

All four harts are instances of the same *hart tile*: an unmodified VestaRV core plus its own private TCM, replicating the proven single-core memory path, with every connection to the rest of the chip registered at the tile boundary. Since every hart sees the same address map — shared boot ROM, shared peripherals, private TCM at the same local address — the four harts are architecturally identical and code is hart-portable by construction.

Hart 0 is additionally the *management hart*. It is the only hart attached to the SPI-flash boot path and the extended flash region ( $\geq 0x20000$ ), and by convention it owns system management: the SYSTEM controller’s clock configuration registers reprogram the master clock that all four harts run on, so only the management hart may touch them, and only after quiescing the other harts (see the SYSTEM chapter). Peripheral interrupts are uniform across harts: every hart, hart 0 included, routes and receives them through its IRQROUTER row and the claim/complete mechanism (see the IRQROUTER chapter).

Software discovers which hart it is running on by reading the `mhartid` CSR:

```
csrr    t0, mhartid      # t0 = 0 .. (number of harts - 1)
```

## 4.2 The Shared Window

Everything below the extended flash region except each hart’s private TCM is behind the arbiter. Every hart sees the same physical devices at the same addresses:

| Address range   | Device                   | Notes                                   |
|-----------------|--------------------------|-----------------------------------------|
| 0x00000–0x03FFF | Boot ROM (16 KiB)        | Shared, read-only; all harts reset here |
| 0x04000–0x04FFF | Peripheral slots         | 16 slots of 256 bytes, legacy numbering |
| 0x05000–0x05FFF | CLINT                    | IPIs (MSIPx), MTIME/MTIMECMPx           |
| 0x06000–0x06FFF | MUTEX bank               | 16 hardware mutexes                     |
| 0x07000–0x07FFF | IRQROUTER                | Per-hart routing rows + CLAIM/COMPLETE  |
| 0x0C000–0x0FFFF | NPU staging RAM (16 KiB) | Vector storage; NPU-owned during a run  |
| 0x10000–0x1FFFF | Shared bulk RAM (64 KiB) | Locks, mailboxes, inter-hart data       |

(0x08000–0x0BFFF is each hart’s **private** TCM and never reaches the arbiter.) The sixteen peripheral slots at 0x4000 keep the original single-core VestaRV slot numbering — GPIO0 at 0x4000, UART0 at 0x4400, SYSTEM at 0x4900, and so on — so single-core driver code runs unchanged; the difference is that every hart can now reach every peripheral.

Instruction fetch from the shared window is supported — the boot flow depends on it, and programs may execute directly from shared RAM. One caveat: compressed (RVC) instructions in shared-window code are not currently validated; code intended to execute from shared memory should be built without the C extension. Code in the private TCM has no such restriction.

### 4.2.1 Arbitration and Stalling

A round-robin arbiter serializes all shared-window transactions. The arbiter grants one hart at a time and holds the grant until that hart’s whole transaction completes, then advances the round-robin pointer, so no hart can be starved. While a hart waits for its grant, its clock is stalled transparently — software never sees a partial transaction and needs no special code for shared accesses. Because a stalled hart contributes nothing until its turn comes, frequently-pollled data structures (spin locks in particular) should use a backoff strategy (see Section 4.4).

An AMO (atomic read-modify-write) instruction holds the arbiter grant across both halves of its read-modify-write pair, so AMOs to shared RAM are atomic with respect to all other harts. A shared-window access costs a few master-clock cycles uncontended (versus a single cycle to the private TCM), which is why per-hart code and stacks belong in the TCM.

## 4.3 Boot and Tile Loading

All four harts come out of reset at PC 0x0 and fetch the shared boot ROM through the arbiter. The ROM immediately dispatches on mhartid; the whole flow is summarized in Figure 3:

**Hart 0** runs the full boot sequence, exactly as on the single-core chip: it configures GPIO0/SPI0, copies the program from SPI flash into the load window 0x8000–0xFFFFC (its TCM plus the NPU staging RAM), and jumps to the program at 0x8200. Before any tile can be released, the boot ROM zeroes the shared-RAM mailbox region 0x10000–0x107FF: shared RAM powers up with *unknown* contents on silicon, and this write-before-read guarantee is what makes “mailbox reads as zero” checks safe.

**Harts 1–3** set up a minimal interrupt context (a stack at the top of their TCM and a software-interrupt vector) and park in low-power sleep. A parked tile wakes only when hart 0 (or any running program)

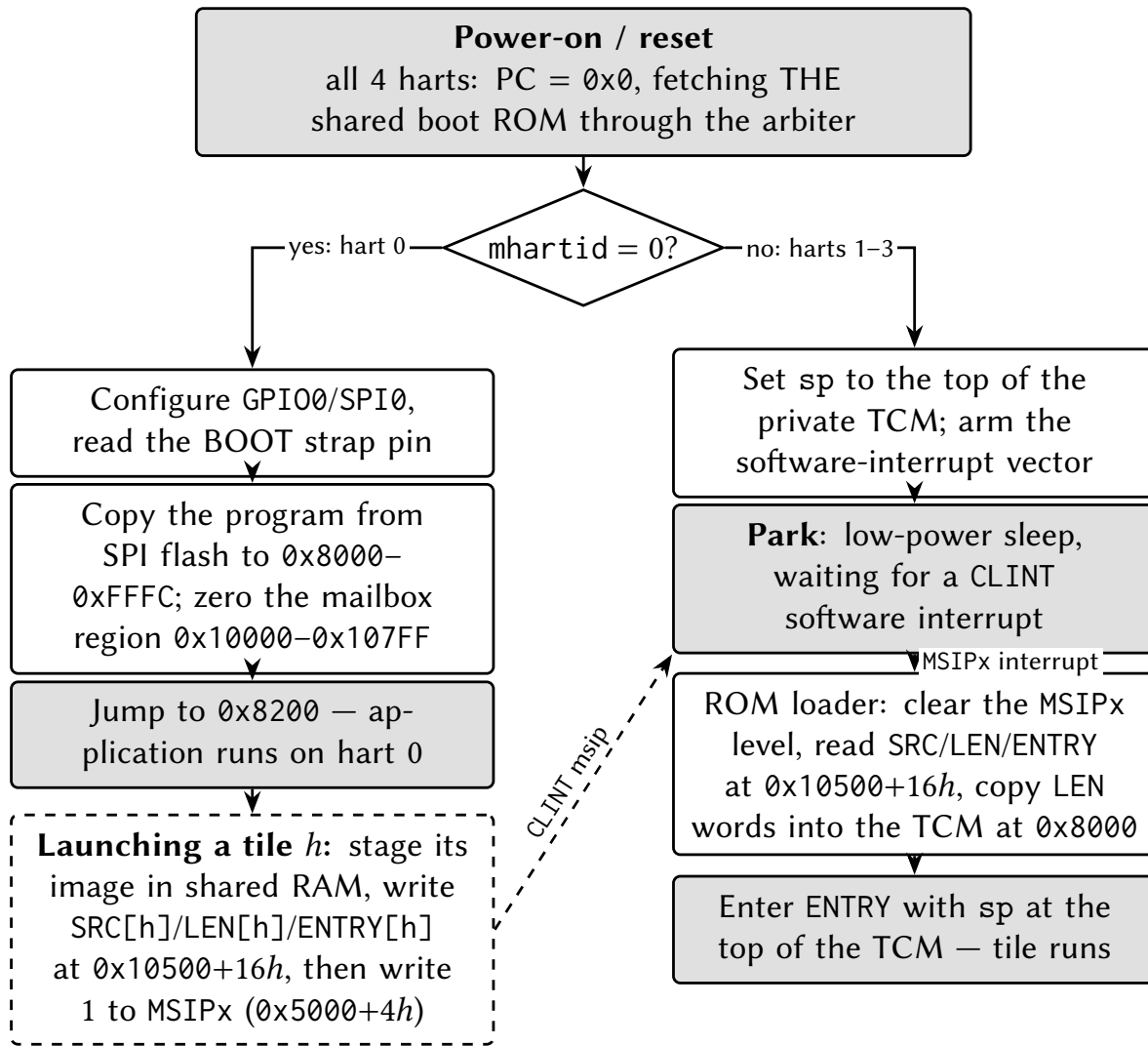


Figure 3: Boot and tile-loading flow. All harts execute the same shared boot ROM; hart 0 runs the legacy SPI-flash boot while every other hart parks in low-power sleep until it is loaded and released through its loader mailbox row and a CLINT software interrupt.

sends it a CLINT software interrupt. On wake, the ROM-resident loader reads the tile's *loader mailbox row* in shared RAM:

| Mailbox  | Address             | Meaning                                                |
|----------|---------------------|--------------------------------------------------------|
| SRC[h]   | $0x10500 + 16h + 0$ | Word-aligned source address in shared space            |
| LEN[h]   | $0x10500 + 16h + 4$ | Words to copy to the tile's TCM at $0x8000$ (0 = none) |
| ENTRY[h] | $0x10500 + 16h + 8$ | PC the tile enters with                                |

The loader clears the tile's own MSIP level, copies LEN[h] words from SRC[h] into the tile's private TCM at  $0x8000$  (a LEN of 4096 loads the full 16 KiB image — vector table, code, and interrupt handlers included), and enters ENTRY[h] with the stack pointer set to the top of the tile's TCM and all other registers undefined.

The canonical release sequence for hart  $h$  is therefore: stage the tile's program somewhere in shared RAM, write SRC[h] and LEN[h] and ENTRY[h], then write 1 to the hart's MSIP register ( $0x5000 + 4h$ ).

Because the mailbox row is consumed by the ROM only at wake time, the row must be complete *before* the MSIP write. Software interrupts sent to a tile *after* it has been loaded are handled by the loaded program's own vector table, so the same CLINT mechanism serves both boot-time loading and run-time inter-processor interrupts.

**Stacks:** each hart needs its own stack in its own private TCM. When an interrupt is taken, the hardware pushes the return program counter at  $sp - 4$  (see Section 6), so every hart — including a freshly loaded tile hart — must have a valid stack pointer before enabling interrupts or sleeping. The boot ROM guarantees this for parked and freshly-entered tiles; application code keeps the convention of placing each hart's stack at the top of its TCM, growing downward.

## 4.4 Synchronization

Castalia offers three synchronization mechanisms, from cheapest to most flexible. Figure 4 summarizes how to choose between them:

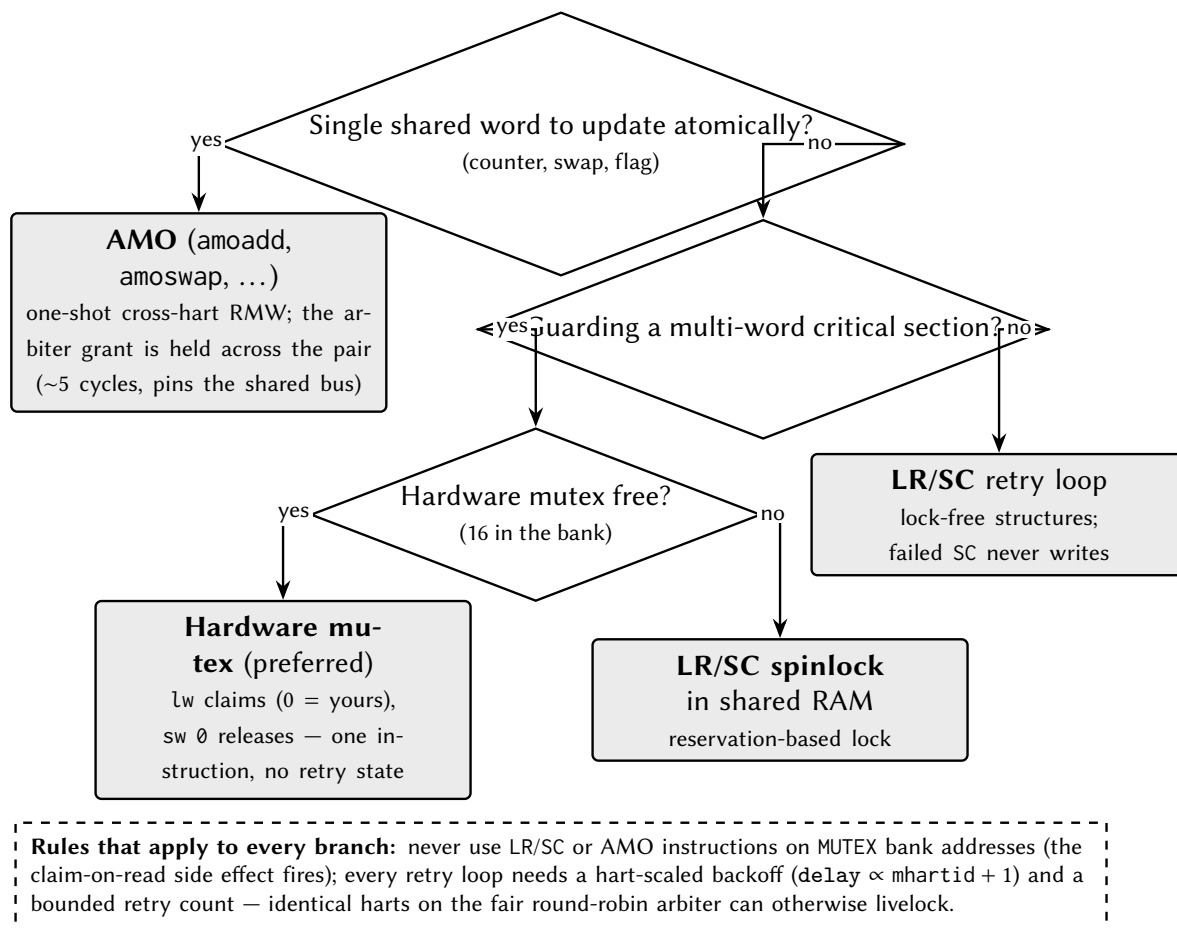


Figure 4: Choosing a synchronization primitive. The dashed box lists the two rules that apply to every branch.

1. **Hardware mutexes** (MUTEX bank): a single load claims a free mutex; a single store releases it. No retry loop hardware state, no reservation. This is the preferred lock for most uses. See the MUTEX chapter.

2. **LR/SC spinlocks in shared RAM:** standard RISC-V reservation-based locks. A failed SC has its write suppressed by the reservation unit, so it is safe under contention.
3. **AMO instructions to shared RAM:** atomic add, swap, and friends; the arbiter locks the grant across the read-modify-write pair.

Two rules apply to all of them:

- **Never** use LR/SC or AMO instructions on MUTEX bank addresses — hardware mutexes are claimed with plain loads and released with plain stores only.
- Under contention, spin with a *hart-scaled backoff*: delay between retries by an amount proportional to `mhartid` (for example `delay = k · (mhartid + 1)`) and bound the retry count. Multiple harts hammering the arbiter in lockstep can otherwise livelock a naive retry loop.
- Additionally, do not access the NPU staging RAM (`0xC000–0xFFFF`) while an NPU run may be active — poll the NPU's busy flag first (see the NPU chapter).

There are no data caches, so there is no cache-coherence protocol to consider: a shared-RAM write is globally visible as soon as the arbiter completes the transaction.

## 5 Startup and Reset Behavior

To place the MCU in reset mode, drive the `resetn` pin LOW (asserted). To take the MCU out of reset mode, let the `resetn` pin go high (deasserted). The `resetn` pin has an internal 47 k $\Omega$  pullup resistor which constantly attempts to pull the `resetn` pin HIGH. If the user does not wish to use the `resetn` pin, it is safe to leave it unconnected.

When the MCU is first powered up, a power-on reset (POR) circuit resets the MCU. As the core voltage, beginning at 0 V, begins to climb, the POR circuit does not assert the internal reset. Once the core voltage reaches a particular value (which is temperature and process dependent, but typically around 0.7 V), the POR circuit asserts the internal reset signal for about 20 ns and then deasserts it. If either the `resetn` pin or the POR reset signal is asserted, the MCU will be in reset mode. In reset, the memory mapped registers and CPU state variables are all reset to their powerup values.

When the MCU comes out of reset, it immediately begins to execute the bootloader stored in the ROM, setting the program counter to address `0x0000`. If the `BOOT` pin is LOW, the bootloader launches a Forth interpreter which communicates over `UART0` at 115,200 baud (see Section 8 for details concerning the Forth interpreter). If the `BOOT` pin is HIGH, the bootloader reads and executes the program stored on an external SPI flash on `SPI0` (see Section 10 for details on how to program the MCU). When the bootloader reads the program stored in the SPI flash, it dumps the contents of the SPI flash directly into RAM, completely filling the RAM from beginning to end using an address-to-address copy. After the bootloader has copied the program stored in the SPI flash into RAM, it jumps (moves the program counter) to address `0x8200` and begins executing the code.

Only hart 0 executes the boot ROM and the SPI-flash boot sequence described above. Harts 1–3 come out of reset executing from their private RAM and park until hart 0 releases them through the boot mailboxes in shared RAM; see Section 4.3.

Note that the boot ROM reprograms the clock system during boot, so the `SMCLK` source at application entry is a boot-ROM implementation detail — in particular, it may be left on the slow `LFXT` (32.768 kHz) source. Applications that use `SMCLK`-domain peripherals (the `UARTs`, `SPIs`, `I2Cs`, or a timer clock-source switch) should therefore always select their desired `SMCLK` source in the `SYSTEM` peripheral first, rather than relying on the boot-time clock state; see Section 15.1 and the clock handoff note in the `TIMERx` chapter.

## 6 Interrupts

Table 6: Interrupt Vectors

| Priority | Interrupt Source                      |
|----------|---------------------------------------|
| 0        | CPU Internal Timer                    |
| 1        | EBREAK, ECALL, or Illegal Instruction |
| 2        | Bus Error (unaligned memory access)   |
| 0        | SYSTEM                                |
| 1        | GPI00                                 |
| 9        | SPI0                                  |
| 11       | SPI1                                  |
| 13       | UART0                                 |
| 16       | TIMER0                                |
| 22       | TIMER1                                |
| 28       | GPI01                                 |
| 36       | GPI02                                 |
| 44       | GPI03                                 |
| 52       | UART1                                 |
| 57       | I2C0                                  |
| 70       | I2C1                                  |
| 83       | CLINT                                 |
| 98       | GPI04                                 |
| 106      | GPI05                                 |
| 120      | NPU                                   |

The VestaRV CPU implements an interrupt system with 121 distinct interrupt sources as enumerated in the table above: the 83 peripheral and system vectors, plus the two CLINT vectors (83, software/inter-processor interrupt; 84, timer interrupt) added for multi-core operation. Each hart takes three interrupt lines: its own CLINT software and timer interrupts (vectors 83 and 84, hardwired enabled, delivered on dedicated per-hart wires) and the external interrupt `meip` (vector 85), which delivers *all* peripheral and system sources through the `IRQROUTER`'s claim/complete mechanism: the vector-85 handler reads the router's `CLAIM` register to discover and claim the pending source, dispatches on the returned vector number, and writes it back to complete (see the `IRQROUTER` chapter). Which sources reach which hart is programmed per hart in the router's routing rows — identical for every hart, hart 0 included. All peripheral sources are masked by default upon MCU reset and must be explicitly routed in software before use; source priority is fixed (the lowest pending vector number is claimed first). Additionally, peripherals generating interrupts must be configured to enable interrupt generation at the peripheral level.

### 6.1 Interrupt Vector Table

The interrupt vector table (IVT) is located at the base of RAM, beginning at address `0x8000`. Unlike traditional pointer-based interrupt vector tables, the VestaRV IVT contains direct jump instructions to the interrupt service routines (ISRs). Each entry in the IVT is a 4-byte RISC-V jump instruction (e.g., `j handler`) that vectors execution immediately to the corresponding ISR when an interrupt occurs. This architecture eliminates the indirection overhead of loading function pointers and provides deterministic, low-latency interrupt response.



While the VestaRV hardware does not mandate a specific IVT structure, it is strongly recommended to populate the IVT with jump instructions during system initialization. The automatic code generation tools produce linker scripts and initialization code that configure the IVT at the beginning of RAM. The memory map header files provide macros to facilitate proper ISR placement and IVT entry configuration.

## 6.2 Interrupt Entry, Exit, and Recursion

When an interrupt is taken, the hardware saves a minimal context: it pushes the return program counter — one 32-bit word — onto the system stack at `sp - 4`, decrements `sp` by 4, and vectors execution through the IVT entry for the interrupt. **No other state is saved by hardware.** Software is responsible for saving and restoring every general-purpose register the interrupt service routine (ISR) uses. The generated runtime provides an interrupt entry wrapper (in the startup code) that saves the general-purpose registers on the stack — about 120 bytes per interrupt level — calls the C-level handler, restores the registers, and returns with the `retirq` instruction, so C-coded ISRs need no special handling. Hand-written assembly ISRs must perform the equivalent save/restore themselves. Executing `retirq` pops the hardware-saved program counter, increments `sp` by 4, and resumes execution at the interrupted location.

Because the very first thing interrupt entry does is a store at `sp - 4`, **the stack pointer must point at valid, writable private memory before interrupts are enabled.** This applies to every hart: a freshly released tile hart must load its stack pointer before unmasking interrupts or going to sleep (see Section 4).

Interrupt handling is non-recursive: a running ISR is never preempted, and further pending interrupts (including a pending CLINT interrupt during a vector-85 service) are taken after the current ISR returns. Stack allocation must cover one interrupt level: the 4-byte hardware-saved return address plus the handler's register frame (about 120 bytes for the runtime wrapper) and the ISR's local variables. (The single-core chip's priority-tier and recursion machinery was retired together with the SYSTEM interrupt registers; source arbitration now happens in the IRQROUTER.)

## 6.3 Sleep Mode and Interrupt Wake-Up

If an interrupt occurs while the CPU is in sleep mode, the processor will automatically wake from sleep and service the interrupt. Upon execution of the `retirq` instruction, the CPU will return to sleep mode unless the ISR or woken code explicitly reconfigures the clock and power settings. During sleep mode, the CPU clock is gated to conserve power, but all register contents and the program counter are retained. If the clock source for MCLK is disabled during sleep, any enabled interrupt will automatically power up and enable the source clock for the duration of interrupt servicing.

Clock and power management, including sleep mode configuration, are controlled through the SYSTEM peripheral. Note that if SMCLK is disabled, peripherals may be unable to assert interrupt signals, potentially preventing wake from sleep.

## 7 CPU Instructions and Assembly

The Castalia MCU integrates the VestaRV CPU core, a custom 32-bit RISC-V processor implementing the RV32IMAC instruction set with bit manipulation extensions (Zba, Zbb, Zbc, Zbs). This configuration provides integer arithmetic (I), hardware multiplication and division (M), atomic operations (A), compressed 16-bit instructions (C), and enhanced bit manipulation capabilities. The RISC-V instruction set architecture (ISA) defines the binary encoding of machine instructions, their operational semantics, and the register architecture used for computation and control flow.

### 7.1 CPU Registers

There are 32 CPU registers defined in the RISC-V ISA. Though the user can technically use any register for any purpose (except for x0/zero, which is read-only and is always equal to 0x00000000), it is generally advisable to use each register for its intended purpose and follow the convention that the RISC-V ISA describes. The convention was created to create a standard use for each register so that assembly functions can be written that will be compatible with any RISC-V assembly program. The RISC-V compiler always follows this convention. Therefore, if the user also follows the convention, then the user's assembly code will be drop-in compatible with other compiled code (such as C code). If the user does not follow the convention, then it may cause many runtime errors.

Each register has a specific purpose and a designated “saver” that is allowed to write to it. The saver is either the caller (the code that calls a function) or the callee (the function that was called). This allows for communication between caller and callee such that arguments can be passed from caller to callee and returned data can be sent back from callee to caller. For example, the registers a0-a7 are intended for function arguments and are written to by the caller. Registers s0-s11 are preserved across function calls and are written to by the callee. Furthermore, registers t0-t6 are temporary register that can be used for any purpose and are written to by the caller.

In assembly, registers can be referred to interchangeably by their register number or name (e.g. you can either type x1 or ra to refer to the same register). Generally, one should only use the register names since they give hints as to the purpose of the register.

A CPU instruction can have up to two source register rs1 and rs2, and up to one destination register rd. rs1, rs2, and rd can be any of the 32 CPU registers (e.g. sp, t6, zero, etc.). The source registers are used as “arguments” to the instruction and are not modified by the instruction, and the destination register is modified by the instruction as its output.

A list of all 32 CPU registers is provided in Table 7.

Table 7: CPU Registers

| Register | Name  | Description                       | Saver  |
|----------|-------|-----------------------------------|--------|
| x0       | zero  | Hard-wired zero (read-only)       | N/A    |
| x1       | ra    | Return Address                    | Caller |
| x2       | sp    | Stack Pointer                     | Callee |
| x3       | gp    | Global Pointer                    | –      |
| x4       | tp    | Thread Pointer                    | –      |
| x5       | t0    | Temporary/Alternate Link Register | Caller |
| x6-x7    | t1-t2 | Temporary Registers               | Caller |
| x8       | s0/fp | Saved Register/Frame Pointer      | Callee |
| x9       | s1    | Saved Register                    | Callee |
| x10-x11  | a0-a1 | Function Arguments/Return Values  | Caller |

| Register | Name   | Description         | Saver  |
|----------|--------|---------------------|--------|
| x12-x17  | a2-a7  | Function Arguments  | Caller |
| x18-x27  | s2-s11 | Saved Registers     | Callee |
| x28-x31  | t3-t6  | Temporary Registers | Caller |

## 7.2 Load/Store Memory Instructions

These instructions interface between the memory bus (containing RAM, ROM, and memory-mapped registers) and the CPU registers. They are used to load data from memory into a CPU register or store data from a CPU register into memory. The immediate parameter (i) refers to a hard-coded 12-bit constant that is stored inside the CPU instruction itself and is added to the value of one of the registers. All load/store instructions take 2 MCLK cycles to execute.

- Load Signed Byte: `lb rd, I(rs1)` : loads the signed byte (8 bits) stored at memory location pointed to by (rs1 + I) into the destination CPU register rd
- Load Unsigned Byte: `lbu rd, I(rs1)` : loads the unsigned byte (8 bits) stored at memory location pointed to by (rs1 + I) into the destination CPU register rd
- Load Signed Half Word: `lh rd, I(rs1)` : loads the signed half word (16 bits) stored at memory location pointed to by (rs1 + I) into the destination CPU register rd
- Load Unsigned Half Word: `lhu rd, I(rs1)` : loads the unsigned half word (16 bits) stored at memory location pointed to by (rs1 + I) into the destination CPU register rd
- Load Word: `lw rd, I(rs1)` : loads the signed or unsigned word (32 bits) stored at memory location pointed to by (rs1 + I) into the destination CPU register rd
- Store byte: `sb rs1, I(rs2)` : stores the signed or unsigned byte (8 bits) from source CPU register 1 rs1 into the memory location pointed to by (rs2 + I)
- Store half word: `sh rs1, I(rs2)` : stores the signed or unsigned half word (16 bits) from source CPU register 1 rs1 into the memory location pointed to by (rs2 + I)
- Store word: `sw rs1, I(rs2)` : stores the signed or unsigned word (32 bits) from source CPU register 1 rs1 into the memory location pointed to by (rs2 + I)

## 7.3 Shift Instructions

These instructions perform bit shift operations (e.g. <<). The immediate parameter (I) refers to a hard-coded 12-bit constant that is stored inside the CPU instruction itself and is used as the number of bits to shift. Each shift instruction takes 1 MCLK cycle to execute.

- Shift Left: `sll rd, rs1, rs2` : Logical shifts the value in rs1 left by rs2 bits and stores the result in rd.
- Shift Left Immediate: `slli rd, rs1, I` : Logical shifts the value in rs1 left by I bits and stores the result in rd.
- Shift Right: `srl rd, rs1, rs2` : Logical shifts the value in rs1 right by rs2 bits and stores the result in rd.

- Shift Right Immediate: `slli rd, rs1, I` : Logical shifts the value in `rs1` right by `I` bits and stores the result in `rd`.
- Shift Right Arithmetic: `sll rd, rs1, rs2` : Arithmetic shifts the value in `rs1` right by `rs2` bits and stores the result in `rd`.
- Shift Right Arithmetic Immediate: `slli rd, rs1, I` : Arithmetic shifts the value in `rs1` right by `I` bits and stores the result in `rd`.

## 7.4 Arithmetic Instructions

These instructions perform arithmetic operations. The immediate parameter (`I`) refers to a hard-coded 12-bit signed integer that is stored inside the CPU instruction itself and is added to the operation (except for in the `auipc` instruction). These instructions take 1 MCLK cycle to execute.

- Addition: `add rd, rs1, rs2` : Adds the signed/unsigned integer values in `rs1` and `rs2` together and stores the sum in `rd`
- Addition Immediate: `addi rd, rs1, I` : Adds the signed/unsigned integer value in `rs1` with the immediate 12-bit signed integer `I` together and stores the sum in `rd`
- Subtraction: `sub rd, rs1, rs2` : Subtracts the signed/unsigned integer values in `rs1` from `rs2` and stores the result in `rd` (i.e.  $rd \leftarrow rs1 - rs2$ )
- Load Upper Immediate: `lui rd, I` : Loads the 20-bit immediate value `I` into the upper 20 bits of the destination CPU register `rd` (the lower 12 bits are all cleared to 0s)
- Add Upper Immediate to PC: `auipc rd, I` : Adds the 12-bit signed integer immediate `I` to the program counter to jump the program's execution to a location `I` away from the current program counter. Note that `I` can be negative.

## 7.5 Bitwise Instructions

These instructions perform bitwise operations. The immediate parameter (`I`) refers to a hard-coded 12-bit signed integer that is stored inside the CPU instruction itself and is used as one of the operands of the bitwise operations. These instructions take 1 MCLK cycle to execute.

- Bitwise exclusive OR: `xor rd, rs1, rs2` : Performs the bitwise XOR of `rs1` and `rs2` and stores the result in `rd`.
- Bitwise exclusive OR Immediate: `xori rd, rs1, I` : Performs the bitwise XOR of `rs1` and the immediate `I` and stores the result in `rd`.
- Bitwise OR: `or rd, rs1, rs2` : Performs the bitwise OR of `rs1` and `rs2` and stores the result in `rd`.
- Bitwise OR Immediate: `ori rd, rs1, I` : Performs the bitwise OR of `rs1` and the immediate `I` and stores the result in `rd`.
- Bitwise AND: `and rd, rs1, rs2` : Performs the bitwise AND of `rs1` and `rs2` and stores the result in `rd`.
- Bitwise AND Immediate: `andi rd, rs1, I` : Performs the bitwise AND of `rs1` and the immediate `I` and stores the result in `rd`.

## 7.6 Comparison Instructions

These instructions perform comparison operations. The results of these instructions are either true (0x00000001) or false (0x00000000). The immediate parameter (I) refers to a hard-coded 12-bit signed or unsigned integer that is stored inside the CPU instruction itself and is used as one of the operands of the bitwise operations. These instructions take 1 MCLK cycle to execute.

- Set when Less Than: `slt rd, rs1, rs2`: If  $rs1 < rs2$  (using signed integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.
- Set when Less Than Immediate: `slti rd, rs1, I`: If  $rs1 < I$  (using signed integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.
- Set when Less Than Unsigned: `sltu rd, rs1, rs2`: If  $rs1 < rs2$  (using unsigned integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.
- Set when Less Than Immediate Unsigned: `sltiu rd, rs1, I`: If  $rs1 < I$  (using unsigned integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.

## 7.7 Branch Instructions

These instructions are conditional instructions that branch (jump) the program execution to a new location if the condition is met, or proceed on to the next instruction like normal if the condition is not met. The immediate parameter (I) refers to a hard-coded 12-bit signed integer that is stored inside the CPU instruction itself and is added to the program counter if the condition is met. Usually, in assembly, the program writer can simply provide the *label* to the desired branch location/function in the program (e.g. `labelToFunction`;) rather than its actual memory location, which is usually not known before being built. These instructions take 1 MCLK cycle to execute regardless of whether the branch is taken or not.

- Branch if Equal: `beq rs1, rs2, I`: Branches to  $PC + I$  if  $rs1 = rs2$ , proceeds to the next instruction like normal otherwise.
- Branch if Not Equal: `bne rs1, rs2, I`: Branches to  $PC + I$  if  $rs1 \neq rs2$ , proceeds to the next instruction like normal otherwise.
- Branch if Less Than: `blt rs1, rs2, I`: Branches to  $PC + I$  if  $rs1 < rs2$  (signed integers), proceeds to the next instruction like normal otherwise.
- Branch if Greater Than or Equal: `bge rs1, rs2, I`: Branches to  $PC + I$  if  $rs1 \geq rs2$  (signed integers), proceeds to the next instruction like normal otherwise.
- Branch if Less Than Unsigned: `bltu rs1, rs2, I`: Branches to  $PC + I$  if  $rs1 < rs2$  (unsigned integers), proceeds to the next instruction like normal otherwise.
- Branch if Greater Than or Equal Unsigned: `bgeu rs1, rs2, I`: Branches to  $PC + I$  if  $rs1 \geq rs2$  (unsigned integers), proceeds to the next instruction like normal otherwise.

## 7.8 Jump and Link Instructions

The jump and link instructions write the address of the next consecutive instruction in the program to the destination register, but then jump the program counter to another location in memory. These instructions

are used to call new functions while preserving the “return address”, or the memory location of the instruction to go back to after the function returns. The immediate parameter (I) refers to a hard-coded 20-bit signed integer that is stored inside the CPU instruction itself and is used to modify the program counter. Usually, in assembly, the program writer can simply provide the *label* to the desired location/function in the program (e.g. `labelToFunction:`) rather than its actual memory location, which is usually not known before being built. All jump and link instructions take 1 MCLK cycle to execute.

- **Jump and Link:** `jal rd, I` Stores the return address to rd (which is usually the return address register ra) and then adds I to the program counter, effectively jumping to PC + I.
- **Jump and Link Register:** `jalr rd, rs1, I` Stores the return address to rd (which is usually the return address register ra) and then sets the program counter to rs1 + I, effectively jumping to rd1 + I.

## 7.9 Atomic Memory Operations (A Extension)

The VestaRV core implements the RISC-V Atomic (A) extension, providing atomic read-modify-write operations for multiprocessor synchronization. On the Castalia MCU these instructions are the standard way to build locks and shared counters in the arbitrated shared RAM: an AMO instruction holds the shared-window arbiter grant across its whole read-modify-write pair, and a failed `sc.w` has its write suppressed by the reservation unit, so both are atomic with respect to all four harts (see Section 4.4). Do not use atomic instructions on hardware mutex addresses (see the MUTEX peripheral chapter). All atomic memory operations operate on 32-bit words and take 5 MCLK cycles to execute.

- **Load Reserved:** `lr.w rd, (rs1)` : Loads a 32-bit word from memory address in rs1 into rd and reserves the memory location for atomic access.
- **Store Conditional:** `sc.w rd, rs2, (rs1)` : Conditionally stores the 32-bit word in rs2 to the memory address in rs1. Returns 0 in rd on success, nonzero on failure.
- **Atomic Swap:** `amoswap.w rd, rs2, (rs1)` : Atomically swaps the 32-bit word at memory address in rs1 with the value in rs2, storing the original memory value in rd.
- **Atomic Add:** `amoadd.w rd, rs2, (rs1)` : Atomically adds rs2 to the 32-bit word at memory address in rs1, storing the original memory value in rd.
- **Atomic XOR:** `amoxor.w rd, rs2, (rs1)` : Atomically performs bitwise XOR of rs2 with the 32-bit word at memory address in rs1, storing the original memory value in rd.
- **Atomic AND:** `amoand.w rd, rs2, (rs1)` : Atomically performs bitwise AND of rs2 with the 32-bit word at memory address in rs1, storing the original memory value in rd.
- **Atomic OR:** `amoor.w rd, rs2, (rs1)` : Atomically performs bitwise OR of rs2 with the 32-bit word at memory address in rs1, storing the original memory value in rd.
- **Atomic Minimum:** `amomin.w rd, rs2, (rs1)` : Atomically compares rs2 with the 32-bit signed word at memory address in rs1, storing the minimum value to memory and the original memory value in rd.
- **Atomic Maximum:** `amomax.w rd, rs2, (rs1)` : Atomically compares rs2 with the 32-bit signed word at memory address in rs1, storing the maximum value to memory and the original memory value in rd.

- Atomic Minimum Unsigned: `amominu.w rd, rs2, (rs1)` : Atomically compares `rs2` with the 32-bit unsigned word at memory address in `rs1`, storing the minimum value to memory and the original memory value in `rd`.
- Atomic Maximum Unsigned: `amomaxu.w rd, rs2, (rs1)` : Atomically compares `rs2` with the 32-bit unsigned word at memory address in `rs1`, storing the maximum value to memory and the original memory value in `rd`.

## 7.10 Bit Manipulation Instructions (Zb\* Extensions)

The VestaRV core implements several bit manipulation extensions (Zba, Zbb, Zbc, Zbs) that provide efficient operations for bit-level data manipulation, cryptographic primitives, and address generation. All bit manipulation instructions execute in 1 MCLK cycle.

### 7.10.1 Address Generation (Zba)

- Shift Left Add: `sh1add rd, rs1, rs2` : Shifts `rs1` left by 1 bit, adds `rs2`, and stores result in `rd`. Equivalent to `rd = (rs1 « 1) + rs2`.
- Shift Left Add 2: `sh2add rd, rs1, rs2` : Shifts `rs1` left by 2 bits, adds `rs2`, and stores result in `rd`. Equivalent to `rd = (rs1 « 2) + rs2`.
- Shift Left Add 3: `sh3add rd, rs1, rs2` : Shifts `rs1` left by 3 bits, adds `rs2`, and stores result in `rd`. Equivalent to `rd = (rs1 « 3) + rs2`.

### 7.10.2 Basic Bit Manipulation (Zbb)

- AND with Inverted: `andn rd, rs1, rs2` : Performs bitwise AND of `rs1` with the bitwise complement of `rs2`, storing result in `rd`.
- OR with Inverted: `orn rd, rs1, rs2` : Performs bitwise OR of `rs1` with the bitwise complement of `rs2`, storing result in `rd`.
- XOR with Inverted: `xnor rd, rs1, rs2` : Performs bitwise XOR of `rs1` with the bitwise complement of `rs2`, storing result in `rd`.
- Count Leading Zeros: `clz rd, rs1` : Counts the number of consecutive zero bits starting from the most significant bit in `rs1`, storing the count in `rd`.
- Count Trailing Zeros: `ctz rd, rs1` : Counts the number of consecutive zero bits starting from the least significant bit in `rs1`, storing the count in `rd`.
- Count Population: `cpop rd, rs1` : Counts the number of set bits (1s) in `rs1`, storing the count in `rd`.
- Maximum: `max rd, rs1, rs2` : Stores the maximum of `rs1` and `rs2` (signed comparison) in `rd`.
- Maximum Unsigned: `maxu rd, rs1, rs2` : Stores the maximum of `rs1` and `rs2` (unsigned comparison) in `rd`.
- Minimum: `min rd, rs1, rs2` : Stores the minimum of `rs1` and `rs2` (signed comparison) in `rd`.

- Minimum Unsigned: `minu rd, rs1, rs2` : Stores the minimum of `rs1` and `rs2` (unsigned comparison) in `rd`.
- Sign Extend Byte: `sext.b rd, rs1` : Sign-extends the least significant byte in `rs1` to 32 bits, storing result in `rd`.
- Sign Extend Halfword: `sext.h rd, rs1` : Sign-extends the least significant halfword (16 bits) in `rs1` to 32 bits, storing result in `rd`.
- Zero Extend Halfword: `zext.h rd, rs1` : Zero-extends the least significant halfword in `rs1` to 32 bits, storing result in `rd`.
- Rotate Left: `rol rd, rs1, rs2` : Rotates `rs1` left by the number of bits specified in the lower 5 bits of `rs2`, storing result in `rd`.
- Rotate Right: `ror rd, rs1, rs2` : Rotates `rs1` right by the number of bits specified in the lower 5 bits of `rs2`, storing result in `rd`.
- Rotate Right Immediate: `rori rd, rs1, shamt` : Rotates `rs1` right by `shamt` bits, storing result in `rd`.
- OR Combine: `orc.b rd, rs1` : Sets each byte in `rd` to 0xFF if the corresponding byte in `rs1` is nonzero, otherwise sets to 0x00.
- Byte Reverse: `rev8 rd, rs1` : Reverses the byte order in `rs1`, storing result in `rd` (converts between big-endian and little-endian).

### 7.10.3 Carryless Multiplication (Zbc)

- Carryless Multiply Low: `clmul rd, rs1, rs2` : Performs carryless multiplication of `rs1` and `rs2`, storing the lower 32 bits of the result in `rd`. Useful for CRC calculations and cryptographic operations.
- Carryless Multiply High: `clmulh rd, rs1, rs2` : Performs carryless multiplication of `rs1` and `rs2`, storing the upper 32 bits of the result in `rd`.
- Carryless Multiply Reversed: `clmulr rd, rs1, rs2` : Performs carryless multiplication with a bit-reversed result, storing bits [62:31] of the 64-bit carryless product in `rd`.

### 7.10.4 Single-Bit Manipulation (Zbs)

- Bit Clear: `bclr rd, rs1, rs2` : Clears the bit in `rs1` at the position specified by the lower 5 bits of `rs2`, storing result in `rd`.
- Bit Clear Immediate: `bclri rd, rs1, shamt` : Clears the bit in `rs1` at position `shamt`, storing result in `rd`.
- Bit Extract: `bext rd, rs1, rs2` : Extracts the bit in `rs1` at the position specified by the lower 5 bits of `rs2`, storing the single bit (zero-extended) in `rd`.
- Bit Extract Immediate: `bexti rd, rs1, shamt` : Extracts the bit in `rs1` at position `shamt`, storing the single bit (zero-extended) in `rd`.



- Bit Invert: `binv rd, rs1, rs2` : Inverts the bit in rs1 at the position specified by the lower 5 bits of rs2, storing result in rd.
- Bit Invert Immediate: `binvi rd, rs1, shamt` : Inverts the bit in rs1 at position shamt, storing result in rd.
- Bit Set: `bset rd, rs1, rs2` : Sets the bit in rs1 at the position specified by the lower 5 bits of rs2, storing result in rd.
- Bit Set Immediate: `bseti rd, rs1, shamt` : Sets the bit in rs1 at position shamt, storing result in rd.

### 7.11 Pseudo-Instructions

The RISC-V assembler also includes several pseudo-instructions for convenience. These pseudo-instructions are actually special cases of regular instructions, and can all be expressed as regular instructions.

- No operation: `nop` → `addi zero, zero, 0`
- Move: `mv rd, rs1` → `addi rd, rs1, 0`
- Bitwise Complement/Inversion: `not rd, rs1` → `xor rd, rs1, -1`
- Set if Equal to Zero: `seqz rd, rs1` → `sltiu rd, rs1, 1`
- Set if Not Equal to Zero: `snez rd, rs1` → `sltu rd, zero, rs1`
- Set if Less Than Zero: `sltz rd, rs1` → `slt rd, rs1, zero`
- Set if Greater Than Zero: `sgtz rd, rs1` → `slt rd, zero, rs1`
- Jump (without linking): `j I` → `jal zero, I`
- Return: `ret` → `jalr zero, ra, 0`
- Call function: `call I` → `auipc t1 I; jalr ra t1 I`
- Load immediate: `li rd, I` (*this could compile to several instructions*)
- Branch if Greater Than: `bgt rs1, rs2, I` → `blt rs2, rs1, I`
- Branch if Less Than or Equal To: `ble rs1, rs2, I` → `bge rs2, rs1, I`
- Branch if Greater Than Unsigned: `bgtu rs1, rs2, I` → `bltu rs2, rs1, I`
- Branch if Less Than or Equal To Unsigned: `bleu rs1, rs2, I` → `bgeu rs2, rs1, I`
- Branch if Equal to Zero: `beqz rs1, I` → `beq rs1, zero, I`
- Branch if Not Equal to Zero: `bnez rs1, I` → `bne rs1, zero, I`
- Branch if Less Than or Equal To Zero: `blez rs1, I` → `bge zero, rs1, I`

- Branch if Greater Than or Equal To Zero: `bgez, rs1, I` → `bge rs1, zero, I`
- Branch if Less Than Zero: `bltz rs1, I` → `blt rs1, zero, I`
- Branch if Greater Than Zero: `bgtz rs1, I` → `blt zero, rs1, I`

## 8 Forth Interpreter

When the MCU comes out of reset and the BOOT pin is LOW, the MCU boots to an interactive Forth interpreter based on the Forth language. Forth is a stack-based language where variables are pushed to a stack in LIFO (last in, first out) order. When a variable is set, it is pushed to the top of the stack (TOS). When a variable is used, it is popped (or consumed) from the TOS. Functions that take arguments pop them from the TOS. Functions that have return values push them to the TOS (after first popping its arguments). Variables are not assigned names, and the only way of distinguishing them from one another is by their order on the stack. All variables are treated as 32-bit signed integers. Variables may be pushed to the stack through the command line by simply typing the variable value into the interpreter as a plain text integer value or as a plain text hexadecimal value. Hexadecimal values must be preceded by the string `0x` without spaces in between (e.g. `0xA4`), but the value may be upper or lower case and does not need leading zeros. Variables are not pushed or popped and functions are not executed until a newline (or line feed) `\n` character is received, at which point the entire received line will be executed in order. New functions may be declared, and may use the stack and any already defined functions to perform their procedures. If the Forth interpreter does not recognize an input as a variable or a function, it prints the `?` char over the UART.

The Forth interpreter contained in the ROM of the MCU communicates over UART0 at 2,048 baud. The baud may be changed by reconfiguring SMCLK and the UART0 peripheral. When the Forth interpreter first starts up, it prints the string `rv4th!\n>`, and is then ready to receive characters over the UART. The Forth interpreter does not perform error handling on stack underflows. In the case of a stack underflow, it simply uses the value 0 as the TOS. By default, the echo is enabled, which repeats all characters received by the Forth interpreter back to the host. This is useful for typing into a terminal emulator manually, but may be switched off with the command `0 echo`.

Several useful built-in Forth functions are listed below. The syntax is read as follows: The Forth function keyword is surrounded by quotations (though you do not actually type the quotes into the Forth interpreter), and is followed by the list of arguments and return values in parentheses. Left of the double dash is the list of arguments in the order that they would be typed into the Forth interpreter (reverse stack order), and right of the double dash is the list of return values in reverse stack order. Reverse stack order means that the rightmost item is at the TOS, and items to the left of it are below it on the stack. Arguments are always popped from the stack before the function executes, and return values are always pushed to the stack after the function finishes.

### 8.1 Memory Access Functions

- `@ ( addr -- )`

Read directly from the memory address and push the value to the TOS

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address to read from (must be aligned to a 4-byte (32-bit) boundary)

Return values: none

- `! ( x addr -- )`

Write directly to memory address

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)

2. x: The value to write to the memory address

Return values: none

- **+**! ( x addr -- )

Add to value, direct memory access (\*addr += x)

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. x: The value to add to the value at the memory address

Return values: none

- **sbi** ( bitnum addr -- )

Set bit, direct memory access (\*addr |= (1 « bitnum))

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. bitnum: The bit number

Return values: none

- **cbi** ( bitnum addr -- )

Clear bit, direct memory access (\*addr &= ~(1 « bitnum))

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. bitnum: The bit number

Return values: none

- **mask** ( bitmask x addr -- )

Change only masked bits, direct memory access (\*addr = (\*addr & ~bitmask) | (x & bitmask))

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. x: The value to write (only masked bits will be written)
3. bitmask: The bit mask. Only bits with 1s can be changed

Return values: none

## 8.2 Print and Input Functions

- **.** ( x -- )

Prints the TOS as an integer

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) The value to print

Return values: none

- **h.** ( x -- )

Prints the TOS as a hex string

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) The value to print

Return values: none

- **echo** ( bool -- )

Change the echo state, which controls whether the Forth interpreter repeats all of its received characters

Arguments: (in order from TOS down, reverse text order):

1. bool: (TOS) If 0, does not echo. Otherwise, it does

Return values: none

- **emit** ( x -- )

Prints the char at the TOS (integers are mapped to their ASCII counterparts)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) The ASCII value of the char to print

Return values: none

- **key** ( -- char )

Waits for a char over UART and pushes the ASCII value of the char to the TOS

Arguments: none

Return values: (in order from TOS down, reverse text order):

1. char: (TOS) The char that was received over UART

- **list** ( -- )

Prints all currently available functions

Arguments: none

Return values: none

### 8.3 Arithmetic Functions

- **+** ( y x -- ret )

Adds two numbers (y + x)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **-** ( y x -- ret )

Subtracts two numbers (y - x)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **\*** ( y x -- retprodhi retprodlo )

Multiplies two numbers (y \* x)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. retprodlo: (TOS) Lower 32 bits of the return product
2. retprodhi: Upper 32 bits of the return product

- **/%** ( y x -- retdiv retrem )

Divides two numbers with remainder (y / x and y % x)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. retrem: (TOS) The remainder (or modulo) result
2. retdiv: The integer division result

- **neg** ( x -- ret )

Returns the negative (twos complement) of x

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **abs** ( x -- ret )

Absolute value ( $|x|$ )

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

## 8.4 Bitwise Logic Functions

- **~** ( x -- ret )

Bitwise complement/inversion ( $\sim x$ )

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **|** ( y x -- ret )

Bitwise OR ( $y | x$ )

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **&** ( y x -- ret )

Bitwise AND ( $y \& x$ )

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **^** ( y x -- ret )

Bitwise XOR ( $y \wedge x$ )

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- `*2 ( x -- ret )`

Shift left 1 bit ( $x \ll 1$ )

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- `/2 ( x -- ret )`

Shift right 1 bit ( $x \gg 1$ )

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- `sll ( x bits -- ret )`

Shift left logical ( $x \ll \text{bits}$ )

Arguments: (in order from TOS down, reverse text order):

1. bits: (TOS) Signed integer
2. x: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- `srl ( x bits -- ret )`

Shift right logical ( $x \gg \text{bits}$ ), no sign extension

Arguments: (in order from TOS down, reverse text order):

1. bits: (TOS) Signed integer
2. x: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value



## 8.5 Comparison Functions

- **>** ( y x -- ret )

Greater than. Returns 1 if  $y > x$ , returns 0 otherwise.

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) 1 if true, 0 if false

- **<** ( y x -- ret )

Less than. Returns 1 if  $y < x$ , returns 0 otherwise.

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) 1 if true, 0 if false

- **==** ( y x -- ret )

Equals. Returns 1 if  $y == x$ , returns 0 otherwise.

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) 1 if true, 0 if false

## 8.6 Boolean Functions

- **not** ( x -- ret )

Logical not (!x). Returns 1 if x is equal to 0, returns 0 otherwise

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **and** ( y x -- ret )

Logical and (y && x). Returns 1 if y and x are both true (nonzero), returns 0 otherwise

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **or** ( y x -- ret )

Logical or ( $y \parallel x$ ). Returns 1 if y or x are true (nonzero), returns 0 otherwise

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

## 8.7 Stack Manipulation Functions

- **dup** ( x -- x x )

Duplicates the TOS (pops the TOS and then pushes the value to the TOS twice)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Value to duplicate

Return values: (in order from TOS down, reverse text order):

1. x: (TOS) The duplicated value
2. x: The original TOS

- **drop** ( x -- )

Pops the TOS and sets it as the internal Forth i value. Usually used to delete the TOS.

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) The value that is dropped

Return values: none

- **swap** ( y x -- x y )

Swaps the TOS with the value below the TOS

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Old TOS
2. y: Old value below the TOS

Return values: (in order from TOS down, reverse text order):

1. y: (TOS) Old value below the TOS
2. x: Old TOS

- **over** ( y x -- y x y )

Copies the value below the TOS and pushes it to the TOS

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed Integer
2. y: Signed Integer

Return values: (in order from TOS down, reverse text order):

1. y: (TOS) Signed Integer
2. x: Signed Integer
3. y: Signed Integer

- **tuck** ( y x -- x y x )

Copies the TOS and inserts it two after the TOS

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed Integer
2. y: Signed Integer

Return values: (in order from TOS down, reverse text order):

1. x: (TOS) Signed Integer
2. y: Signed Integer
3. x: Signed Integer

- **roll** ( n -- )

Removes value at index n of the stack and pushes it to the TOS (zero indexed)

Arguments: (in order from TOS down, reverse text order):

1. n: (TOS) Stack index (zero indexed)

Return values: none

- **pick** ( n -- )

Copies value at index n of the stack and pushes it to the TOS (zero indexed)

Arguments: (in order from TOS down, reverse text order):

1. n: (TOS) Stack index (zero indexed)

Return values: none

- **depth** ( -- ret )

Gets size of stack and pushes it to the top of the stack (making the new stack size the value of TOS + 1)

Arguments: none

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

## 8.8 SPI Flash R/W and Programming Functions

- **fr** ( bin length addr -- )

Reads data from the SPI flash.

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address in the SPI flash to begin reading from
2. length: The number of bytes to read
3. bin: If true (nonzero), transmits binary data, otherwise transmits ASCII hex string

Return values: none

- **fe** ( conf addr -- eraseSuccessful )

Erases a 256-byte page from the SPI flash memory. The start address must be 256-byte aligned.

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The start address of the page to erase in the SPI flash. Must be 256-byte aligned.
2. conf: If equal to 123, erases the page and returns 1, otherwise does not erase and returns 0

Return values: (in order from TOS down, reverse text order):

1. eraseSuccessful: (TOS) Nonzero if successful, zero if failed

- **fw** ( bin addr -- )

Writes data to the SPI flash. Must write exactly 256 bytes at a time. The start address must be 256-byte aligned.

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address in the SPI flash to begin writing to. Must be 256-byte aligned.
2. bin: If true (nonzero), expects to receive binary data, otherwise expects to receive an ASCII hex string

Return values: none

## 8.9 Miscellaneous Functions

- **rst** ( -- )

Performs a soft reset (turns on all memory cells and jumps to address 0 in the ROM, does not reset clocks or hardware)

Arguments: none

Return values: none

- **clk** ( meastime clkselect -- freq )

Measures selected clock frequency

Arguments: (in order from TOS down, reverse text order):

1. clkselect: (TOS) Clock select. 0 = SMCLK; 1 = MCLK; 2 = LFXT; 3 = HFXT

2. meastime: Measurement time. 0 = 1 s; 1 = 0.5 s; 2 = 0.25 s; 3 = 0.125 s

Return values: (in order from TOS down, reverse text order):

1. freq: (TOS) Selected clock frequency in Hz

- **min** ( y x -- ret )

Returns minimum value of y and x

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **max** ( y x -- ret )

Returns maximum value of y and x

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **swpb** ( x -- ret )

Swap bits 15 downto 8 with bits 7 downto 0 of TOS, also causes bits 31 downto 16 to all be 0

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **swphw** ( x -- ret )

Swap upper 16 bits with lower 16 bits of TOS

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

## 8.10 Creating New Forth Functions

To create a new Forth function on-the-fly, simply use the syntax `: funcname definition ;` where `funcname` is the name of the new function and `definition` is the body of the function. The new function may be called at any time after the semicolon (even before a new line, though it will not actually be executed until a new line) by typing the function name, just like all other Forth functions. New Forth functions can only add or remove variables from the stack and call other defined Forth functions. The arguments to custom Forth functions are the initial values on the stack that the custom function pops off the TOS. Likewise, the return values are any new values the custom function pushes to the TOS.

Listed below is an example program that increments the TOS by one when it is called:

```
: inc 1 + ;
```

## 8.11 Conditionals

The `if then` functions implement a conditional statement which may only be used inside a user defined function. The conditional is used with the syntax:

```
if procedureIfTrue then
```

inside a function (do not forget the space and semicolon which must come after `forthinlinethen`). The `if` function pops the TOS and then executes whatever is defined within `procedureIfTrue` if and only if the TOS is true (nonzero). The `then` function marks the end of the conditional procedure. For example, a function to print the text cool if the TOS is equal to 5 might look like this:

```
: cool? 5 == if 108 111 111 99 emit emit emit emit then ;
```

The `else` keyword may be inserted after `procedureIfTrue` if you want to execute something if the conditional is false. The syntax then becomes:

```
if procedureIfTrue else procedureIfFalse then
```

## 8.12 Loops

Forth has the equivalent of C-like while loops (or, more accurately, `do while` loops) with its `begin until` loops, which may only be executed inside a user defined function. The syntax is:

```
begin loopProcedure until
```

which first executes the loop procedure. When it gets to the `until` statement, it then checks if the TOS is true (nonzero) or false (zero). If the TOS is false, it loops back to the beginning of the loop. If true, it exits the loop and continues on.

Forth also has the equivalent of C-like for loops with its `do loop` functions. Again, this may only be used inside a user defined function. It uses the syntax:

```
stop start do loopProcedure loop
```

where `stop` is the index the loop stops on (it does NOT execute the loop procedure when the index is equal to `stop`), `start` is the first loop index, and `loopProcedure` is any functions or variables that you want to execute inside the loop. There is a special function `i` that may only be called from within a loop which pushes the loop index to the TOS. The functions `j` and `k` behave similarly, except they return the indices of the second and third nested loops respectively when using nested loops. The `i` function always returns the index of the deepest (or most frequent) loop.

Below is an example Forth program that prints the numbers from 0 to 9 using a `do loop` procedure:

```
: looptest 10 0 do i . loop ;
```

## 9 Software Development and Build Process

### 9.1 The RISC-V Toolchain and its Extensions

To compile software for the VestaRV CPU core, you will need the RISC-V GNU toolchain, an open source compiler suite supporting multiple programming languages. The base instruction set, RV32I (Integer), provides fundamental integer arithmetic operations, excluding multiplication, division, and modulo. All RV32I instructions are 32 bits in length without exception.

The Castalia MCU's VestaRV core implements several standard and extended instruction sets beyond the base RV32I:

- **M Extension (Integer Multiplication and Division):** Adds hardware multiply (`mul`), divide (`div`), and modulo/remainder (`rem`) instructions with both signed and unsigned variants. Without this extension, the compiler must emulate these operations in software, significantly impacting performance.
- **A Extension (Atomic Instructions):** Provides atomic memory operations including load-reserved/store-conditional (`lr.w/sc.w`) and atomic read-modify-write operations (`amoadd.w`, `amoswap.w`, etc.). While primarily designed for multiprocessor synchronization, these instructions ensure atomic operations in interrupt-driven single-core systems.
- **C Extension (Compressed Instructions):** Introduces 16-bit compressed encodings for frequently used instructions, reducing program size by approximately 20-30% according to RISC-V documentation. Uncompressed instructions on non-4-byte-aligned addresses may incur additional execution cycles. This extension significantly improves code density without sacrificing functionality.
- **Zba Extension (Address Generation):** Provides shift-and-add instructions (`sh1add`, `sh2add`, `sh3add`) optimized for efficient array indexing and pointer arithmetic, particularly beneficial for accessing elements in arrays with 2, 4, or 8-byte strides.
- **Zbb Extension (Basic Bit Manipulation):** Includes instructions for bit counting (`clz`, `ctz`, `cpop`), logical operations with inverted operands (`andn`, `orn`, `xnor`), min/max operations, sign/zero extension, rotations, and byte manipulation. These instructions accelerate bit-level algorithms, cryptographic primitives, and data packing/unpacking.
- **Zbc Extension (Carryless Multiplication):** Implements carryless multiplication instructions (`clmul`, `clmulh`, `clmulr`) essential for efficient CRC calculations, error detection codes, and certain cryptographic operations such as Galois field arithmetic.
- **Zbs Extension (Single-Bit Manipulation):** Offers single-bit set, clear, invert, and extract operations (`bset`, `bclr`, `binv`, `bext`) with both register and immediate variants, facilitating efficient manipulation of bit fields and flags.

When compiling for the Castalia MCU, specify the complete architecture string `-march=rv32imac_zba_zbb_zbc_zbs` to enable all supported extensions. Individual extensions may be selectively disabled by omitting them from the `-march` flag if desired for compatibility testing or code size optimization.

### 9.2 Getting the RISC-V Toolchain on Linux

There are two primary methods for obtaining the RISC-V toolchain on Linux: installing a prebuilt binary distribution or building from source. For most users, the prebuilt xPack distribution is recommended due to its simplicity and time savings.

### 9.2.1 Option 1: Prebuilt xPack Toolchain (Recommended)

The xPack project provides precompiled RISC-V GCC toolchains with support for all standard extensions including RV32IMAC and bit manipulation (Zb\*). This is the fastest and simplest installation method.

1. Download the latest xPack RISC-V Embedded GCC release from:

<https://github.com/xpack-dev-tools/riscv-none-elf-gcc-xpack/releases>

Select the appropriate package for your system (typically `xpack-riscv-none-elf-gcc*-linux-x64.tar.gz` for 64-bit Linux systems).

2. Create a directory for the toolchain and extract the archive:

```
mkdir -p ~/riscv-toolchain
```

```
tar -xf xpack-riscv-none-elf-gcc-*.tar.gz -C ~/riscv-toolchain/
```

3. Set the toolchain path as an environment variable. Add the following to your `~/.bashrc` or `~/.profile` file:

```
export RISCVC_TOOLCHAIN_DIR=~/riscv-toolchain/xpack-riscv-none-elf-gcc-13.2.0-2
```

```
export PATH=$RISCVC_TOOLCHAIN_DIR/bin:$PATH
```

Replace 13.2.0-2 with your downloaded version number.

4. Reload your shell configuration or open a new terminal:

```
source ~/.bashrc
```

5. Verify the installation:

```
riscv-none-elf-gcc --version
```

You should see the GCC version information confirming successful installation.

### 9.2.2 Option 2: Building from Source

Building the toolchain from source provides maximum flexibility and ensures the latest features, but requires significantly more time and disk space. You can find the source code for the RISC-V toolchain at <https://github.com/riscv/riscv-gnu-toolchain>, which also contains instructions for building the toolchain. The following procedure builds a toolchain with full support for RV32IMAC and bit manipulation extensions.

First, install the required build dependencies on Ubuntu or Linux Mint:

```
sudo apt install autoconf automake autotools-dev curl libmpc-dev
libmpfr-dev libgmp-dev gawk build-essential bison flex texinfo gperf
libtool patchutils bc zlib1g-dev libexpat-dev
```

The instructions are as follows:

1. Switch to your home directory with

```
cd ~
```

2. Make a new directory with

```
mkdir riscv-download
```

and go into it with

```
cd riscv-download
```



3. Download the RV32IMC toolchain with

```
git clone --recursive https://github.com/riscv/riscv-gnu-toolchain riscv-gnu-toolchain-rv32imc
```

4. Enter into the new directory with

```
cd riscv-gnu-toolchain-rv32imc
```

5. Create a build directory with

```
mkdir build
```

Enter into the new build directory with `cd build`

6. Configure the build with

```
../configure --with-arch=rv32imc --prefix=/opt/riscv/rv32imc
```

7. Begin the build process with

```
sudo make
```

This will take quite a while. If you wish to speed up the process, you can allocate more CPU cores to the build process by using `make -jn` instead of `make` where `n` can be replaced by the number of CPU cores you want to use to build the toolchain.). Once the build process has finished successfully, the toolchain will be located at `/opt/riscv/rv32imc`

8. Add a permanent environment variable to your system that defines the location of the `riscv` directory (perhaps in your `.bashrc` script) with something like

```
export RISCV=/opt/riscv
```

You may also wish to add the `/opt/riscv/rv32imc/bin` directory to your `PATH`, though this is optional.

9. You may now safely delete the `riscv-download` directory with

```
sudo rm -rf ~/riscv-download
```

### 9.3 Getting the RISC-V Toolchain on Windows

The RISC-V toolchain has no official support for Windows, but several options exist to get it working. For Windows 10 users, it is recommended to install the Windows Subsystem for Linux (WSL) in Ubuntu mode, which can be found in the Microsoft Store app. Then, in WSL, follow the installation instructions in Section 9.2. You must then use WSL to compile all software for the RISC-V MCU. If you have an older version of Windows, it is suggested that you use an Ubuntu or Linux Mint virtual machine to build the RISC-V toolchain and compile your MCU software.

### 9.4 Compilation

The RISC-V toolchain includes a GCC compiler. Thus, the compilation process is similar to other toolchains that use GCC. To compile a source file into an object file, use:

```
riscv32-unknown-elf-gcc -c $CFLAGS srcFile -o objectFile.o
```

or for the xPack toolchain:

```
riscv-none-elf-gcc -c $CFLAGS srcFile -o objectFile.o
```

where `$CFLAGS` should be replaced by any compilation flags, `srcFile` is the source code file (C, C++, assembly, or other GCC-supported languages), and `objectFile.o` is the output object file. The compiler determines which language is used in a source file by the file extension (`.c` is for C, `.cpp` is for C++, `.S` is for assembly code which must be preprocessed, and `.s` is for assembly code which is not preprocessed).

There are several compilation flags that are suggested:

- `-march=rv32imac_zba_zbb_zbc_zbs` Architecture flag. This specifies which ISA extensions are enabled during compilation. For the Castalia MCU with full VestaRV capabilities, use the complete architecture string as shown to enable the base integer instructions (I), hardware multiplication/division (M), atomic operations (A), compressed instructions (C), and all bit manipulation extensions (Zba, Zbb, Zbc, Zbs). Individual extensions may be selectively disabled by omitting them from the string (e.g., `-march=rv32imc` for basic support without atomics or bit manipulation, or `-march=rv32imac` to include atomics but exclude bit manipulation extensions).
- `-mabi=ilp32` Application Binary Interface (ABI) flag. Specifies the calling convention and data model. Use `ilp32` for integer-only ABI without floating point support, which is appropriate for the VestaRV core.
- `-Os` Optimize for program size flag. Attempts to minimize the compiled program size, which is particularly important for memory-constrained embedded systems.
- `-O<number>` Optimization level flag. Controls compiler optimization aggressiveness. Common values are `-O0` (no optimization, useful for debugging), `-O1` (basic optimizations), `-O2` (recommended for production), `-O3` (aggressive optimizations, may increase code size), and `-Og` (optimize for debugging experience).
- `-I <include_directory>` Include directory flag. Adds the specified directory to the list of directories searched for header files. Any files in the include directory may be included with the `#include <filename.h>` preprocessor directive.
- `-g` GDB debugger enable flag. Includes debugging symbols in the compiled output. While the VestaRV core does not support hardware debugging, adding the `-g` flag to both the compiler and linker commands will allow disassembly listings (`.lst` files) to show the correspondence between source code and generated assembly instructions.
- `-Wall -Wextra` Warning flags. Enable comprehensive compiler warnings to catch potential issues during development. The `-Wall` flag enables common warnings, while `-Wextra` adds additional checks.

#### Example compilation command for the Castalia MCU:

```
riscv-none-elf-gcc -c \
-march=rv32imac_zba_zbb_zbc_zbs \
-mabi=ilp32 -O2 -Wall -Wextra \
-I./include -g \
main.c -o main.o
```

You must compile every source file that you wish to use in your project. If your project is in C or C++, it is suggested that you use the provided `start.S` assembly file to call your main function. This assembly file needs to have its first instruction at address `0x8200`, so it must be the first file to be given to the linker. You must have one and only one `int main()` function throughout all of your source files in a single project. For C++ projects, you need to start the main function with `extern "C" int main()` to prevent the C++ compiler from name-mangling the main function.

## 9.5 Linking

Once all source files have been compiled, they must be linked to produce the output program binary file, the `.elf` file. The linker assigns memory addresses to every variable and function. The MCU contains no writable non-volatile memory, so the program and the variables all share the RAM (with a notable exception for the bootloader and Forth interpreter stored on the ROM). The linker must be aware of the RAM and ROM start address and size, the interrupt vector table (IVT) start address and size (along with the address of each ISR pointer within the IVT), the first program instruction address that the bootloader jumps to after loading the program into RAM, and the address of the master interrupt handler.

Defining the aforementioned memory segments and addresses is done using a linker script. It is highly suggested to use the provided linker script `MCU.ld` and its include files and tweak it to suit your need.

To execute the linker, use:

```
riscv32-unknown-elf-gcc $LFLAGS $OBJECTS -o program.elf
```

where `$LFLAGS` is a list of linker flags, `$OBJECTS` is a list of all the compiled object files needed for the project, and `program.elf` is the output ELF binary containing the program.

There are several linker flags that are suggested:

- `-march=rv32imac_zba_zbb_zbc_zbs` Architecture flag. This specifies which extensions are enabled in the linking process. The VestaRV core supports the RV32IMAC instruction set plus the Zba, Zbb, Zbc, and Zbs bit manipulation extensions. This flag must match the architecture flags used during compilation.
- `-flto` Link-time optimization flag. This instructs the linker to perform optimizations to the final binary.
- `-nostdlib` Disables the use of the standard library for linking, decreasing software bloat. However, using the flag precludes using standard library functions such as software emulation of integer multiplication and division, floating point numbers, initialization routines, memory allocation, etc. It is suggested that you only use this flag if you have a specific need to reduce the software size.
- `-T linker_script.ld` Path to the linker script
- `-Map mapfile.map` Path to the output map file, which contains the symbol table for your project with their addresses
- `-L linkerfiles_dir` Path to the linker scripts directory that contains any files that are included in the main linker script
- `-g` GDB debugger enable flag. While the VestaRV core does not support hardware debugging, adding the `-g` flag to both the compiler and linker commands will allow the `.lst` file to show the assembly commands your code compiles into.

## 9.6 Intel Hex File Generation

Once the final ELF file has been created from the linker, an Intel hex file can be created, which converts the binary program data into an easy-to-read ASCII file. Furthermore, the Python package `IntelHex` can read the file, allowing you to use Python to upload a new program to the SPI flash. A Python program that utilizes `IntelHex` to upload new programs, called `forthprog.py`, is provided.

## 9.7 A Note on Using C++

C++ generates some extra data sections that C and assembly do not use. In particular, it generates the `.eh_frame` section, which is used for exception handling. You may not need exception handling, in which case you can pass in the compiler flags:

```
-fno-exceptions -fno-unwind-tables -fno-asynchronous-unwind-tables
```

which will significantly reduce the size of `.eh_frame`. If you wish to eliminate it entirely, you can create a phony section in the linker script at an unused memory address and point the `.eh_frame` section to it.

## 10 Uploading Program to SPI Flash

### 10.1 SPI Flash Boot Process

In order to boot in SPI flash mode, there must first be a valid program stored in the SPI flash memory in the proper location. In SPI flash mode, the bootloader reads the data in the SPI flash memory starting at address 0x8200 and copies every 32-bit word into RAM, also starting at address 0x8000. This particular address is the beginning of the RAM on the MCU. The bootloader stops copying from the SPI flash once completely fills the RAM.

Note the distinction between copying 32-bit words as opposed to 8-bit bytes. Though bytes and words are both MSB-first, they are also transmitted starting with the lowest address to the greatest address from the SPI flash. If the bootloader copied every 8-bit byte, the transmission sequence it would expect would be:

byte0, byte1, byte2, byte3, byte4, byte5, byte6, byte7 ...

However, because the bootloader expects MSB-first 32-bit words, the sequence must be:

byte3, byte2, byte1, byte0, byte7, byte6, byte5, byte4 ...

Therefore, the program data on the SPI flash must follow the 32-bit word sequence.

### 10.2 Upload Process

The Forth boot mode has a mechanism for writing data to the SPI flash, which may be used to upload a new program for use in the SPI flash boot mode. When in Forth boot mode, the flash write `fw` function writes a page of data (which is exactly 256 bytes long and begins on an address that is a multiple of 256) to the SPI flash. The Forth function is used with the syntax:

```
bin addr fw
```

If `bin` is true (nonzero), the command expects 256 bytes of binary data to be transmitted over the UART. If false, it expects a 512-character long ASCII hex string representing the data to be transmitted over the UART. The `addr` argument must be the address of the beginning of the 256-byte page, and must be aligned to a 256-byte boundary.

To execute the command, a newline (or line feed) character must be transmitted to the MCU over the UART. Once the command begins executing, it initiates some handshaking to ensure a proper transaction. The MCU prints a `$` character to indicate it is ready for the page data to be transmitted. After that, you must transmit the page data in the format specified by the `bin` argument. Remember to transmit the data in the proper order (byte3, byte2, byte1, byte0, byte7, byte6, byte5, byte4 ...). Once the page data has been transmitted, the MCU will compute a CRC value upon the data using the CRC16\_CDMA2000 algorithm (polynomial = 0xC857, initial CRC value = 0xFFFF, no final XOR, no data reflection, no output reflection) and then transmit the CRC value as a 4-character long hex string. If this value matches your own CRC computation (or if you do not care and simply want it to write the data), transmit a single `Y` character without anything else to tell the MCU to write the data to the SPI flash. Any other character will abort the process, leaving the SPI flash memory unchanged. You are technically supposed to wait an average of 10 ms (25 ms at the greatest) before writing more data to the SPI flash. However, the substantial handshaking and transmission times likely fulfill this requirement, and it is likely that no delay is needed.

If the entire page is blank, you may instead use the flash erase `fe` function to erase the page, making every byte in the page equal to 0xFF. This is much quicker than writing data to the SPI flash, and can speed up programming times substantially for smaller programs. The syntax to use it is:

```
conf addr fe
```

If the `conf` parameter is equal to 123, the function will erase the 256-byte page at address `addr` (which must be 256-byte aligned) and then return 1. If not equal to 123, it will not erase the page and will return 0. You are technically supposed to wait an average of 6 ms (25 ms at the greatest) before another erase

or write operation. Because the erase function is much quicker than the write function, it may become necessary to delay between erases at higher bauds.

You may read back the data stored on the SPI flash with the `fr` function. See Section 8.8 for details.

In order to write an entire program to the SPI flash, you must write/erase a total of 64 pages (each with a length of 256 bytes) beginning with page address 0x8000. After the data is written, you may execute the newly uploaded program by putting the MCU in reset, setting the BOOT pin to SPI flash mode, and taking the chip out of reset. Or, you can simply set the BOOT pin and move the program counter to address 0x0000, which may be done with the `rst` Forth function.

It is highly suggested to use the provided Python program `forthprog.py` to upload new programs to the SPI flash, which takes an Intel hex file containing the new program and sends it to the MCU and the SPI flash using the process described above.

## 11 GPIOx Peripheral

The MCU has several GPIOx peripherals, each of which controls a set of general purpose input/output (GPIO) pins. The VestaRV implementation supports GPIO ports with 8, 16, or 32 pins; every port on this device has 8 pins. Each GPIO peripheral (typically called a port) constitutes a parallel port, allowing all pins in the port to be updated simultaneously. Each GPIO pin may also be configured and updated individually, allowing for design flexibility. Each GPIO pin is referred to as Px.y, where x refers to the port number and y refers to the corresponding bit in the GPIO registers that controls the pin. For example, P2.5 refers to port 2, bit 5. Like every peripheral on this multi-core device, the GPIO ports live in the shared peripheral window and are accessible to all harts through the bus arbiter.

Every GPIO pin on the MCU can be in one of two modes: primary/GPIO mode, or alternate function (peripheral) mode. In alternate function mode, each pin further selects *which* of up to 8 alternate functions (AF0 through AF7) governs it, using the pin's field in the PxAFS register. Section 2 details the alternate functions available at each GPIO pin.

### 11.1 Primary/GPIO Mode Functionality

In primary/GPIO mode, the GPIO pin state is controlled by the GPIO registers PxDIR, PxOUT, and PxREN. The PxDIR register determines if the pin is in input or output mode, the PxOUT register determines if the pin outputs a logic high or logic low level when the pin is in output mode, and the PxREN register enables or disables the internal pullup/pulldown resistor. Table 8 shows the various states available to every GPIO pin.

| PxDIR[y] | PxOUT[y] | PxREN[y] | Px.y State                              |
|----------|----------|----------|-----------------------------------------|
| 0        | -        | 0        | High impedance input, resistor disabled |
| 0        | -        | 1        | Pullup/pulldown resistor enabled        |
| 1        | 0        | -        | Output, strongly driven low             |
| 1        | 1        | -        | Output, strongly driven high            |

Table 8: GPIO Configurations

The PxIN register always reads the digital state of the pin, regardless of whether it is in input or output mode, and regardless of whether it is in primary/GPIO mode or alternate function mode. The pin Px.y is at logic low level if PxIN[y] is 0, and is at logic high level if it is 1. Note that PxIN is latched on the falling edge of the memory enable signal to ensure stable readings during memory access.

### 11.2 Register Access Convenience Features

The GPIO peripheral provides convenient set, clear, and toggle registers for efficient bit manipulation without read-modify-write operations:

- PxOUTS: Writing 1 to a bit sets the corresponding output pin high. Writing 0 has no effect. Reading returns the current PxOUT value.
- PxOUTC: Writing 1 to a bit clears the corresponding output pin low. Writing 0 has no effect. Reading returns the inverted PxOUT value.
- PxOUTT: Writing 1 to a bit toggles the corresponding output pin. Writing 0 has no effect. Reading returns the current PxOUT value.

These registers eliminate race conditions in interrupt-driven or multi-threaded code where multiple code paths may modify GPIO outputs.

### 11.3 Alternate Function (Peripheral) Mode

Most GPIO pins have one or more alternate functions, each controlled by one of the peripherals in the MCU. Two registers together determine what drives a pin:

- PxSEL selects the pin's *mode*: if PxSEL[y] is 0, Px.y is in primary/GPIO mode; if it is 1, Px.y is in alternate function mode.
- PxAFS selects *which* alternate function governs the pin while it is in alternate function mode. Each pin owns a 4-bit field in PxAFS (pin y occupies bits 4y+3 down to 4y; only the low 3 bits are implemented). A field value of *n* selects alternate function AF*n*.

PxAFS resets to 0, so out of reset every pin in alternate function mode drives its AF0 function — AF0 is the pin's classic (legacy) peripheral function, and software that only ever touches PxSEL behaves exactly as it did before multiple alternate functions existed.

When in alternate function mode, the selected peripheral function seizes control over the pin's DIR, OUT, and REN controls, overriding PxDIR[y], PxOUT[y], and PxREN[y]. For example, when the SCK1 pin is in alternate function mode with AF0 selected, the SPI1 peripheral controls the state of the pin, and the corresponding PxDIR[y], PxOUT[y], and PxREN[y] registers have no effect until PxSEL[y] returns to 0. If the pin's PxAFS field selects an AF plane for which the pin has no function defined, the pin becomes a high-impedance input.

To make board layout easy, the alternate-function planes are populated generously with a shared pool of relocatable *output* functions — the UART0/UART1 transmitters, the TIMER0/TIMER1 compare (PWM) outputs, and the SPI1 clock and master-out — fanned across all four ports. Each of these output functions is therefore reachable on roughly two dozen pins, so a peripheral output can be routed to whichever pin is most convenient for the PCB. Because there is exactly one instance of each peripheral resource, a given function is *selected* on at most one pin at a time (like the Raspberry Pi's fixed alternate-function map): populating many pins provides placement choice, not duplicate peripherals. Functions belonging to a peripheral that a configuration omits simply leave the pool: their plane slots become unassigned (high-impedance input). Section 2 tabulates every pin's full AF plane assignment.

A handful of plane slots carry *bidirectional bus completions* rather than pool outputs, so that whole buses (not just their output halves) can land on alternate pins: both I2C buses relocate as pairs (SDA0/SCL0 on P1.6/7 or P2.6/7, and SDA1/SCL1 on P2.2/3 or P1.4/5), RX0 on P3.5 AF2 pairs with TX0 on P3.4 AF2 to form a complete relocated UART0, and MIS01 on P3.6 AF7 joins SCK1/MOSI1 on P3.4/P3.5 AF7 to complete an SPI1 master bus on the DTP pins.

For example, to output the T0CMP0 compare waveform (TIMER0 PWM) on pin P1.0 — where it is alternate function 1 — instead of on its home pin P2.0:

1. Write 1 to the pin's PxAFS field: P1AFS |= 0x1 (field of pin 0)
2. Set the pin to alternate function mode: P1SEL[0] = 1

Some pins with alternate functions may only be used as inputs by the peripheral. For example, timer capture pins like T0CAP0 must be manually configured as inputs in PxDIR before use, even when PxSEL[y] is set to enable the alternate function.



### 11.3.1 Relocatable Peripheral Inputs

Peripheral *input* functions (for example UART receivers, timer captures, and the I2C data/clock lines) that appear at more than one pin observe the pad selected by PxAFS *regardless of* PxSEL, mirroring the always-visible behavior of PxIN: a peripheral input is routed from a pin whenever that pin's PxAFS field selects the function's AF plane, and from its home (AF0) pin otherwise. If the same input function is selected at more than one pin simultaneously, a fixed priority applies: candidate locations are checked in a fixed order (the newest alternate location first, then the older alternate, then home — the pin function tables in Section 2 list each function's locations), and the highest-priority selected pin sources the input. Selecting the same input on two pins at once is a software configuration error; the priority merely makes the outcome deterministic. Because input routing ignores PxSEL, a pin can be left in GPIO output mode while its PxAFS field routes the pad into a peripheral input — a useful software loopback for self-test.

Note that output functions may be mapped to (and driven on) multiple pins simultaneously by selecting the function's plane on each pin; each pin's mux is independent.

## 11.4 Pin Interrupts

The VestaRV GPIO implementation provides individual interrupt outputs for each GPIO pin. Each pin can generate an interrupt independently, allowing for flexible interrupt handling through the system's interrupt vector table. Pin interrupts remain available in alternate function mode in addition to any interrupts the governing peripheral may generate.

To enable interrupts on a pin:

1. Unmask the corresponding GPIO pin interrupt in the system interrupt controller
2. Set PxIE[y] to 1 for the desired pin Px.y
3. Configure the interrupt edge using PxIES[y]: 0 for low-to-high (rising edge), 1 for high-to-low (falling edge)

When the pin Px.y experiences the edge selected by PxIES[y] while PxIE[y] is enabled, the interrupt flag PxIF[y] is immediately set to 1 and the corresponding pin-specific ISR is executed. The interrupt flag must be cleared by writing 1 to PxIF[y] before the ISR returns, otherwise the interrupt will re-trigger.

**Important:** Modifying PxIES[y] while PxIE[y] is enabled may immediately generate a spurious interrupt. Always disable the pin interrupt (PxIE[y] = 0) before changing PxIES[y].

The PxIF register is also latched on the falling edge of the memory enable signal to ensure stable readings during interrupt service routines.

## 11.5 Register Description

### 11.5.1 PIN Register

*Register memory slot: 0, offset: 0 bytes, size: 1 byte*

GPIO read pin register. Each bit corresponds to the input logic state of the GPIO pin of the same number. The register is latched on the falling edge of the memory enable signal. Reading a 0 in a bit indicates a logic low pin state; reading a 1 indicates a logic high state. This register always reflects the pin state regardless of pin direction or peripheral select settings.

| 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
|-------|-------|-------|-------|-------|-------|-------|-------|
| IN[7] | IN[6] | IN[5] | IN[4] | IN[3] | IN[2] | IN[1] | IN[0] |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |

### 11.5.2 POUT Register

*Register memory slot: 1, offset: 4 bytes, size: 1 byte*

GPIO output drive register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is set to GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to make the corresponding pin output a logic low value; write a 1 to output a logic high value.

| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
|--------|--------|--------|--------|--------|--------|--------|--------|
| OUT[7] | OUT[6] | OUT[5] | OUT[4] | OUT[3] | OUT[2] | OUT[1] | OUT[0] |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

### 11.5.3 POUTS Register

*Register memory slot: 2, offset: 8 bytes, size: 1 byte*

GPIO output drive set register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is in GPIO (primary) mode in PxSEL. Write a 1 to the desired bit to set the corresponding pin (make the pin output a logic high value). Writing a 0 has no effect. Reading this register is equivalent to reading the output drive register PxOUT.

| 7       | 6       | 5       | 4       | 3       | 2       | 1       | 0       |
|---------|---------|---------|---------|---------|---------|---------|---------|
| OUTS[7] | OUTS[6] | OUTS[5] | OUTS[4] | OUTS[3] | OUTS[2] | OUTS[1] | OUTS[0] |
| rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) |

### 11.5.4 POUTC Register

*Register memory slot: 3, offset: 12 bytes, size: 1 byte*

GPIO output drive clear register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is in GPIO (primary) mode in PxSEL. Write a 1 to the desired bit to clear the corresponding pin (make the pin output a logic low value). Writing a 0 has no effect. Reading this register yields the inversion of the output drive register PxOUT.

| 7       | 6       | 5       | 4       | 3       | 2       | 1       | 0       |
|---------|---------|---------|---------|---------|---------|---------|---------|
| OUTC[7] | OUTC[6] | OUTC[5] | OUTC[4] | OUTC[3] | OUTC[2] | OUTC[1] | OUTC[0] |
| rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) |

### 11.5.5 POUTT Register

*Register memory slot: 4, offset: 16 bytes, size: 1 byte*

GPIO output drive toggle register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is in GPIO (primary) mode in PxSEL. Write a 1 to the desired bit to toggle the corresponding pin state. Writing a 0 has no effect. Reading this register is equivalent to reading the output drive register PxOUT.

| 7       | 6       | 5       | 4       | 3       | 2       | 1       | 0       |
|---------|---------|---------|---------|---------|---------|---------|---------|
| OUTT[7] | OUTT[6] | OUTT[5] | OUTT[4] | OUTT[3] | OUTT[2] | OUTT[1] | OUTT[0] |
| rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) |

### 11.5.6 PDIR Register

*Register memory slot: 5, offset: 20 bytes, size: 1 byte*

GPIO pin direction register. Each bit corresponds to the GPIO pin of the same number. Only has an effect if the pin is configured in GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to set the corresponding pin to input mode; write a 1 to set to output mode.

| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
|--------|--------|--------|--------|--------|--------|--------|--------|
| DIR[7] | DIR[6] | DIR[5] | DIR[4] | DIR[3] | DIR[2] | DIR[1] | DIR[0] |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

### 11.5.7 PIF Register

*Register memory slot: 6, offset: 24 bytes, size: 1 byte*

GPIO interrupt flag register. Each bit corresponds to the GPIO pin of the same number. The register is latched on the falling edge of the memory enable signal. Reading a 0 in a bit indicates there is no pending interrupt for the corresponding pin; reading a 1 indicates there is a new interrupt pending for the corresponding pin. Write a 1 to each bit for which you wish to clear the interrupt flag. Writing 0 has no effect.

| 7       | 6       | 5       | 4       | 3       | 2       | 1       | 0       |
|---------|---------|---------|---------|---------|---------|---------|---------|
| IF[7]   | IF[6]   | IF[5]   | IF[4]   | IF[3]   | IF[2]   | IF[1]   | IF[0]   |
| rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) |

### 11.5.8 PIES Register

*Register memory slot: 7, offset: 28 bytes, size: 1 byte*

GPIO interrupt edge select register. Each bit corresponds to the GPIO pin of the same number. Write a 0 to the desired bit to set the corresponding pin interrupt to trigger on low-to-high (rising) edge; write a 1 to set to high-to-low (falling) edge triggering.

| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
|--------|--------|--------|--------|--------|--------|--------|--------|
| IES[7] | IES[6] | IES[5] | IES[4] | IES[3] | IES[2] | IES[1] | IES[0] |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

### 11.5.9 PIE Register

*Register memory slot: 8, offset: 32 bytes, size: 1 byte*

GPIO interrupt enable register. Each bit corresponds to the GPIO pin of the same number. Write a 0 to the desired bit to disable the pin interrupt; write a 1 to enable the pin interrupt. Each pin has an individual interrupt output that connects to the system interrupt vector table. Interrupts function in both GPIO (primary) and secondary function (peripheral) modes.

| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
|--------|--------|--------|--------|--------|--------|--------|--------|
| IE[7]  | IE[6]  | IE[5]  | IE[4]  | IE[3]  | IE[2]  | IE[1]  | IE[0]  |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

### 11.5.10 PSEL Register

*Register memory slot: 9, offset: 36 bytes, size: 1 byte*

GPIO peripheral select register. Each bit corresponds to the GPIO pin of the same number. Write a 0 to the desired bit to set the corresponding pin to GPIO (primary) mode; write a 1 to set the pin to alternate function (peripheral) mode. When a pin is in alternate function (peripheral) mode, the governing peripheral takes control of the pin output, direction, and resistor enable states, and the PxOUT, PxDIR, and PxREN registers have no effect on the pin. WHICH alternate function governs the pin is selected by the pin's field in the PxAFS register: at reset all PxAFS fields are 0, selecting alternate function 0 (AF0). Pin interrupts remain available when in alternate function (peripheral) mode in addition to any interrupts the governing peripheral may generate. If a pin has no alternate function defined at the selected PxAFS plane, setting PxSEL to 1 will configure the pin as a high-impedance input.

| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
|--------|--------|--------|--------|--------|--------|--------|--------|
| SEL[7] | SEL[6] | SEL[5] | SEL[4] | SEL[3] | SEL[2] | SEL[1] | SEL[0] |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

### 11.5.11 PREN Register

*Register memory slot: 10, offset: 40 bytes, size: 1 byte*

GPIO resistor enable register. Each bit corresponds to the GPIO pin of the same number. Only has an effect if the pin is configured in GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to disable the pin pullup/pulldown resistor; write a 1 to enable the pin pullup/pulldown resistor.

| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
|--------|--------|--------|--------|--------|--------|--------|--------|
| REN[7] | REN[6] | REN[5] | REN[4] | REN[3] | REN[2] | REN[1] | REN[0] |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

### 11.5.12 PAFS Register

Register memory slot: 11, offset: 44 bytes, size: 1 byte

GPIO alternate function select register. One 4-bit field per pin (pin  $y$  occupies bits  $4y+3$  down to  $4y$ ; only the low 3 bits of each field are implemented, the top bit is reserved and reads 0). When a pin is in alternate function (peripheral) mode (PxSEL bit = 1), the value of its PxAFS field selects WHICH alternate function (AF0-AF7) controls the pin. Each pin's available alternate functions are listed in the pin configuration table in Section 2. The register resets to 0, so every pin comes out of reset selecting its AF0 (legacy) function. Selecting an AF plane with no function defined for the pin configures the pin as a high-impedance input. While a pin is in GPIO (primary) mode its PxAFS field has no effect on the pad, but it still routes relocatable peripheral INPUTS: a peripheral input function relocated to this pin (e.g. a UART receiver or timer capture) observes the pin whenever the PxAFS field selects it, regardless of PxSEL.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| Unused | AFS1   |        |        | Unused | AFS0   |        |        |
|        | rw-(0) | rw-(0) | rw-(0) |        | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range      | Description                                               |
|----------|------------|-----------------------------------------------------------|
| Unused   | Bit 31     |                                                           |
| AFS7     | Bits 30-28 | Alternate function select for pin 7 (0 = AF0 ... 7 = AF7) |
| Unused   | Bit 27     |                                                           |
| AFS6     | Bits 26-24 | Alternate function select for pin 6 (0 = AF0 ... 7 = AF7) |
| Unused   | Bit 23     |                                                           |
| AFS5     | Bits 22-20 | Alternate function select for pin 5 (0 = AF0 ... 7 = AF7) |
| Unused   | Bit 19     |                                                           |
| AFS4     | Bits 18-16 | Alternate function select for pin 4 (0 = AF0 ... 7 = AF7) |
| Unused   | Bit 15     |                                                           |
| AFS3     | Bits 14-12 | Alternate function select for pin 3 (0 = AF0 ... 7 = AF7) |
| Unused   | Bit 11     |                                                           |
| AFS2     | Bits 10-8  | Alternate function select for pin 2 (0 = AF0 ... 7 = AF7) |
| Unused   | Bit 7      |                                                           |
| AFS1     | Bits 6-4   | Alternate function select for pin 1 (0 = AF0 ... 7 = AF7) |
| Unused   | Bit 3      |                                                           |
| AFS0     | Bits 2-0   | Alternate function select for pin 0 (0 = AF0 ... 7 = AF7) |

## 12 SPIx Peripheral

The Serial Peripheral Interface (SPI) peripheral is a high speed, full-duplex, synchronous, multipoint serial interface. It transfers one bit at a time. The SPI bus requires three wires (SCK, MOSI, and MISO), along with one CS wire for every slave. The main features of the SPI peripheral are listed below:

- Master or slave mode (SPI0 is master-only; SPI1, in configurations that include it, supports both master and slave modes)
- LSB-first or MSB-first transfers
- 8-, 16-, or 32-bit transfer lengths
- Master mode supports SPI modes 0-3
- Slave mode supports SPI modes 1 and 3 (CPHA = 1 only)
- Optional byte swap for multi-byte transfers with systems of different endiannesses
- Programmable transfer rate with configurable baud rate divider
- Continuous transfer operation to reduce dead time between frames
- Interrupts for transfer complete and TX register empty
- Flash extended memory feature on SPI0 for memory-mapped read access to external SPI flash (the boot flash), available to hart 0

### 12.1 SPI Pins

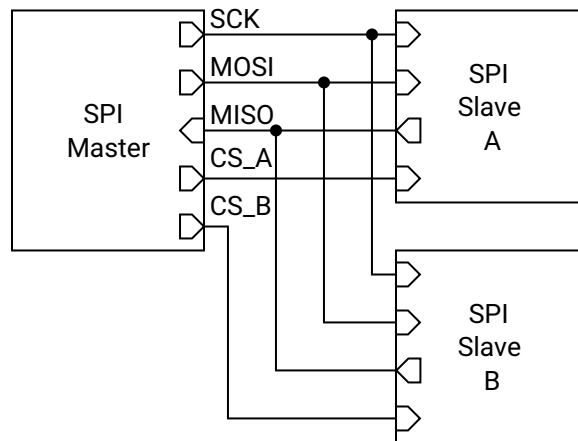


Figure 5: SPI Connection Diagram

The SPI bus has four pins: SCKx, MOSIx, MISOx, and CSx (present only on SPI1). A connection diagram for the SPI bus is shown in Figure 5.

SCKx is the serial clock pin, which governs when each data bit is both latched and updated. It must be attached to the SCK pin on every other device that shares the SPI bus. When the corresponding PxSEL[y] bit is a 1, the SPIx peripheral seizes the DIR, OUT, and REN controls of SCKx.

MOSIx is the master out slave in pin, which transfers each data bit from the master to the slave. It is an output when the SPI port is enabled and in master mode, otherwise it is a high impedance input. It must be attached to the MOSI pin on every other device that shares the SPI bus. When the corresponding PxSEL[y] bit is a 1, the SPIx peripheral seizes the DIR, OUT, and REN controls of MOSIx.

MISOx is the master in slave out pin, which transfers each data bit from the slave to the master. It is an output when the SPI port is enabled, in slave mode, and the CSx pin is asserted (low), otherwise it is a high impedance input. It must be attached to the MISO pin on every other device that shares the SPI bus. When the corresponding PxSEL[y] bit is a 1, the SPIx peripheral seizes the DIR, OUT, and REN controls of MISOx.

CS1 is the chip select pin for the SPI1 peripheral (SPI0 is master-only and has no CS input). It is a high impedance input that waits to be asserted (pulled low) by an external master when SPI1 is in slave mode, which begins the SPI transfer. One CS wire needs to be connected between a GPIO pin of the master and the CS pin of the slave for every slave on the bus. Each slave CS wire must be attached to a dedicated GPIO pin on the master in a one-to-one fashion. The SPI1 peripheral never seizes control of the CS1 pin (regardless of the state of the corresponding PxSEL[y] bit), and as such it must be configured as a high impedance input manually.

SPI0 additionally owns the dedicated CS\_FLASH pin, the chip select of the external boot flash. When its PxSEL[y] bit is a 1, SPI0 drives CS\_FLASH automatically as part of the flash extended memory protocol; when the bit is a 0 it is an ordinary GPIO output, which is how the boot ROM toggles it during the boot sequence.

Note that SPI1's SCK1 and MOSI1 outputs are also members of the shared alternate-function output pool, so they can be relocated onto other GPIO pins through the PxAfS plane selects (see the GPIO chapter).

## 12.2 Shared Access on Castalia

The SPI peripherals live in the shared peripheral window behind the multi-core arbiter (see Section 4), so any hart can use them, each at its original memory-mapped slot: SPI0 at 0x4200, and SPI1 (in configurations that include it) at 0x4300. SPI0 is the boot-flash master: its bus pins are wired to the external SPI flash, it is used by the boot ROM to load programs, and it provides the flash extended memory feature.

The arbiter makes every individual register access atomic: one hart's read or write of SPIxTX, SPIxSR, and so on completes as a whole before any other hart's access is served. Transfer-level ownership, however, is software's responsibility — if two harts push frames at the same time, their frames interleave on the wire and their received words race for SPIxRX. Software that shares an SPI port between harts should serialize whole transfers (or whole packets) with a lock: claim a hardware mutex (MUTEX bank) or an LR/SC session lock in shared RAM, perform the transfer, then release. See Section 4.4.

Beware that some SPI register reads have side effects that now fire on *any* hart's access: reading SPIxRX clears the SPITCIF flag. A hart must therefore never read SPIxRX speculatively while another hart owns the transfer — it would consume the owner's transfer-complete event. When ownership is ambiguous, clear flags explicitly by writing 1s to SPIxSR instead of relying on the read strobe.

The SPI interrupt lines (transfer complete, TX register empty) are wired to hart 0's interrupt enables in the SYSTEM peripheral and can additionally be routed to any other hart through the IRQROUTER peripheral. A hart's interrupt service routine must clear the interrupt flag (through the shared window) before returning.

Note that the SPI core runs in the SMCLK clock domain, and the boot ROM may leave SMCLK sourced from LFXT (32.768 kHz) when it jumps to the application — in that state SCK runs at a small fraction of that frequency and transfers appear to hang. Always configure the SMCLK source in the SYSTEM peripheral (Section 15.1) before using an SPI port, rather than relying on the boot-time clock state.

The flash extended memory region is the one exception to shared access: it is decoded on hart 0's

private bus, not through the arbiter. Only hart 0 can read the flash through it; another hart's access to that address range reads zeros (see Section 12.10).

### 12.3 Generalized SPI Transfer Process

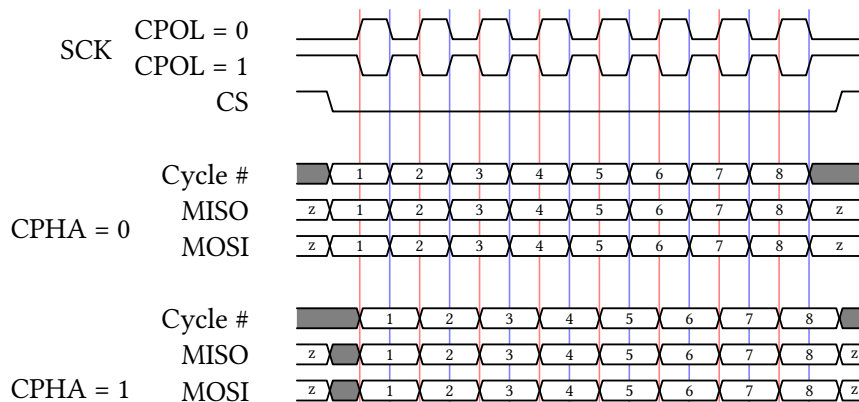


Figure 6: SPI Timing Diagram

When no transfer is occurring, the SPI bus is idle. The transfer begins when the master asserts (pulls low) the CS wire attached to the desired slave. Immediately, the master seizes control of the MOSI wire by making its MOSI pin an output, and the slave does the same for the MISO wire. Note that the master always has control of the SCK and CS wires, which are configured as outputs. Then, the master issues SCK clock pulses in increments of 8, 16, or 32 (one frame of data). On every transition of SCK, both the master and slave alternate between updating and latching the values of MOSI and MISO, the master updating MOSI or latching MISO, and the slave updating MISO or latching MOSI. When the wire is updated, the next bit of data is pushed out of the TX shift register and written to the wire. When the wire is latched, that bit is read from the wire and pushed into the RX shift register. As the SPI port is full-duplex, the master and slave both send data to the other simultaneously. After each frame of data has been transferred, both the master and slave RX shift registers contain their respective received words (the 8-, 16-, or 32-bit data transmitted in the frame). At that point, the master may either continue the process by transferring another frame, or cease the transfer by deasserting (pulling high) the CS wire. The frames that the master issues while the slave's CS wire is low is called a packet, and the master may issue as many frames in a packet as it wants before deasserting CS.

The order of the update/latch process, as well as the polarity of the SCK wire, are determined by two 1-bit parameters: clock polarity (CPOL) and clock phase (CPHA). CPOL determines the idle state and transition direction of SCK, while CPHA determines when the data in MOSI and MISO is updated and latched. These parameters together are referred to as the SPI mode, or SPIMODE, shown in Table 10. Note that this SPI peripheral supports all four SPI modes while in master mode, but only supports SPI modes 1 and 3 while in slave mode (CPHA = 1). Figure 6 shows a SPI timing diagram for all four SPI modes, with a single-frame packet with 8-bit frames.

Note that during the very first frame of a transfer, the slave is required to send data to the master on the MISO wire, regardless of if it has any valid data to send. If it has no data to send, a slave will issue a “dummy” word, which the master must ignore.



| SPIMODE | CPOL | CPHA | SCK Idle | Data Update Edge | Data Latch Edge  |
|---------|------|------|----------|------------------|------------------|
| 0       | 0    | 0    | Low      | Trailing/Falling | Leading/Rising   |
| 1       | 0    | 1    | Low      | Leading/Rising   | Trailing/Falling |
| 2       | 1    | 0    | High     | Trailing/Rising  | Leading/Falling  |
| 3       | 1    | 1    | High     | Leading/Falling  | Trailing/Rising  |

Table 10: SPIMODE Key

## 12.4 Bit and Byte Ordering

The SPI peripheral has the ability to transfer the most significant bit (MSB) first or the least significant bit (LSB) first using SPIMSB. When SPIMSB is 0, the LSB is transferred first, and when SPIMSB is 1, the MSB is transferred first.

For 16- and 32-bit transfers, the byte ordering may optionally be swapped using SPITXSB and SPIRXSB, which swap the bytes being transmitted and received, respectively. For example, during a 16-bit transfer, if SPITXSB is enabled, then byte 1 of SPITX will be transmitted followed by byte 0. If SPIRXSB is enabled, then the first byte received will be placed in byte 1 of SPIRX, and the next 8 bits received will be placed in byte 0. In a 32-bit transfer, the byte order would thus be 3, 2, 1, 0. Figure 7 illustrates this ordering.

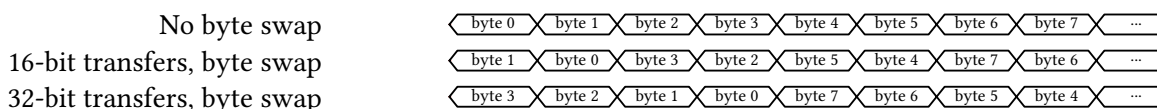


Figure 7: SPI Byte Ordering

Note that the bits in the frame are ordered MSB-first or LSB-first *before* the byte swap operation takes place. As such, a 16-bit, MSB-first transfer with byte swapping activated would transmit the bits 7 down to 0, and then 15 down to 8. Figure 8 shows the bit ordering for a 16-bit transfer.

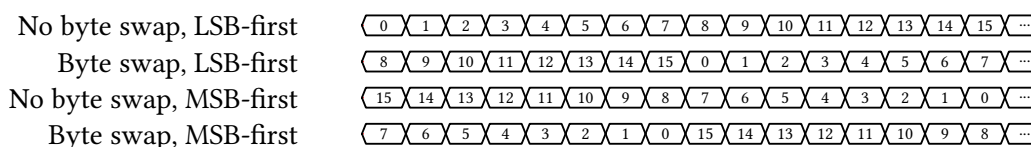


Figure 8: SPI Bit Ordering (16-bit transfer)

## 12.5 SPI Baud Clock

When in master mode, the frequency of the SCKx clock pin is determined by the frequency of SMCLK and the SPI baud control SPIBR, given by the expression:

$$f_{\text{SCK}} = \frac{f_{\text{SMCLK}}}{2 \times (1 + \text{SPIBR})}$$

Note that the maximum frequency of SCKx is half that of SMCLK.

## 12.6 Transmit Buffer Register Queue

Writing to the SPI transmit buffer register SPIxTX queues up the word written to it as the next frame to transmit. If in master mode and a transfer is not currently ongoing (i.e., SCKx is not clocking), the SPI

peripheral immediately begins transmitting the word written to SPIxTX, and SPITEIF is set immediately. If a transfer is currently ongoing, the SPI peripheral waits for the current frame to finish transferring. As soon as it finishes, the SPI peripheral begins transmitting the new frame written to SPIxTX and sets SPITEIF. If in slave mode, the SPI peripheral latches and begins transferring the last word written to SPIxTX and sets SPITEIF whenever the master starts the first clock cycle of SCKx.

The TX queue allows the SPI peripheral to constantly transfer data without any dead time in between frames, ensuring maximum transfer speed. However, the user must take care not to overflow the queue, which happens if the SPIxTX is written two or more times during a single frame transfer. Queue overflows can be prevented by waiting for the SPITEIF flag to be 1 or SPIBUSY to be 0 before writing to SPIxTX.

## 12.7 Flags and Interrupts

The SPI peripheral has two interrupt flags (SPITCIF and SPITEIF) and one status indicator (SPIBUSY). If the corresponding interrupts are enabled in the SPIxCR register and the SPIx ISR is not masked, the interrupt flags will generate interrupts when set to 1. Both interrupt flags must be cleared before the ISR returns to prevent spurious re-execution.

The SPITCIF interrupt flag is set when a SPI transfer completes. It can be cleared by writing 1 to it or by reading the SPIxRX register (see the shared-access note above: the read-clear fires on any hart's read).

The SPITEIF interrupt flag is set when the SPIxTX transmit buffer becomes empty and ready to accept new data. In master mode without an ongoing transfer, it sets immediately when writing to SPIxTX. During an active transfer, it sets when the current frame completes and the queued data begins transmission. In slave mode, it sets when the master begins the first SCK clock cycle. This flag must be cleared by writing 1 to it.

The SPIBUSY indicator reflects the peripheral's transfer state. In master mode, it is 1 during active frame transmission (SCK actively clocking) and 0 when idle. Note that SPIBUSY can be 0 between frames if no new data is queued before the previous frame completes. In slave mode, SPIBUSY is 1 when the CSx pin is asserted (low) and 0 when deasserted (high).

Note that flash extended memory reads (Section 12.10) are handled entirely in hardware and do not set SPITCIF or SPITEIF.

## 12.8 Master Mode Transfer Procedure

To use the SPI peripheral in master mode, the user must first initialize the SPIx peripheral by configuring the SPIxCR register. Of note, SPIEN must be 1 and SPIISM must be 0 to enable the SPI peripheral in master mode. Then, the GPIO pins multiplexed with SCKx, MISOx, and MOSIx must be set to secondary/peripheral mode with their respective bits in the PxSEL registers. Finally, the GPIO pins attached to each slave's CS pin need to be set to output a logic high voltage (note that these CS pins are *not* the same as SPI1's own CS1 pin, which is used when this device is in slave mode).

To initiate a SPI transfer, the master should first clear the SPI status register SPIxSR by writing 0 to it, and then must assert (pull low) the slave's CS pin. Then, the master may begin transferring as many frames as it wants in the packet. To send a frame, the master must first check if the SPIxTX register is empty (i.e. ready to accept a new word to queue for transfer) by ensuring that either SPITEIF is 1 or SPIBUSY is 0. Then, the master may write the next word to transmit to SPIxTX. Next, if SPITCIF is 1, the master should read the value of the SPIxRX register to read the value the slave transmitted on the prior frame. Then, the master must clear the status register and may either start another frame or end the packet by deasserting (pulling high) the CS wire.

A simplified procedure for producing master mode SPI transfers is shown in Listing 1 that does not use the TX queue for continuous transfers.

```

1  /** Includes **/
2  #include <MemoryMap.h>
3
4  /** Function Declarations **/
5  uint32_t spi_transfer(uint32_t tx_data)
6
7  /** Main Function **/
8  int main()
9  {
10     // Initialize the SPI peripheral
11     SPIxCR = SPIEN;
12
13     // Configure the SCKx, MOSIx, and MISOx pins
14     SCKx_PxSEL |= SCKx_BIT;
15     MOSIx_PxSEL |= MOSIx_BIT;
16     MISOx_PxSEL |= MISOx_BIT;
17
18     // Configure the GPIO pin attached to the slave's CS pin as an output high
19     PxSEL &= ~(BITy);
20     PxOUT |= BITy;
21     PxDIR |= BITy;
22
23     // Assert the CS pin
24     PxOUT &= ~(BITy);
25
26     // Do a single SPI transfer
27     uint32_t tx_data = 57;
28     uint32_t rx_data;
29
30     rx_data = spi_transfer(tx_data);
31
32     // Deassert the CS pin
33     PxOUT |= BITy;
34
35     return 0;
36 }
37
38 /** Function Definitions **/
39 uint32_t spi_transfer(uint32_t tx_data)
40 {
41     // Begin a SPI frame by writing the transmit data to SPIxTX
42     SPIxTX = tx_data;
43
44     // Wait for the frame to finish transmitting
45     while (SPIxSR & SPIBUSY) {}
46
47     // Return the received data from the slave
48     return SPIxRX;
49 }

```

Listing 1: Example Program for SPI Master Transfers

## 12.9 Slave Mode Transfer Procedure

Slave mode is available only on SPI1, in configurations that include it (SPI0 is master-only). To use SPI1 in slave mode, the user must first initialize the peripheral by configuring the SPI1CR register. Of note, SPIEN must be 1 and SPIISM must be 1 to enable the SPI peripheral in slave mode. Then, the GPIO pins multiplexed with SCK1, MISO1, and MOSI1 must be set to secondary/peripheral mode with their respective bits in the PxSEL registers. Finally, the CS1 pin must be set as a high impedance input in GPIO mode to allow the master to assert this SPI1 peripheral.

Immediately after initialization, a word should be written to the SPI1TX register, which will be transmitted to the master when the master asserts the CS1 pin and begins sending SCK pulses. Whatever data

is contained in SPI1TX will always be transmitted to the master when the master begins each frame regardless of if the slave has updated the value of SPI1TX, so care must be taken to always put valid data in the SPI1TX register before the master begins the next frame. Once the master begins sending a frame, the SPITEIF flag will become 1, indicating that the next word to transmit to the master should be written to SPI1TX. At the end of the frame, the SPITCIF flag will become 1, and the data in SPI1RX will contain the data received from the master. SPI1RX should then be read, and the status register SPI1SR must be cleared. Note that the SPI1RX register must be read before the end of the next frame, else the data will be overwritten and unrecoverable.

There is no dedicated interrupt indicating when the CS1 pin is asserted or deasserted, but this functionality can be achieved using the ordinary GPIO pin interrupts.

## 12.10 External SPI Flash Extended Memory Access

The SPI0 peripheral includes a flash extended memory feature that enables memory-mapped read access to the external SPI flash memory. When enabled via the SPIFEN control bit, this feature allows hart 0 to read from the SPI flash using standard load instructions (and to execute code in place from it), as if the flash were directly addressable ROM on the memory bus. This capability is designed to support programs larger than the internal RAM capacity, up to the full size of the SPI flash.

The flash region is decoded on hart 0's private bus, *not* through the multi-core arbiter: memory accesses by hart 0 at or above the extended flash base address (0x200000) are automatically translated to SPI flash read operations. The other harts have no path to the flash — their accesses in this range read zeros (without stalling). The SPI0 peripheral manages the flash read protocol transparently, including chip select control (via CS\_FLASH), command sequencing, and data retrieval; the hart is stalled for the duration of each flash access.

**Important:** Using flash extended memory significantly reduces execution speed compared to RAM-based execution, as each 32-bit read requires multiple SCK clock cycles to retrieve data from the external flash (and SCK itself runs from SMCLK). This feature is best suited for read-only data, lookup tables, or code sections that are not performance-critical.

### 12.10.1 Flash Address Translation

The SPI0 peripheral translates CPU memory addresses to 24-bit SPI flash addresses using the SPI0FOS (SPI Flash Offset) register. Only the low 24 bits of the CPU address participate; the actual flash address accessed is computed as:

$$\text{Flash Address} = (\text{CPU Address} + \text{SPI0FOS}) \bmod 2^{24}$$

This offset mechanism allows flexible mapping of flash contents to different virtual addresses in the CPU's address space. The 24-bit address wraps at 0xFFFFF, discarding any overflow bits. By modifying SPI0FOS, software can remap flash regions without physically moving data in the flash memory. For example, to make flash address 0 appear at the extended flash base address itself, set SPI0FOS to the two's complement of the base address modulo  $2^{24}$ .

### 12.10.2 Flash Initialization Procedure

Before enabling the flash extended memory feature, the SPI flash must be properly initialized. The initialization sequence depends on the specific flash chip, but typically includes:

1. Configure SPI0 in master mode with appropriate SPI mode (typically SPIMODE3), 8-bit transfers initially, and SCK frequency within flash specifications

2. Set SCK0, MOSI0, and MISO0 pins to secondary/peripheral mode via PxSEL
3. Configure the flash chip select pin (CS\_FLASH) as GPIO output, initially high (deasserted)
4. Wake the flash from deep power-down mode (command varies by flash chip, often 0xAB) — note that the boot ROM deliberately puts the flash *into* deep power-down at the end of every SPI flash boot, so this step is required after a flash boot
5. Configure flash-specific features such as page size or addressing mode using vendor-specific commands
6. Transfer control of CS\_FLASH to the SPI peripheral by setting it to secondary/peripheral mode
7. Enable flash extended memory by setting SPIFEN = 1 in SPI0CR

Once initialized, the flash extended memory feature operates transparently. Read operations by hart 0 in the designated address range automatically trigger SPI flash transactions without software intervention.

### 12.11 SPI0 Boot Behavior

The SPI0 peripheral is used during the boot process to load programs from the external SPI flash memory. All harts reset into the shared boot ROM (see Section 5); hart 0 then samples the BOOT pin to decide the boot mode. If BOOT is low, the interactive Forth interpreter is launched from ROM over UART0. If BOOT is high, the boot ROM uses SPI0 to read the program image from the attached SPI flash, copies its segments into RAM (the load window is 0x8000–0xFFFFC), puts the flash into deep power-down, switches SMCLK to LFXT, and jumps to the program entry point at 0x8200. The other harts remain parked in the ROM until launched through the boot-ROM loader (see Section 4).

During the boot sequence the ROM drives CS\_FLASH (as a GPIO output) and clocks the flash, and the flash drives the MISO0 line. If other slave devices share the SPI0 bus, they must not drive MISO0 during the boot sequence. This conflict can occur if a slave's chip select (CS) pin is controlled by a GPIO pin that defaults to high-impedance input at reset, potentially allowing the slave's CS to float low (asserted state).

To prevent bus contention during boot:

- Use GPIO pins with pull-up resistors or start-high reset behavior to control slave CS pins
- Ensure all non-flash slaves on the SPI0 bus have their CS pins driven high during MCU reset
- Consider dedicating SPI0 exclusively to the boot flash, using SPI1 (in configurations that include it) for other SPI devices
- Add external pull-up resistors to slave CS lines if GPIO reset behavior is insufficient

After the boot sequence completes and the program begins executing, SPI0 becomes available for normal SPI operations, including the flash extended memory feature if enabled by software.

## 12.12 Register Description

### 12.12.1 SPICR Register

Register memory slot: 0, offset: 0 bytes, size: 4 bytes

SPI control register

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| Unused |        |        |        |        |        |        |        |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| Unused |        |        |        | FEN    | SM     | TXSB   | RXSB   |
|        |        |        |        | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| BR     |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| EN     | MSB    | TCIE   | TEIE   | DL     |        | CPOL   | CPHA   |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range      | Description                                                                                                                                                                                                                                                                                                                                                                            |
|----------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unused   | Bits 31-20 |                                                                                                                                                                                                                                                                                                                                                                                        |
| FEN      | Bit 19     | <p>SPI Flash extended memory enable. When enabled, allows native read-only memory access to the SPI flash memory via the system memory bus. The SPI peripheral automatically handles flash read commands and provides transparent memory-mapped access. Available only on SPI0; reads as 0 on SPI1.</p> <p>0 Flash extended memory disabled</p> <p>1 Flash extended memory enabled</p> |
| SM       | Bit 18     | <p>SPI slave mode select. Configures the SPI peripheral for master or slave operation. SPI0 is master-only; this bit has no effect on SPI0. SPI1 supports both master and slave modes.</p> <p>0 Master mode</p> <p>1 Slave mode</p>                                                                                                                                                    |
| TXSB     | Bit 17     | <p>SPI TX swap bytes. Swaps the byte order in 16- and 32-bit transmissions. In 32-bit transmissions, bytes 3 and 0 are swapped and bytes 2 and 1 are swapped. In 16-bit transmissions, bytes 1 and 0 are swapped. Does not affect 8-bit transmissions.</p> <p>0 Bytes not swapped</p> <p>1 Bytes swapped</p>                                                                           |
| RXSB     | Bit 16     | <p>SPI RX swap bytes. Swaps the byte order in 16- and 32-bit receptions. In 32-bit receptions, bytes 3 and 0 are swapped and bytes 2 and 1 are swapped. In 16-bit receptions, bytes 1 and 0 are swapped. Does not affect 8-bit receptions.</p> <p>0 Bytes not swapped</p>                                                                                                              |

|      |           |                                                                                                                                                                                                                                                                                                    |
|------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      |           | 1 Bytes swapped                                                                                                                                                                                                                                                                                    |
| BR   | Bits 15-8 | SPI clock (SCK) baud rate control for master mode. Baud rate = $SMCLK / (2 * (1 + SPIBR))$ . For example, with SMCLK at 24 MHz: SPIBR=0 gives 12 MHz, SPIBR=1 gives 8 MHz, SPIBR=2 gives 6 MHz, SPIBR=5 gives 4 MHz, SPIBR=11 gives 2 MHz, SPIBR=23 gives 1 MHz.                                   |
| EN   | Bit 7     | SPI enable. When disabled, all SPI operations cease and the peripheral is held in reset state.<br>0 Disabled<br>1 Enabled                                                                                                                                                                          |
| MSB  | Bit 6     | Bit endianness select. Determines whether data is transmitted and received MSB-first or LSB-first.<br>0 LSB-first<br>1 MSB-first                                                                                                                                                                   |
| TCIE | Bit 5     | SPI transmit complete interrupt enable. Interrupt triggers when a full SPI transfer (transmit and receive) completes.<br>0 Interrupt disabled<br>1 Interrupt enabled                                                                                                                               |
| TEIE | Bit 4     | SPI transmit register empty interrupt enable. Interrupt triggers when SPIxTX register is empty and ready to accept new data for the next transfer.<br>0 Interrupt disabled<br>1 Interrupt enabled                                                                                                  |
| DL   | Bits 3-2  | SPI transmission data length select. Determines the number of bits transferred per SPI transaction.<br>00 SPIDL_8: 8-bit transfers<br>01 SPIDL_16: 16-bit transfers<br>10 SPIDL_32: 32-bit transfers<br>11 SPIDL_RES: Reserved (do not use)                                                        |
| CPOL | Bit 1     | SPI clock (SCK) polarity. Determines the idle state of the SCK line.<br>0 SCK idles low (SPIMODE0 or SPIMODE1)<br>1 SCK idles high (SPIMODE2 or SPIMODE3)                                                                                                                                          |
| CPHA | Bit 0     | SPI clock (SCK) phase. Determines when data is sampled relative to the SCK edge. In slave mode, only SPICPHA=1 is supported.<br>0 Data sampled on leading edge, shifted on trailing edge (SPIMODE0 or SPIMODE2)<br>1 Data shifted on leading edge, sampled on trailing edge (SPIMODE1 or SPIMODE3) |

### 12.12.2 SPISR Register

Register memory slot: 1, offset: 4 bytes, size: 1 byte

SPI status register. Provides real-time status of SPI transfer operations and interrupt flags.

|        |   |   |   |   |       |         |         |
|--------|---|---|---|---|-------|---------|---------|
| 7      | 6 | 5 | 4 | 3 | 2     | 1       | 0       |
| Unused |   |   |   |   | BUSY  | TCIF    | TEIF    |
|        |   |   |   |   | r-(0) | rw1-(0) | rw1-(0) |

| Bitfield      | Range    | Description                                                                                                                                                                                                                                                                    |
|---------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Unused</i> | Bits 7-3 |                                                                                                                                                                                                                                                                                |
| BUSY          | Bit 2    | Indicates whether a SPI transfer is currently in progress. In master mode, set when a transfer starts and cleared when complete. In slave mode, set when chip select is asserted (driven low) and cleared when deasserted.<br>0 SPI is idle<br>1 SPI transfer in progress      |
| TCIF          | Bit 1    | SPI transfer complete interrupt flag. Set when a SPI transfer completes. Must be cleared by writing 1 to this bit or by reading SPIxRX register.<br>0 No transfer completed<br>1 Transfer completed                                                                            |
| TEIF          | Bit 0    | SPI transmit register empty interrupt flag. Set when SPIxTX register is empty and ready to accept new data. The data will not be transmitted until any current transfer completes. Must be cleared by writing 1 to this bit.<br>0 SPIxTX not empty<br>1 SPIxTX empty and ready |

### 12.12.3 SPITX Register

Register memory slot: 2, offset: 8 bytes, size: 4 bytes

SPI transmit buffer register. In master mode, writing to this register initiates a new SPI transfer. In slave mode, writing to this register queues data to be transmitted during the next master-initiated transfer. Actual number of bits transmitted is determined by SPIDL setting.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| TX     |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| TX     |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| TX     |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| TX     |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description |
|----------|-----------|-------------|
| TX       | Bits 31-0 |             |

### 12.12.4 SPIRX Register

Register memory slot: 3, offset: 12 bytes, size: 4 bytes



SPI receive buffer register. Contains the data received during the most recent SPI transfer. Reading this register also clears the SPITCIF flag. Valid data width depends on SPIDL setting; unused upper bits read as 0.

|       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|
| 31    | 30    | 29    | 28    | 27    | 26    | 25    | 24    |
| RX    |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 23    | 22    | 21    | 20    | 19    | 18    | 17    | 16    |
| RX    |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 15    | 14    | 13    | 12    | 11    | 10    | 9     | 8     |
| RX    |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| RX    |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |

| Bitfield | Range     | Description |
|----------|-----------|-------------|
| RX       | Bits 31-0 |             |

### 12.12.5 SPIFOS Register

*Register memory slot: 4, offset: 16 bytes, size: 4 bytes*

SPI Flash memory address offset. This 24-bit value is added to memory access addresses when flash extended memory mode is enabled (SPIFEN=1). Allows remapping of flash memory to different virtual addresses. The addition wraps around at 0x00FFFFFF. Available only on SPI0; reads as 0 on SPI1.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| Unused |        |        |        |        |        |        |        |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| FOS    |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| FOS    |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| FOS    |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

---

| Bitfield      | Range      | Description |
|---------------|------------|-------------|
| <i>Unused</i> | Bits 31-24 |             |
| FOS           | Bits 23-0  |             |

---

## 13 UARTx Peripheral

The Universal Asynchronous Receiver/Transmitter (UART) is an asynchronous, full-duplex serial interface with hardware parity support. The main features of the UART are listed below:

- Full-duplex, independent receive and transmit operation
- Asynchronous (no serial clock wire)
- Two wires, one for TX and one for RX
- Communicates with one external device
- 8-bit data frames
- Adjustable baud generator with 16× oversampling
- Hardware parity generation and checking (even or odd)
- Error detection: framing error, parity error, and receive overflow flags
- Three separate interrupt sources: receive complete, transmit complete, and TX register empty
- Latched register reads for noise immunity

### 13.1 UART Pins

The UARTx peripheral has two pins: RXx and TXx.

The TXx pin transmits each data bit from this UARTx peripheral to the external device. It must be attached to the RX pin of the external device. When the corresponding PxSEL[y] bit is a 1, the UARTx peripheral seizes control of the DIR, OUT, and REN controls of TXx, making it an output.

The RXx pin receives each data bit from the external device. It must be attached to the TX pin of the external device. When the corresponding PxSEL[y] bit is a 1, the UARTx peripheral seizes control of the DIR, OUT, and REN controls of RXx, making it a high impedance input.

The UART transmitter and receiver are independent of one another. Thus, transmissions and receptions need not occur at the same time or with the same external device. Indeed, both pins do not necessarily need to be used if one-way communications are all that is needed.

A connection diagram for the UART is displayed in Figure 9.

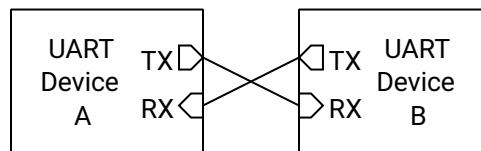


Figure 9: UART Connection Diagram

### 13.2 Shared Access on Castalia

The UARTs live in the shared peripheral window behind the multi-core arbiter (see Section 4), so any hart can use them, each at its original memory-mapped slot: UART0 at 0x4400, and UART1 (in configurations that include it) at 0x4500. UART0 is the *console UART*: it carries the boot-time Forth console.

The arbiter makes every individual register access atomic: one hart’s read or write of UARTxTX, UARTxSR, and so on completes as a whole before any other hart’s access is served. Message-level ownership, however, is software’s responsibility — if two harts transmit at the same time, their *bytes* interleave on the wire. Software that shares a UART between harts should serialize whole messages with a lock: claim a hardware mutex (MUTEX bank) or an LR/SC session lock in shared RAM, transmit the message, then release. See Section 4.4.

The UART interrupt lines (receive complete, TX register empty, transmit complete) are wired to hart 0’s interrupt enables in the SYSTEM peripheral and can additionally be routed to any other hart through the IRQROUTER peripheral. A hart’s interrupt service routine must clear the interrupt flag (through the shared window) before returning.

Note that the UART core runs in the SMCLK clock domain, and the boot ROM may leave SMCLK sourced from LFXT (32.768 kHz) when it jumps to the application — in that state UART frames take roughly 10 ms each. Always configure the SMCLK source in the SYSTEM peripheral (Section 15.1) before using a UART, rather than relying on the boot-time clock state.

### 13.3 Generalized UART Transmission Process



Figure 10: UART Data Frame Timing Diagram

Each UART wire is unidirectional, with the TX pin of the transmitting device driving the logic level of the wire and the RX pin of the receiving device reading the level of the pin. The only distinction between the two wires is the direction that data travels, and as such, this section will only depict one wire.

When no UART transmission is occurring, the UART wire is idle, and the TX pin drives the UART wire high. To start a transmission, the TX pin transmits a start bit by driving the wire low for  $T_B = 1/\text{baud}$  seconds. It then begins transmitting each data bit, LSB-first, every  $T_B$  seconds until all 8 bits have been transmitted. Then, it optionally transmits a parity bit, which helps detect transmission errors. Finally, it transmits a stop bit by driving the wire high for  $T_B$  seconds. After that point, it may begin another transmission by sending another start bit, or it can let the wire be idle.

Figure 10 shows a timing diagram for the UART transmission process. Note that the parity bit [P] is optional.

### 13.4 UART Baud Generation

Both devices communicating over the UART must agree on the baud rate, or the number of bits transmitted per second. In the absence of a clock wire, the UART uses the baud rate to sample the RX wire every  $T_B = 1/\text{baud}$  seconds, recording the logic levels in sequence. The transmitter must send bits at that same rate. Any significant deviation of the baud rate can result in bit errors.

The UART uses 16× oversampling for improved noise immunity and accurate bit detection. The baud rate for the UARTx peripheral is configured through the 12-bit UARTxBR register:

$$\text{UART baud} = \frac{f_{\text{SMCLK}}}{16 \times (\text{BR} + 1)}$$

For example, with SMCLK running at 24 MHz:

- BR = 12 gives 115,385 baud (within 0.2 % of standard 115,200)
- BR = 25 gives 57,692 baud (within 0.2 % of standard 57,600)
- BR = 51 gives 28,846 baud (within 0.2 % of standard 28,800)

### 13.5 Parity Bit

A UART frame has an optional parity bit after the last data bit and before the stop bit. The purpose of the parity bit is to verify that the data received is indeed the data that was transmitted. If the received parity bit does not match the expected parity bit, then the receiver knows that the received data has been corrupted.

If the parity bit is enabled with UPEN, the transmitter calculates the parity of the byte to transmit by taking the exclusive OR of all 8 bits in the transmitted byte, the result of which is then put through a final exclusive OR gate with the parity select bit UPSEL:

$$p = b_7 \oplus b_6 \oplus \cdots \oplus b_1 \oplus b_0 \oplus \text{UPSEL}$$

When UPSEL is 0, the parity is said to be “even”, and when it is 1, “odd”. This operation is functionally counting how many bits are 1s in the transmitted byte. When even parity is selected, the parity bit is 0 when the number of 1s is even and 1 when the number of 1s is odd. That result is inverted when odd parity is selected.

On the receiver end, the parity bit is capable of detecting a single bit error in the transmission. That is, if one of the eight data bits (or the parity bit itself) is incorrectly received, then the parity error flag UPEF will detect a bit error. However, if two bits are incorrectly received, then the parity error flag will not detect the bit error. Note that if the number of bit errors is an odd number, the parity error flag will detect the error, but will not when the number of bit errors is even.

If a parity bit is enabled on the transmitter side, it must also be enabled on the receiver side with UPEN.

### 13.6 Transmit Buffer Register Queue

Writing to the UART transmit buffer register UARTxTX queues up the byte written to it as the next frame to transmit. If a transmission is not currently ongoing, the UARTx peripheral immediately begins transmitting the byte written to UARTxTX, and TEIF is set immediately. If a transfer is currently ongoing, the UARTx peripheral waits for the current frame to finish transferring. As soon as it finishes, the UARTx peripheral begins transmitting the new frame written to UARTxTX and sets TEIF.

The TX queue allows the UARTx peripheral to constantly transfer data without any dead time in between frames, ensuring maximum transfer speed. However, the user must take care not to overflow the queue, which happens if the UARTxTX is written two or more times during a single frame transfer. Queue overflows can be prevented by waiting for the TEIF flag to be 1 or TXBF to be 0 before writing to UARTxTX.

### 13.7 Flags and Interrupts

The UARTx peripheral has five status flags and three interrupt flags in the UARTxSR register. If the corresponding interrupts are enabled in the UARTxCR register and the UARTx ISR is not masked, then the three interrupt flags will generate an interrupt whenever they are equal to 1. Any and all interrupt flags must be cleared before the UARTx ISR returns, else the ISR will mistakenly re-execute immediately upon the prior return.

The UARTxSR and UARTxRX registers are latched on the falling edge of the memory enable signal for improved noise immunity during reads.

The UART receiver busy flag RXBF shows when the UARTx peripheral is currently receiving a byte of data when it is 1. Likewise, TXBF indicates that a transmission is ongoing or pending when 1.

The UART framing error flag FEF indicates that a framing error occurred on the last reception when it is a 1. A framing error typically occurs if the UART transmitter settings do not match the receiver settings, such as baud rate, number of bits to transmit, or the presence/absence of a parity bit. Specifically, a framing error is detected if the receiver does not detect a high level at the expected stop bit time. If FEF is a 1 after a reception, that reception must be treated as unrecoverable, corrupted data. FEF is cleared when the UARTxRX register is read.

The UART parity error flag PEF indicates that a parity error occurred on the last reception when it is a 1. A parity error can only happen if the parity bit is enabled by setting PEN to 1. A parity error occurs when the parity bit appended to the received data byte does not match the expected value calculated from the received data. If PEF is a 1 after a reception, that reception must be treated as corrupted data. PEF is cleared when the UARTxRX register is read.

The UART receive data overflow flag OVF indicates that the user did not read the UARTxRX register in time before another byte of data was received by the UARTx peripheral when it is a 1. In this case, the previous received data has been lost and must be treated as incomplete. OVF is cleared when the UARTxRX register is read.

The UART receive complete interrupt flag RCIF is set to 1 immediately when the UARTx peripheral finishes receiving a frame of data from the transmitter. The new data byte becomes available in the UARTxRX register as soon as RCIF is set. This is also the moment that FEF, PEF, and OVF flags will be updated, if the proper conditions are met. RCIF is cleared when the UARTxRX register is read or by writing a 1 to this bit in UARTxSR.

The UART transmit register empty interrupt flag TEIF is set the moment the UART TX buffer register UARTxTX becomes empty (i.e., is ready to queue up the next byte to transmit). If a frame is not currently being transmitted, the flag is set immediately when a new byte is written to UARTxTX. If a frame is currently being transmitted, it is set as soon as the current transfer is completed, whereupon the value in the UARTxTX register begins to be transmitted. Write a 1 to this bit in UARTxSR to clear TEIF.

The UART transmit complete interrupt flag TCIF is set the moment the UART transmission completes (all bits sent) and the transmitter becomes idle. Write a 1 to this bit in UARTxSR to clear TCIF.

### 13.8 Transmit Procedure

To transmit a byte of data via the UART, first initialize the UARTx peripheral by configuring the UARTxCR register and setting the baud rate with the UARTxBR register. Of note, the UART enable bit EN must be a 1 to activate the UART. When EN is 0, the UART peripheral is held in reset state. The GPIO pin multiplexed with Txx must be set to secondary/peripheral mode in the corresponding PxSEL register.

To transmit a byte of data over the UART, first clear the status register by reading it and discarding its value. Then, wait until the UART is not transmitting (when TXBF is 0) or the UART TX buffer is empty (when TEIF is 1). Next, write the byte to transmit to the UARTxTX register. The byte will be finished transmitting once TXBF is 0.

A simplified procedure for transmitting a byte over the UART is shown in Listing 2.

```

1  /** Includes **/
2  #include <MemoryMap.h>
3
4  /** Function Definitions **/
5  void uartx_putc(char tx_data)
6  {

```

```

7      // Wait for the transmitter to finish any previous transmissions
8      while (UARTxSR & TXBF) { }
9
10     // Send the new data byte
11     UARTxTX = tx_data;
12 }

```

Listing 2: Example Function for UART Transmissions

### 13.9 Receive Procedure

To receive a byte of data via the UART, first initialize the UARTx peripheral by configuring the UARTxCR register and setting the baud rate with the UARTxBR register. Of note, the UART enable bit EN must be a 1 to activate the UART. The GPIO pin multiplexed with RXx must be set to secondary/peripheral mode in the corresponding PxSEL register.

Once done setting up the UART, wait for the UART receive complete interrupt flag RCIF to be 1. Then, read the received byte contained in the UARTxRX register. Reading UARTxRX automatically clears the RCIF, FEF, PEF, and OVF flags.

A simplified procedure for receiving a byte over the UART is shown in Listing 3.

```

1  /** Includes **/
2  #include <MemoryMap.h>
3
4  /** Function Definitions **/
5  char uartx_getc()
6  {
7      // Wait for the UART to receive a new character
8      while ((UARTxSR & RCIF) == 0) { }
9
10     // Return the new character that is in the receive buffer register
11     return UARTxRX;
12 }

```

Listing 3: Example Function for UART Receptions

### 13.10 UART0 Boot Behavior

When booting in Forth interpreter mode, the UART0 peripheral provides an interactive command processor with an external device, such as computer with a USB to UART device attached. When the MCU is powered up or reset and the BOOT pin is low, the Forth interpreter will launch over UART0, which can be used to program or debug the MCU.

Section 5 elaborates on the power up and reset behavior of the MCU.

## 13.11 Register Description

### 13.11.1 UARTCR Register

Register memory slot: 0, offset: 0 bytes, size: 1 byte

UART control register. Controls UART enable, parity configuration, and interrupt enable bits. When UCR.EN is disabled, the UART peripheral is held in reset state.

|        |   |        |        |        |        |        |        |
|--------|---|--------|--------|--------|--------|--------|--------|
| 7      | 6 | 5      | 4      | 3      | 2      | 1      | 0      |
| Unused |   | EN     | PEN    | PSEL   | CIE    | TEIE   | TCIE   |
|        |   | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range    | Description                                                                                                                                                                                        |
|----------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unused   | Bits 7-6 |                                                                                                                                                                                                    |
| EN       | Bit 5    | UART enable. When disabled, the UART peripheral is in reset state and the transmitter is idle.<br>0 UART disabled<br>1 UART enabled                                                                |
| PEN      | Bit 4    | Parity enable. Enables parity generation on transmit and parity checking on receive.<br>0 Parity disabled<br>1 Parity enabled                                                                      |
| PSEL     | Bit 3    | Parity select. Selects even or odd parity when parity is enabled.<br>0 Even parity<br>1 Odd parity                                                                                                 |
| CIE      | Bit 2    | Receive complete interrupt enable. Enables interrupt generation when a byte is successfully received.<br>0 RX complete interrupt disabled<br>1 RX complete interrupt enabled                       |
| TEIE     | Bit 1    | Transmit empty interrupt enable. Enables interrupt generation when the transmit buffer becomes empty and ready for new data.<br>0 TX empty interrupt disabled<br>1 TX empty interrupt enabled      |
| TCIE     | Bit 0    | Transmit complete interrupt enable. Enables interrupt generation when transmission is complete and the transmitter is idle.<br>0 TX complete interrupt disabled<br>1 TX complete interrupt enabled |

### 13.11.2 UARTSR Register

Register memory slot: 1, offset: 4 bytes, size: 1 byte

UART status register. Contains receiver and transmitter status flags and interrupt flags. Error flags (FEF, PEF, OVF, RCIF) are cleared by reading UARTxRX. Interrupt flags (RCIF, TEIF, TCIF) can also be cleared by writing a 1 to the respective bit.



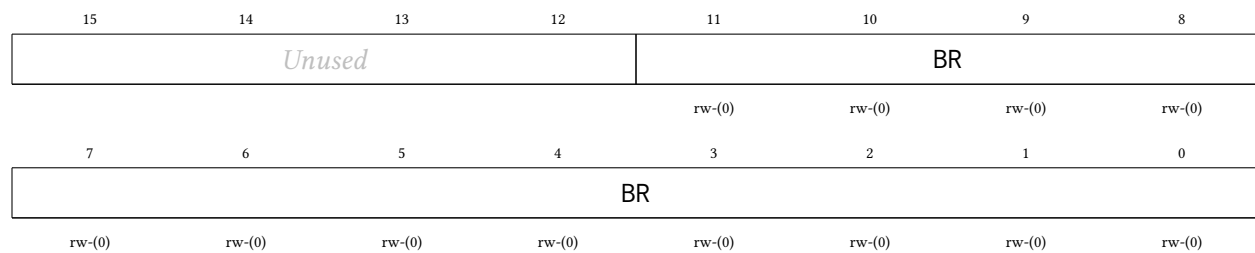
|       |       |       |       |       |         |         |         |
|-------|-------|-------|-------|-------|---------|---------|---------|
| 7     | 6     | 5     | 4     | 3     | 2       | 1       | 0       |
| RXBF  | TXBF  | FEF   | PEF   | OVF   | RCIF    | TEIF    | TCIF    |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | rw1-(0) | rw1-(0) | rw1-(0) |

| Bitfield | Range | Description                                                                                                                                                                                                                      |
|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RXBF     | Bit 7 | Receiver busy flag. Set while the UART receiver is actively receiving a byte.<br>0 Receiver idle<br>1 Reception in progress                                                                                                      |
| TXBF     | Bit 6 | Transmitter busy flag. Set while the UART transmitter is actively transmitting a byte or has a pending transmission.<br>0 Transmitter idle<br>1 Transmission in progress                                                         |
| FEF      | Bit 5 | Framing error flag. Set when the stop bit is not detected (RX line not high at expected stop bit time). Cleared by reading UARTxRX.<br>0 No framing error<br>1 Framing error detected on last reception                          |
| PEF      | Bit 4 | Parity error flag. Set when received parity does not match expected parity (when parity is enabled). Cleared by reading UARTxRX.<br>0 No parity error<br>1 Parity error detected on last reception                               |
| OVF      | Bit 3 | Receive overflow flag. Set when a new byte is received before the previous byte in UARTxRX was read by the processor. Cleared by reading UARTxRX.<br>0 No receive data overrun<br>1 Receive data overflow detected               |
| RCIF     | Bit 2 | Receive complete interrupt flag. Set when a byte is successfully received and placed in UARTxRX. Cleared by reading UARTxRX or writing a 1 to this bit.<br>0 No pending RX complete interrupt<br>1 RX complete interrupt pending |
| TEIF     | Bit 1 | Transmit empty interrupt flag. Set when the transmit buffer is empty and ready to accept new data. Write a 1 to this bit to clear it.<br>0 No pending TX empty interrupt<br>1 TX empty interrupt pending                         |
| TCIF     | Bit 0 | Transmit complete interrupt flag. Set when transmission is complete (all bits sent) and the transmitter is idle. Write a 1 to this bit to clear it.<br>0 No pending TX complete interrupt<br>1 TX complete interrupt pending     |

### 13.11.3 UARTBR Register

Register memory slot: 2, offset: 8 bytes, size: 2 bytes

UART baud rate register. Configures the baud rate divisor for the UART. The UART uses 16× oversampling.

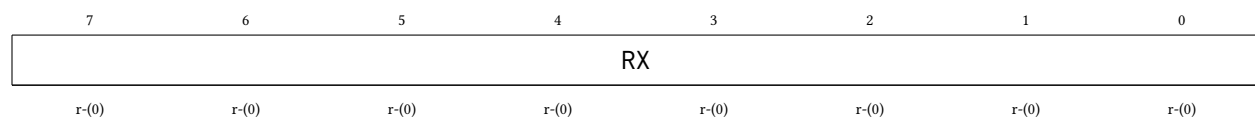


| Bitfield      | Range      | Description                                                                                                                                                                                                  |
|---------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Unused</i> | Bits 15-12 |                                                                                                                                                                                                              |
| BR            | Bits 11-0  | Baud rate divisor. UART baud rate = $SMCLK \div (16 \times (BR + 1))$ . For example, with $SMCLK = 48\text{ MHz}$ : $BR = 25$ gives 115,200 baud; $BR = 51$ gives 57,600 baud; $BR = 103$ gives 28,800 baud. |

### 13.11.4 UARTRX Register

*Register memory slot: 3, offset: 12 bytes, size: 1 byte*

UART receive buffer register. Contains the last received byte. Reading this register clears the FEF, PEF, OVF, and RCIF flags in UARTxSR. The value is latched on the falling edge of the memory enable signal.

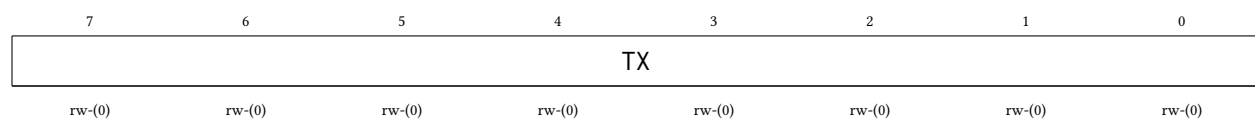


| Bitfield | Range    | Description        |
|----------|----------|--------------------|
| RX       | Bits 7-0 | Received data byte |

### 13.11.5 UARTRX Register

*Register memory slot: 4, offset: 16 bytes, size: 1 byte*

UART transmit buffer register. Writing to this register loads the byte to transmit and initiates transmission. Do not write to this register while TXBF is set in UARTxSR.



| Bitfield | Range    | Description        |
|----------|----------|--------------------|
| TX       | Bits 7-0 | Transmit data byte |

## 14 TIMERx Peripheral

The TIMERx peripheral is a 32-bit general purpose timer/counter unit with glitch-free clock source switching. Its main features are listed below:

- 32-bit timer values, from 0 to 4,294,967,295
- Memory mapped register to read and write current timer value
- Three output compare units, two of which can generate pulse width modulation (PWM) waveforms
- Two input capture units for precise timing measurements of external signals
- Four clock sources: SMCLK, MCLK, LFXT, HFXT
- Configurable clock divider ( $\div 1$  to  $\div 32,768$ )
- Glitch-free clock source multiplexer
- Six separate interrupts: 2 capture, 3 compare, 1 overflow
- Latched register reads for noise immunity

The timers live in the shared peripheral page behind the multi-core arbiter (TIMER0 at 0x4600, and TIMER1, in configurations that include it, at 0x4700), so any hart can own and operate a timer (see Section 4). Register accesses are serialized by the arbiter; a timer's interrupts are wired to hart 0's enables in the SYSTEM peripheral and can be routed to any hart through the IRQROUTER peripheral.

### 14.1 Timer Pins

The TIMERx peripheral has four pins: TxCMP0, TxCMP1, TxCAP0, and TxCAP1.

The TxCMP0 pin outputs the PWM waveform generated by the output compare unit 0. When the corresponding PxSEL[y] bit is a 1, the TIMERx peripheral seizes the DIR, OUT, and REN controls of the pin, forcing it to be in output mode. Similarly, TxCMP1 outputs the PWM waveform of output compare unit 1.

The TxCAP0 pin is an input that waits for a pin state change and then notifies the input capture unit 0, allowing precise timing of pin changes. The corresponding PxSEL[y] bit has no effect on this pin, since its secondary/peripheral function only ever acts as an input. Similarly, TxCAP1 is attached to input capture unit 1.

### 14.2 Clock Sources and Divider

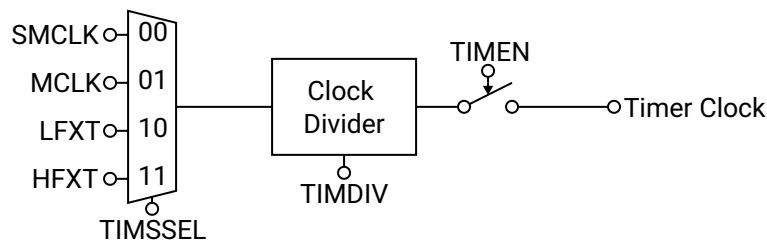


Figure 11: TIMERx Clock Diagram

The clock source for the TIMEx peripheral can be selected using TSSEL from four options: SMCLK, MCLK, LFXT (low frequency external clock), and HFXT (high frequency external clock). A glitch-free clock multiplexer built into the clock source selector prevents spurious clock transitions on the timer clock, allowing the user to switch clock sources freely even while the timer is enabled. However, switching clocks will cause the timer clock to be inactive for several clock cycles of the clock that is being switched from or to, whichever has a smaller frequency.

**Clock handoff after reset:** the glitch-free multiplexer powers up parked on the SMCLK slice, and releasing a slice takes three clock edges *of the clock being released* before the newly selected source can engage. The practical consequence: the first source selection after reset waits for three SMCLK edges, however slow SMCLK currently is. Since the boot ROM may leave SMCLK sourced from LFXT (32.768 kHz) when it jumps to the application, selecting a timer clock source right after boot can stall the timer for roughly 92  $\mu$ s — even when the selected source is MCLK or HFXT. The safe handoff procedure is:

1. Configure SMCLK to a fast source in the SYSTEM peripheral (see Section 15.1) before touching the timer.
2. After enabling the timer, poll TIMxVAL until it has visibly incremented, rather than assuming the timer counts immediately. Two back-to-back reads taken right after enabling can both legitimately return the same value while the handoff completes.

After the clock multiplexer, the timer clock frequency may optionally be reduced with a clock divider controlled by TDIV. The timer value register TIMxVAL increments by 1 on every rising edge of the divided timer clock when TIMEx is enabled with TEN.

A clock diagram for the TIMEx peripheral is shown in Figure 11.

### 14.3 Starting, Halting, Setting, and Reading the Timer

To start the timer, set the TEN bit to 1, whereupon the timer will begin incrementing by 1 with each rising edge of the divided timer clock. To halt the timer at its present value, clear the TEN bit to 0. The timer will retain its value until it is enabled again, its value is set manually, or the MCU is reset. If the timer is reenabled, it will begin counting from the timer value just before it was reenabled.

The timer value may be manually set at any time by writing the new 32-bit timer value to the TIMxVAL register. The timer value is updated immediately, regardless of if the timer is enabled or disabled.

The timer value may be read at any time by reading its value from the TIMxVAL register, regardless of if the timer is enabled or disabled. The TIMxVAL register is latched on the falling edge of the memory enable signal for stable reads during timer operation.

### 14.4 Overflow and Rollover Behavior

When the timer is enabled and its 32-bit value reaches its maximum of 4,294,967,295, the value will roll over back to zero on the next rising edge of the timer clock and the timer overflow interrupt flag TOVIF will be set. The timer will then continue incrementing as normal.

Optionally, the user may configure the timer to roll over before the 32-bit limit is reached by setting the new rollover value in the TIMxCMP2 register and setting the TCMP2RST bit to 1. In this mode, when the timer value becomes equivalent to the value of TIMxCMP2, the timer value will roll over back to zero on the next rising edge of the timer clock and then continue incrementing as normal, as depicted in Figure 12. This mechanism is used for setting the period of the PWM signals for the output compare units. Note: the timer rolls over back to zero when its value is *equal* to that of TIMxCMP2, but not if its value is greater than TIMxCMP2. This becomes important if the user manually sets the timer value above TIMxCMP2, or if

the user sets TIMxCMP2 below the current timer value, whereupon the timer will continue counting all the way to the 32-bit maximum value.

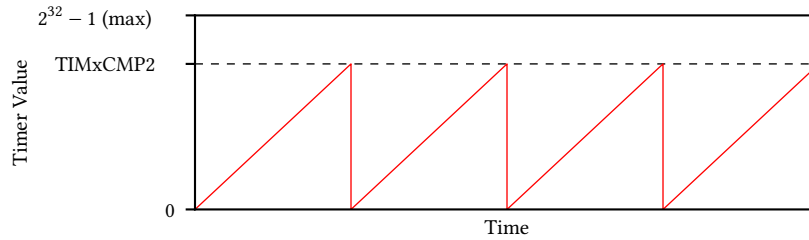


Figure 12: Timer Rollover Operation

## 14.5 Output Compare Units and Pulse Width Modulation

The TIMEx peripheral contains two output compare units that can be used to create two pulse width modulation (PWM) output signals on GPIO pins. Each output compare unit has an output compare register (TIMxCMP0 and TIMxCMP1), which is continually compared to the timer value TIMxVAL. The corresponding output compare pins (TxCMP0 and TxCMP1) toggle when either the timer value TIMxVAL equals the output compare register TIMxCMPy, or the timer rolls over. The TCMP0IH and TCMP1IH bits control the initial state of the TxCMP0 and TxCMP1 pins when the timer value is less than the output compare register. Figure 13 provides an example of PWM operation using an output compare unit.

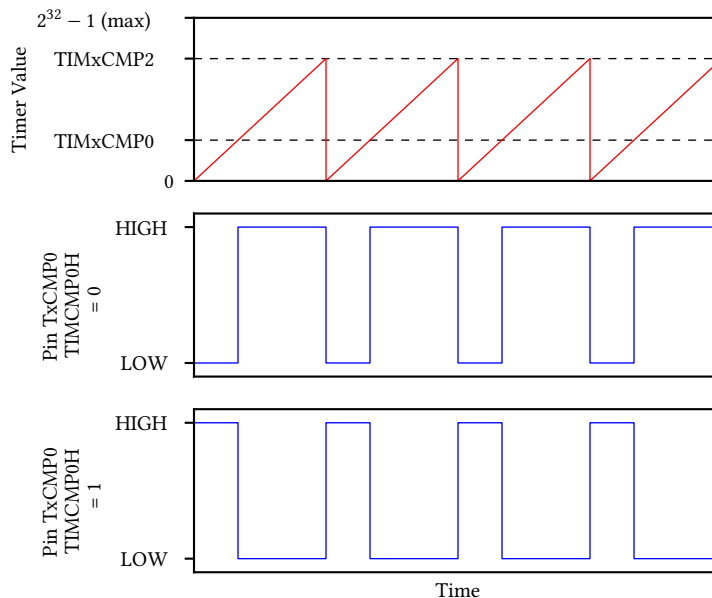


Figure 13: Timer Output Compare and PWM Operation

## 14.6 Input Capture Units

The TIMEx peripheral contains two input capture units which allow the precise timestamping of pin change events. When enabled with TCAPyEN, the input capture unit waits for the TxCAPy pin to toggle in the appropriate edge direction. The TCAPyFE bit selects either rising edge or falling edge sensitivity. When the pin toggles in the appropriate edge direction, the input capture unit stores the current timer value in the TIMxCAPy register and sets the TCAPyIF interrupt flag. The TCAPyIF flag must be cleared before the next input capture event by writing a 1 to it in the TIMxSR register.

Note: changing the value of TCAPyFE while TCAPyEN is enabled can inadvertently cause false input capture triggers. The user should not change TCAPyFE while TCAPyEN is enabled.

## 14.7 Flags and Interrupts

The TIMEx peripheral has two indicator bits and six interrupt flags in the timer status register TIMxSR. The TIMxSR register is latched on the falling edge of the memory enable signal for stable reads.

The two indicator bits TCMP0OUT and TCMP1OUT show the current value of the output compare pins TxCMP0 and TxCMP1 respectively, regardless of if the corresponding GPIO pins are set to primary/GPIO mode or secondary/peripheral mode.

The timer value overflow interrupt flag TOVIF is set when the 32-bit timer value overflows from  $2^{32} - 1$  back to 0. Note that when TCMP2RST is enabled, the timer resets at the TIMxCMP2 value and does not overflow to  $2^{32} - 1$ , so TOVIF will not be set in this mode. The overflow interrupt request is generated when both TOVIF and TOVIE are set. TOVIF is cleared by writing a 1 to it in the status register TIMxSR.

The output compare interrupt flags TCMP0IF, TCMP1IF, and TCMP2IF are set when the timer value equals the respective output compare value (TIMxCMP0, TIMxCMP1, and TIMxCMP2). Each compare interrupt request is generated when both the interrupt flag (TCMPyIF) and the corresponding interrupt enable bit (TCMP0IE, TCMP1IE, or TCMP2IE) are set. Each TCMPyIF flag is cleared by writing a 1 to it in the status register TIMxSR. Note that TCMP2IF is only set when TCMP2RST is enabled and the timer resets at the TIMxCMP2 value.

The input capture interrupt flags TCAP0IF and TCAP1IF are set when a new input capture event occurs on the TxCAP0 and TxCAP1 pins, respectively. The input capture register TIMxCAPy can be read as soon as TCAPyIF is set. Each capture interrupt request is generated when both the interrupt flag (TCAPyIF) and the corresponding interrupt enable bit (TCAP0IE or TCAP1IE) are set. Each TCAPyIF flag is cleared by writing a 1 to it in the status register TIMxSR.

## 14.8 Register Description

### 14.8.1 TIMCR Register

Register memory slot: 0, offset: 0 bytes, size: 4 bytes

Timer/Counter control register. Configures clock source, clock divider, capture/compare settings, and interrupt enables.

|         |        |        |        |        |        |        |        |
|---------|--------|--------|--------|--------|--------|--------|--------|
| 31      | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| Unused  |        |        |        |        |        |        |        |
| 23      | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| Unused  |        |        |        | DIV    |        |        |        |
|         |        |        |        | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15      | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| CMP1IH  | CMP0IH | CAP1FE | CAP0FE | CAP1EN | CAP0EN | SSEL   |        |
| rw-(0)  | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7       | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| CMP2RST | EN     | CAP1IE | CAP0IE | OVIE   | CMP2IE | CMP1IE | CMP0IE |
| rw-(0)  | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range      | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unused   | Bits 31-20 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| DIV      | Bits 19-16 | Timer/Counter clock divider. Divides the clock source selected by SSEL to generate the timer clock for TIMxVAL increments.<br>0000 DIV_1: /1 (no division)<br>0001 DIV_2: /2<br>0010 DIV_4: /4<br>0011 DIV_8: /8<br>0100 DIV_16: /16<br>0101 DIV_32: /32<br>0110 DIV_64: /64<br>0111 DIV_128: /128<br>1000 DIV_256: /256<br>1001 DIV_512: /512<br>1010 DIV_1024: /1,024<br>1011 DIV_2048: /2,048<br>1100 DIV_4096: /4,096<br>1101 DIV_8192: /8,192<br>1110 DIV_16384: /16,384<br>1111 DIV_32768: /32,768 |
| CMP1IH   | Bit 15     | Compare 1 initial PWM output level. Sets the TxCMP1 pin level when TIMxVAL < TIMxCMP1.<br>0 PWM output starts LOW                                                                                                                                                                                                                                                                                                                                                                                        |

|         |          |    |                                                                                                                  |
|---------|----------|----|------------------------------------------------------------------------------------------------------------------|
|         |          | 1  | PWM output starts HIGH                                                                                           |
| CMP0IH  | Bit 14   |    | Compare 0 initial PWM output level. Sets the TxCMP0 pin level when TIMxVAL < TIMxCMP0.                           |
|         |          | 0  | PWM output starts LOW                                                                                            |
|         |          | 1  | PWM output starts HIGH                                                                                           |
| CAP1FE  | Bit 13   |    | Capture 1 edge select. Selects which edge on TxCAP1 pin triggers a capture event.                                |
|         |          | 0  | Capture on rising edge                                                                                           |
|         |          | 1  | Capture on falling edge                                                                                          |
| CAP0FE  | Bit 12   |    | Capture 0 edge select. Selects which edge on TxCAP0 pin triggers a capture event.                                |
|         |          | 0  | Capture on rising edge                                                                                           |
|         |          | 1  | Capture on falling edge                                                                                          |
| CAP1EN  | Bit 11   |    | Capture 1 enable. Enables input capture on TxCAP1 pin.                                                           |
|         |          | 0  | Capture 1 disabled                                                                                               |
|         |          | 1  | Capture 1 enabled                                                                                                |
| CAP0EN  | Bit 10   |    | Capture 0 enable. Enables input capture on TxCAP0 pin.                                                           |
|         |          | 0  | Capture 0 disabled                                                                                               |
|         |          | 1  | Capture 0 enabled                                                                                                |
| SSEL    | Bits 9-8 |    | Timer/Counter clock source select. Glitch-free multiplexer prevents spurious transitions when switching sources. |
|         |          | 00 | SSEL_SMCLK: SMCLK                                                                                                |
|         |          | 01 | SSEL_MCLK: MCLK                                                                                                  |
|         |          | 10 | SSEL_LFXT: LFXT (low frequency crystal)                                                                          |
|         |          | 11 | SSEL_HFXT: HFXT (high frequency crystal)                                                                         |
| CMP2RST | Bit 7    |    | Timer/Counter reset on Compare 2 enable. Resets TIMxVAL to 0 when TIMxVAL equals TIMxCMP2.                       |
|         |          | 0  | Free-running mode (resets at $2^{32}-1$ )                                                                        |
|         |          | 1  | Resets on TIMxVAL = TIMxCMP2                                                                                     |
| EN      | Bit 6    |    | Timer/Counter enable. When disabled, timer clock is gated off and TIMxVAL holds its value.                       |
|         |          | 0  | Timer disabled                                                                                                   |
|         |          | 1  | Timer enabled                                                                                                    |
| CAP1IE  | Bit 5    |    | Capture 1 interrupt enable. Enables interrupt when CAP1IF flag is set.                                           |
|         |          | 0  | Interrupt disabled                                                                                               |
|         |          | 1  | Interrupt enabled                                                                                                |
| CAP0IE  | Bit 4    |    | Capture 0 interrupt enable. Enables interrupt when CAP0IF flag is set.                                           |
|         |          | 0  | Interrupt disabled                                                                                               |
|         |          | 1  | Interrupt enabled                                                                                                |
| OVIE    | Bit 3    |    | Timer/Counter overflow interrupt enable. Enables interrupt when OVIF flag is set.                                |
|         |          | 0  | Interrupt disabled                                                                                               |
|         |          | 1  | Interrupt enabled                                                                                                |
| CMP2IE  | Bit 2    |    | Compare 2 interrupt enable. Enables interrupt when CMP2IF flag is set.                                           |
|         |          | 0  | Interrupt disabled                                                                                               |
|         |          | 1  | Interrupt enabled                                                                                                |
| CMP1IE  | Bit 1    |    | Compare 1 interrupt enable. Enables interrupt when CMP1IF flag is set.                                           |



|        |       |                                                                        |                    |
|--------|-------|------------------------------------------------------------------------|--------------------|
|        |       | 0                                                                      | Interrupt disabled |
|        |       | 1                                                                      | Interrupt enabled  |
| CMP0IE | Bit 0 | Compare 0 interrupt enable. Enables interrupt when CMP0IF flag is set. |                    |
|        |       | 0                                                                      | Interrupt disabled |
|        |       | 1                                                                      | Interrupt enabled  |

### 14.8.2 TIMSR Register

Register memory slot: 1, offset: 4 bytes, size: 1 byte

Timer/Counter status register. Contains current compare output levels and interrupt flags. The register is latched on the falling edge of the memory enable signal for stable reads.

|         |         |         |         |         |         |         |         |
|---------|---------|---------|---------|---------|---------|---------|---------|
| 7       | 6       | 5       | 4       | 3       | 2       | 1       | 0       |
| CMP1OUT | CMP0OUT | CAP1IF  | CAP0IF  | OVIF    | CMP2IF  | CMP1IF  | CMP0IF  |
| r-(0)   | r-(0)   | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) |

| Bitfield | Range | Description                                                                                                                                                                |
|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CMP1OUT  | Bit 7 | Current value of the Compare 1 output pin TxCMP1. Reflects the actual PWM output level.<br>0 TxCMP1 output LOW<br>1 TxCMP1 output HIGH                                     |
| CMP0OUT  | Bit 6 | Current value of the Compare 0 output pin TxCMP0. Reflects the actual PWM output level.<br>0 TxCMP0 output LOW<br>1 TxCMP0 output HIGH                                     |
| CAP1IF   | Bit 5 | Capture 1 interrupt flag. Set when TxCAP1 pin triggers a capture event. Write 1 to clear.<br>0 No pending capture 1 interrupt<br>1 Capture 1 interrupt pending             |
| CAP0IF   | Bit 4 | Capture 0 interrupt flag. Set when TxCAP0 pin triggers a capture event. Write 1 to clear.<br>0 No pending capture 0 interrupt<br>1 Capture 0 interrupt pending             |
| OVIF     | Bit 3 | Timer/Counter overflow interrupt flag. Set when timer overflows from $2^{32}-1$ to 0. Write 1 to clear.<br>0 No pending overflow interrupt<br>1 Overflow interrupt pending |
| CMP2IF   | Bit 2 | Compare 2 interrupt flag. Set when TIMxVAL equals TIMxCMP2. Write 1 to clear.<br>0 No pending compare 2 interrupt<br>1 Compare 2 interrupt pending                         |
| CMP1IF   | Bit 1 | Compare 1 interrupt flag. Set when TIMxVAL equals TIMxCMP1. Write 1 to clear.<br>0 No pending compare 1 interrupt<br>1 Compare 1 interrupt pending                         |

|        |       |                                                                               |
|--------|-------|-------------------------------------------------------------------------------|
| CMP0IF | Bit 0 | Compare 0 interrupt flag. Set when TIMxVAL equals TIMxCMP0. Write 1 to clear. |
|        | 0     | No pending compare 0 interrupt                                                |
|        | 1     | Compare 0 interrupt pending                                                   |

### 14.8.3 TIMVAL Register

*Register memory slot: 2, offset: 8 bytes, size: 4 bytes*

Timer/Counter value register. Reads the current timer count. Writes immediately update the timer value regardless of enable state. The value is latched on the falling edge of the memory enable signal for stable reads.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| VAL    |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| VAL    |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| VAL    |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| VAL    |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description               |
|----------|-----------|---------------------------|
| VAL      | Bits 31-0 | Current timer count value |

### 14.8.4 TIMCMP0 Register

*Register memory slot: 3, offset: 12 bytes, size: 4 bytes*

Timer/Counter Compare 0 threshold register. When TIMxVAL equals this value, CMP0IF is set and the TxCMP0 PWM output toggles.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| CMP0   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| CMP0   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| CMP0   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| CMP0   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description               |
|----------|-----------|---------------------------|
| CMP0     | Bits 31-0 | Compare 0 threshold value |

#### 14.8.5 TIMCMP1 Register

Register memory slot: 4, offset: 16 bytes, size: 4 bytes

Timer/Counter Compare 1 threshold register. When TIMxVAL equals this value, CMP1IF is set and the TxCMP1 PWM output toggles.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| CMP1   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| CMP1   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| CMP1   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| CMP1   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description               |
|----------|-----------|---------------------------|
| CMP1     | Bits 31-0 | Compare 1 threshold value |

#### 14.8.6 TIMCMP2 Register

Register memory slot: 5, offset: 20 bytes, size: 4 bytes

Timer/Counter Compare 2 threshold register. When TIMxVAL equals this value, CMP2IF is set. If CMP2RST is enabled, timer resets to 0 (PWM period control).

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| CMP2   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| CMP2   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| CMP2   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| CMP2   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                            |
|----------|-----------|----------------------------------------|
| CMP2     | Bits 31-0 | Compare 2 threshold value (PWM period) |

#### 14.8.7 TIMCAP0 Register

Register memory slot: 6, offset: 24 bytes, size: 4 bytes

Timer/Counter Capture 0 value register. Automatically latches TIMxVAL when TxCAP0 pin edge triggers a capture event. The captured value is latched on the falling edge of the memory enable signal for stable reads.

|       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|
| 31    | 30    | 29    | 28    | 27    | 26    | 25    | 24    |
| CAP0  |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 23    | 22    | 21    | 20    | 19    | 18    | 17    | 16    |
| CAP0  |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 15    | 14    | 13    | 12    | 11    | 10    | 9     | 8     |
| CAP0  |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| CAP0  |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |

| Bitfield | Range | Description |
|----------|-------|-------------|
|----------|-------|-------------|

|      |           |                                        |
|------|-----------|----------------------------------------|
| CAP0 | Bits 31-0 | Captured timer value from TxCAP0 event |
|------|-----------|----------------------------------------|

### 14.8.8 TIMCAP1 Register

Register memory slot: 7, offset: 28 bytes, size: 4 bytes

Timer/Counter Capture 1 value register. Automatically latches TIMxVAL when TxCAP1 pin edge triggers a capture event. The captured value is latched on the falling edge of the memory enable signal for stable reads.

|       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|
| 31    | 30    | 29    | 28    | 27    | 26    | 25    | 24    |
| CAP1  |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 23    | 22    | 21    | 20    | 19    | 18    | 17    | 16    |
| CAP1  |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 15    | 14    | 13    | 12    | 11    | 10    | 9     | 8     |
| CAP1  |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| CAP1  |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |

| Bitfield | Range     | Description                            |
|----------|-----------|----------------------------------------|
| CAP1     | Bits 31-0 | Captured timer value from TxCAP1 event |

## 15 SYSTEM Peripheral

The SYSTEM peripheral manages a potpourri of system-level blocks, including the MCU clocks, the watchdog timer, and a CRC data verification module.

Its effects are global — MCLK clocks all harts and SMCLK clocks the peripherals — so by convention only hart 0 (the management hart) reconfigures the clocks, after quiescing the other harts. Interrupt routing, masking and delivery are **not** managed here: every hart, hart 0 included, enables peripheral interrupts through its IRQROUTER row and takes them via the claim/complete mechanism, and the CLINT software/timer interrupts (vectors 83 and 84) are hardwired enabled in every hart. (The single-core chip's IRQENx/IRQPRIx/IRQCR registers are retired; their register slots are reserved and read as zero.)

### 15.1 System Clocks and Clock Management

The MCU has four clock sources: HFXT, LFXT, DCO0, and DCO1. HFXT and LFXT are external clock sources, which must be provided to the MCU via the ClkHFXT and ClkLFXT pins, respectively. DCO0 and DCO1 are internal, programmable frequency clock sources provided by digitally controllable oscillator circuits.

The Castalia MCU is designed and characterised for operation with a maximum clock frequency of 24 MHz. HFXT should therefore not exceed 24 MHz. LFXT is suggested to be exactly 32.768 kHz to provide an accurate method of measuring time. The frequencies of DCO0 and DCO1 are configured with the DCO0BIAS and DCO1BIAS registers; higher bias values produce higher frequencies.

**Note on overclocking:** It is possible to operate the Castalia MCU beyond 24 MHz by supplying a higher-frequency signal on the ClkHFXT pin or by configuring the internal DCOs to produce frequencies above 24 MHz. As with any digital circuit operated beyond its rated frequency, doing so may produce setup-time violations, incorrect computational results, and unpredictable behaviour. Operation above 24 MHz is **not guaranteed** and is undertaken entirely at the user's risk.

Two clocks, the main clock (MCLK) and the submain clock (SMCLK), form the roots of the MCU clock tree, shown in Figure 14. MCLK and SMCLK can each select one of the four source clocks as their base and optionally divide the frequency further using CLKDIVCR. MCLK drives all four harts and the multi-core arbiter, while SMCLK drives most of the peripherals. Both clocks use glitch-free multiplexers to prevent spurious transitions when switching sources.

MCLK cannot be turned off, but SMCLK and individual source clocks can be independently disabled via SYSCLKCR. DCO0 and DCO1 are off by default and must be explicitly powered on by setting DCO0ON or DCO1ON. HFXT and LFXT are enabled by default and may be disabled with HFXTOFF and LFXTOFF. If a source clock is currently selected by MCLK or SMCLK, that source will be automatically kept enabled regardless of its disable bit in SYSCLKCR.

### 15.2 Power Saving Features

The MCU has several power-saving features.

The CPU is the most power-hungry part of the MCU, and its dynamic power consumption is proportional to the CPU clock frequency. Reducing the frequency of MCLK is the primary avenue for reducing total dynamic power consumption. This can be done by utilising the MCLK clock divider in CLKDIVCR, by switching MCLK to a lower-frequency source via MCLKSEL, or by reducing the frequency of the currently-selected source. The internal DCOs offer a wide range of programmable frequencies through their bias settings and can be used as a low-power clock source when HFXT is disabled.

Another method to save power is to turn off unused clocks. Remember, however, that the ClkHFXT pin is required to be a valid clock signal while the MCU boots. Failure to provide a valid clock may cause the

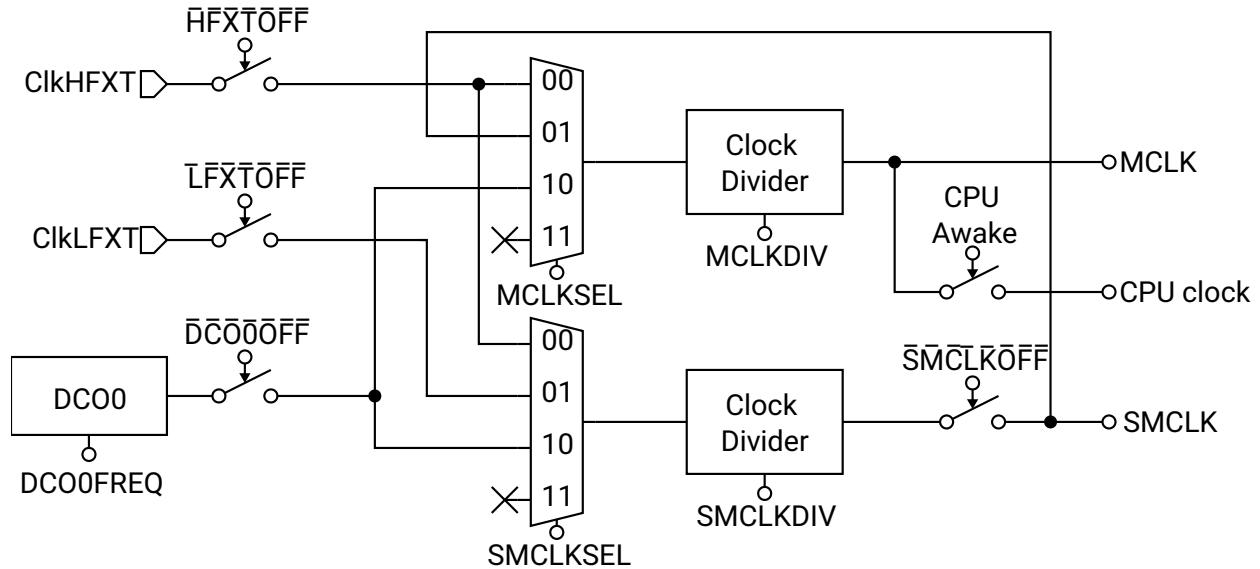


Figure 14: MCU Clock System

chip to boot improperly, or not at all. For this reason, the start-HIGH pin(s) (SHx) have been provided. On the PCB, the start-HIGH pins should be connected to the enable input of the external clock source driving CLKHFXT, ensuring the clock is always present during reset. After boot, the user may switch MCLK to another source and then safely disable CLKHFXT.

The static (leakage) power can be reduced by powering off unused on-chip memory blocks with BLOCKPWR. The ROM and RAM blocks include power-gating circuitry that physically disconnects their supply rail. The ROM can be powered down by setting ROMOFF, and the two RAM blocks by setting RAM0OFF and RAM10FF, respectively. Each powered-down block reduces leakage. Any access to a powered-down block will fail and produce undefined results. The entire contents of a RAM block are lost when it is powered down; data are undefined when it is powered back on until explicitly overwritten. Care must be taken to track the power state of each block to avoid undefined behaviour.

### 15.3 CPU Sleep Mode

Another way to conserve power is to run the CPU only when needed and switch it off when idle. Because the CPU accounts for the majority of dynamic power, duty-cycling its clock can yield significant savings. Peripherals can continue to operate while the CPU sleeps and may wake it when events occur. Long-term average CPU power is:

$$P_{\text{CPU}} = P_{\text{CPU}, 100\%} \times \frac{t_{\text{on}}}{t_{\text{on}} + t_{\text{off}}}$$

Sleep mode is per hart: each hart can execute the custom sleep instruction independently, which gates off that hart's CPU clock and reduces its dynamic power to zero. A sleeping hart can only be woken by an interrupt (typically a CLINT software interrupt or a peripheral interrupt routed to it through the IRQROUTER) or a system reset. When an interrupt fires, the CPU clock is re-enabled for the duration of that interrupt service routine (ISR). Once all pending interrupts have been serviced the CPU returns to sleep, unless one of the ISRs executed the custom wake instruction, in which case the CPU remains awake and returns to the point in the main program where it went to sleep.

A code example to put the CPU to sleep is included in Listing 4.

```
1 /** Includes **/
```

```

2  #include <MemoryMap.h>
3  #include <irq.h>
4
5  /** Interrupt Service Routine Declaration Macros */
6  RVISR(IRQ_TIMER1_VECTOR, ISR_TIMER1)
7
8  /** Main Function */
9  int main()
10 {
11     // Set up TIMER1 here
12
13     // Enable interrupts by unmasking them
14     enable_all_interrupts();
15
16     // Go to sleep
17     cpu_sleep();
18
19     // Insert code to execute after ISR wakes the CPU
20
21     // Wait forever
22     while (1) {}
23 }
24
25 /** Interrupt Service Routines */
26 void ISR_TIMER1()
27 {
28     // Do ISR processing here
29
30     // Wake the CPU
31     cpu_wake();
32 }

```

Listing 4: Example program to use CPU sleep mode

## 15.4 CRC Module

The CRC module enables the computation of a CRC16 checksum over an array of bytes. The checksum can be used to verify the integrity of data in a communication channel or in memory.

The module implements the CRC16-CDMA2000 standard with the following parameters:

- CRC width: 16 bits
- Polynomial: 0xC857
- Initial value: 0xFFFF
- No output XOR
- No input reflection
- No output reflection

To compute a CRC over a byte array, first write 0xFFFF to CRCSTATE to initialise the accumulator. Then write each byte of the array sequentially to CRCDATA; each write updates the running CRC. After the last byte has been written, read CRCSTATE to obtain the final CRC16 value.

## 15.5 Watchdog Timer

The watchdog timer improves system reliability by automatically taking corrective action if software fails to service the watchdog within a configurable timeout period. The watchdog is controlled by WDTCR, which is write-protected and must be unlocked via WDTPASS before any configuration change.



The watchdog uses a 24-bit up-counter that increments on every MCLK cycle when enabled via WDTEN. The timeout period is selected by WDTCDIV, which chooses which bit of the counter triggers the watchdog event. A watchdog event occurs on the 0-to-1 transition of the selected bit, giving a timeout of  $2^{\text{WDTCDIV}+16}$  MCLK cycles. At 24 MHz the timeout ranges from approximately 2.73 ms (WDTCDIV = 0000, bit 16) to approximately 89.5 s (WDTCDIV = 1111, bit 31).

On a watchdog event, the response depends on the configuration:

- If WDTIE is set **and** the watchdog vector (vector 0) is routed to some hart in the IRQROUTER, the watchdog interrupt is delivered through the claim/complete mechanism and the servicing hart executes its handler. If WDTWHRST is also set, the system resets after that hart completes the claim.
- If WDTIE is clear, or vector 0 is not routed to any hart, the system resets immediately on timeout provided WDTWHRST is set.

To prevent timeout, software must periodically write the clear password (0xA0C8A620) to WDTPASS, which resets the counter to zero. To modify WDTCR, first write the unlock password (0x5F3759DF) to WDTPASS; this opens a 64-MCLK-cycle window during which WDTCR is writable.

The WDTSR status register holds two sticky flags. WDTRF is set when the last system reset was caused by the watchdog and persists across resets; it is cleared by writing 1 to the bit. WDTIF is set when a watchdog timeout event occurs and is likewise cleared by writing 1. The current counter value is readable at any time from WDTVAl; the register reads as zero when the watchdog is disabled.

## 15.6 Register Description

### 15.6.1 SYSCLKCR Register

Register memory slot: 0, offset: 0 bytes, size: 2 bytes

System clock control register

|        |         |         |          |          |        |         |        |
|--------|---------|---------|----------|----------|--------|---------|--------|
| 15     | 14      | 13      | 12       | 11       | 10     | 9       | 8      |
| Unused |         |         |          |          |        |         | DCO1ON |
| rw-(0) |         |         |          |          |        |         |        |
| 7      | 6       | 5       | 4        | 3        | 2      | 1       | 0      |
| DCO0ON | HFXTOFF | LFXTOFF | SMCLKOFF | SMCLKSEL |        | MCLKSEL |        |
| rw-(0) | rw-(0)  | rw-(0)  | rw-(0)   | rw-(0)   | rw-(0) | rw-(0)  | rw-(0) |

| Bitfield | Range     | Description                                                                                                                                                                                                                                  |
|----------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unused   | Bits 15-9 |                                                                                                                                                                                                                                              |
| DCO1ON   | Bit 8     | Digitally controlled oscillator 1 (DCO1) power enable. Set to power on DCO1. DCO1 is automatically kept on if it is currently selected as the source for MCLK or SMCLK, regardless of this bit.<br>0 DCO1 powered off<br>1 DCO1 powered on   |
| DCO0ON   | Bit 7     | Digitally controlled oscillator 0 (DCO0) power enable. Set to power on DCO0. DCO0 is automatically kept on if it is currently selected as the source for MCLK or SMCLK, regardless of this bit.<br>0 DCO0 powered off<br>1 DCO0 powered on   |
| HFXTOFF  | Bit 6     | High frequency external crystal clock disable. Cannot be disabled if it is currently selected as the source for MCLK or SMCLK.<br>0 HFXT enabled<br>1 HFXT disabled                                                                          |
| LFXTOFF  | Bit 5     | Low frequency external crystal clock disable. Cannot be disabled if it is currently selected as the source for SMCLK.<br>0 LFXT enabled<br>1 LFXT disabled                                                                                   |
| SMCLKOFF | Bit 4     | Submain clock disable. Globally and unconditionally disables SMCLK, gating all peripherals clocked from SMCLK.<br>0 SMCLK enabled<br>1 SMCLK disabled                                                                                        |
| SMCLKSEL | Bits 3-2  | Submain clock source select<br>00 SMCLKSEL_HFXT: High Frequency Crystal Clock<br>01 SMCLKSEL_LFXT: Low Frequency Crystal Clock<br>10 SMCLKSEL_DCO0: Digitally Controlled Oscillator 0<br>11 SMCLKSEL_DCO1: Digitally Controlled Oscillator 1 |
| MCLKSEL  | Bits 1-0  | Main clock source select (also CPU clock source select)<br>00 MCLKSEL_HFXT: High Frequency Crystal Clock<br>01 MCLKSEL_SMCLK: Submain Clock                                                                                                  |

- 10 MCLKSEL\_DC00: Digitally Controlled Oscillator 0  
 11 MCLKSEL\_DC01: Digitally Controlled Oscillator 1

### 15.6.2 CLKDIVCR Register

Register memory slot: 1, offset: 4 bytes, size: 1 byte

MCLK and SMCLK clock divider control register. Configures clock division for main and submain clocks using glitch-free multiplexers.

|        |   |          |        |        |         |        |        |
|--------|---|----------|--------|--------|---------|--------|--------|
| 7      | 6 | 5        | 4      | 3      | 2       | 1      | 0      |
| Unused |   | SMCLKDIV |        |        | MCLKDIV |        |        |
|        |   | rw-(0)   | rw-(0) | rw-(0) | rw-(0)  | rw-(0) | rw-(0) |

| Bitfield | Range    | Description                                                                                                                                                                                                                                                                                                                                               |
|----------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unused   | Bits 7-6 |                                                                                                                                                                                                                                                                                                                                                           |
| SMCLKDIV | Bits 5-3 | SMCLK clock division selection. Division is applied after clock source selection through glitch-free divider multiplexer.<br>000 SYSSMCLKDIV_1: /1 (no division)<br>001 SYSSMCLKDIV_2: /2<br>010 SYSSMCLKDIV_4: /4<br>011 SYSSMCLKDIV_8: /8<br>100 SYSSMCLKDIV_16: /16<br>101 SYSSMCLKDIV_32: /32<br>110 SYSSMCLKDIV_64: /64<br>111 SYSSMCLKDIV_128: /128 |
| MCLKDIV  | Bits 2-0 | MCLK clock division selection. Division is applied after clock source selection through glitch-free divider multiplexer.<br>000 SYSMCLKDIV_1: /1 (no division)<br>001 SYSMCLKDIV_2: /2<br>010 SYSMCLKDIV_4: /4<br>011 SYSMCLKDIV_8: /8<br>100 SYSMCLKDIV_16: /16<br>101 SYSMCLKDIV_32: /32<br>110 SYSMCLKDIV_64: /64<br>111 SYSMCLKDIV_128: /128          |

### 15.6.3 BLOCKPWR Register

Register memory slot: 2, offset: 8 bytes, size: 1 byte

Block power control register. Controls power gating for on-chip memory blocks.

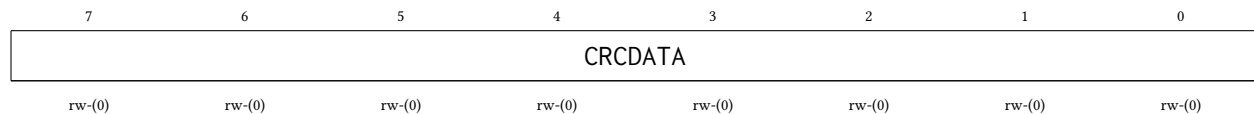
|        |   |   |   |   |         |         |        |
|--------|---|---|---|---|---------|---------|--------|
| 7      | 6 | 5 | 4 | 3 | 2       | 1       | 0      |
| Unused |   |   |   |   | RAM1OFF | RAM0OFF | ROMOFF |
|        |   |   |   |   | rw-(0)  | rw-(0)  | rw-(0) |

| Bitfield      | Range    | Description                                                                                                                                                                                                                               |
|---------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Unused</i> | Bits 7-3 |                                                                                                                                                                                                                                           |
| RAM1OFF       | Bit 2    | RAM block 1 power control. When set, RAM block 1 is powered off. All data becomes undefined and the block no longer responds to memory access. Reduces static power consumption.<br>0 RAM block 1 powered on<br>1 RAM block 1 powered off |
| RAM0OFF       | Bit 1    | RAM block 0 power control. When set, RAM block 0 is powered off. All data becomes undefined and the block no longer responds to memory access. Reduces static power consumption.<br>0 RAM block 0 powered on<br>1 RAM block 0 powered off |
| ROMOFF        | Bit 0    | ROM power control. When set, boot ROM is powered off. ROM no longer responds to read access. Reduces static power consumption.<br>0 ROM powered on<br>1 ROM powered off                                                                   |

#### 15.6.4 CRCDATA Register

Register memory slot: 3, offset: 12 bytes, size: 1 byte

CRC input data register. Write the next byte of data to this register to update the CRC calculation. Uses CRC16-CDMA2000 polynomial 0xC857.

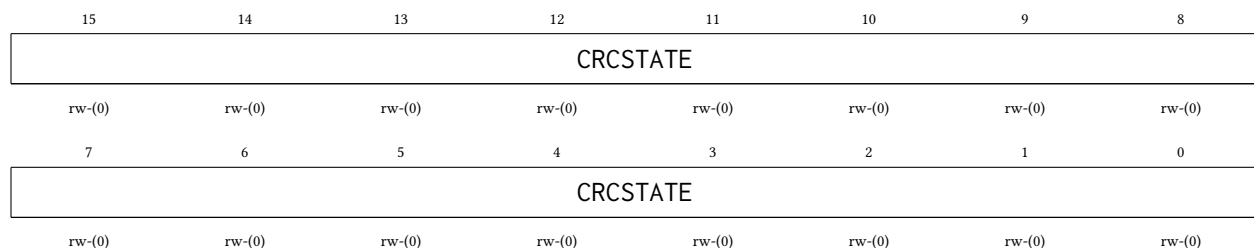


| Bitfield | Range    | Description                                                                                                 |
|----------|----------|-------------------------------------------------------------------------------------------------------------|
| CRCDATA  | Bits 7-0 | CRC data input byte. Writing to this register feeds the byte into the CRC calculation and updates CRCSTATE. |

#### 15.6.5 CRCSTATE Register

Register memory slot: 4, offset: 16 bytes, size: 2 bytes

CRC state register. Contains the current CRC16 calculation result. Write to this register to initialize or restart the CRC calculation. Read to obtain the computed CRC16 checksum.



| Bitfield | Range     | Description                                                                                                                              |
|----------|-----------|------------------------------------------------------------------------------------------------------------------------------------------|
| CRCSTATE | Bits 15-0 | CRC state value. Initialize to 0xFFFF before starting CRC calculation. Read after processing all data bytes to get final CRC16 checksum. |

### 15.6.6 WDTPASS Register

*Register memory slot: 12, offset: 48 bytes, size: 4 bytes*

Watchdog timer password register. Write-only register for two security functions: (1) Write 0x5F3759DF to unlock WDTCSR for 64 MCLK cycles, enabling writes to watchdog configuration. (2) Write 0xA0C8A620 to clear watchdog counter to 0, preventing timeout. Reading always returns 0.

|         |       |       |       |       |       |       |       |
|---------|-------|-------|-------|-------|-------|-------|-------|
| 31      | 30    | 29    | 28    | 27    | 26    | 25    | 24    |
| WDTPASS |       |       |       |       |       |       |       |
| w-(0)   | w-(0) | w-(0) | w-(0) | w-(0) | w-(0) | w-(0) | w-(0) |
| 23      | 22    | 21    | 20    | 19    | 18    | 17    | 16    |
| WDTPASS |       |       |       |       |       |       |       |
| w-(0)   | w-(0) | w-(0) | w-(0) | w-(0) | w-(0) | w-(0) | w-(0) |
| 15      | 14    | 13    | 12    | 11    | 10    | 9     | 8     |
| WDTPASS |       |       |       |       |       |       |       |
| w-(0)   | w-(0) | w-(0) | w-(0) | w-(0) | w-(0) | w-(0) | w-(0) |
| 7       | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| WDTPASS |       |       |       |       |       |       |       |
| w-(0)   | w-(0) | w-(0) | w-(0) | w-(0) | w-(0) | w-(0) | w-(0) |

| Bitfield | Range     | Description                                                                                                                                                         |
|----------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WDTPASS  | Bits 31-0 | Watchdog password. Write 0x5F3759DF (unlock password) to enable WDTCSR writes for 64 MCLK cycles. Write 0xA0C8A620 (clear password) to reset watchdog counter to 0. |

### 15.6.7 WDTCSR Register

*Register memory slot: 13, offset: 52 bytes, size: 1 byte*

Watchdog timer control register. This register is protected and requires password unlock via WDTPASS before writing. Configures watchdog operation mode and timeout period.

|        |        |         |        |        |        |        |          |
|--------|--------|---------|--------|--------|--------|--------|----------|
| 7      | 6      | 5       | 4      | 3      | 2      | 1      | 0        |
| WDTEN  | Unused | WDTCDIV |        |        |        | WDTIE  | WDTHWRST |
| rw-(0) |        | rw-(0)  | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0)   |

| Bitfield | Range | Description |
|----------|-------|-------------|
|----------|-------|-------------|

|               |          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WDTEN         | Bit 7    | <p>Watchdog timer enable. When set, watchdog counter increments on MCLK. Watchdog generates interrupt and/or reset when counter bit selected by WDTCDIV transitions from 0 to 1. Register is write-protected; unlock with WDTPASS first.</p> <p>0 Watchdog disabled</p> <p>1 Watchdog enabled</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <i>Unused</i> | Bit 6    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| WDTCDIV       | Bits 5-2 | <p>Watchdog timer clock divider select. Selects which bit of the 24-bit counter triggers watchdog event. Event occurs when selected bit transitions from 0 to 1. Timeout period = <math>2^{(WDTCDIV+16)}</math> MCLK cycles.</p> <p>0000 SYSWDTCDIV_65536: Bit 16: 65,536 MCLK cycles</p> <p>0001 SYSWDTCDIV_131072: Bit 17: 131,072 MCLK cycles</p> <p>0010 SYSWDTCDIV_262144: Bit 18: 262,144 MCLK cycles</p> <p>0011 SYSWDTCDIV_524288: Bit 19: 524,288 MCLK cycles</p> <p>0100 SYSWDTCDIV_1048576: Bit 20: 1,048,576 MCLK cycles</p> <p>0101 SYSWDTCDIV_2097152: Bit 21: 2,097,152 MCLK cycles</p> <p>0110 SYSWDTCDIV_4194304: Bit 22: 4,194,304 MCLK cycles</p> <p>0111 SYSWDTCDIV_8388608: Bit 23: 8,388,608 MCLK cycles</p> <p>1000 SYSWDTCDIV_16777216: Bit 24: 16,777,216 MCLK cycles</p> <p>1001 SYSWDTCDIV_33554432: Bit 25: 33,554,432 MCLK cycles</p> <p>1010 SYSWDTCDIV_67108864: Bit 26: 67,108,864 MCLK cycles</p> <p>1011 SYSWDTCDIV_134217728: Bit 27: 134,217,728 MCLK cycles</p> <p>1100 SYSWDTCDIV_268435456: Bit 28: 268,435,456 MCLK cycles</p> <p>1101 SYSWDTCDIV_536870912: Bit 29: 536,870,912 MCLK cycles</p> <p>1110 SYSWDTCDIV_1073741824: Bit 30: 1,073,741,824 MCLK cycles</p> <p>1111 SYSWDTCDIV_2147483648: Bit 31: 2,147,483,648 MCLK cycles</p> |
| WDTIE         | Bit 1    | <p>Watchdog timer interrupt enable. When set, watchdog timeout raises the watchdog interrupt level (vector 0), delivered to whichever hart the IRQROUTER routes it to. After the servicing hart completes the claim (IRQROUTER CLAIM/COMPLETE), the system resets if SYSWDTHWRST is set. If the vector is not routed to any hart, the reset (when enabled) occurs immediately on timeout.</p> <p>0 Watchdog interrupt disabled</p> <p>1 Watchdog interrupt enabled</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| WDTHWRST      | Bit 0    | <p>Watchdog hardware reset enable. When set, watchdog timeout causes system reset. If WDTIE is set, reset occurs after interrupt service routine completes. If WDTIE is cleared or IRQ is not enabled, reset occurs immediately on timeout.</p> <p>0 Watchdog reset disabled</p> <p>1 Watchdog reset enabled</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

### 15.6.8 WDTSR Register

Register memory slot: 14, offset: 56 bytes, size: 1 byte

Watchdog timer status register. Contains flags indicating watchdog reset and interrupt events. Flags are cleared by writing 1 to the respective bit.

|        |   |   |   |   |   |         |         |
|--------|---|---|---|---|---|---------|---------|
| 7      | 6 | 5 | 4 | 3 | 2 | 1       | 0       |
| Unused |   |   |   |   |   | WDTIF   | WDTRF   |
|        |   |   |   |   |   | rw1-(0) | rw1-(0) |

| Bitfield | Range    | Description                                                                                                                                                                                                                                                                       |
|----------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unused   | Bits 7-2 |                                                                                                                                                                                                                                                                                   |
| WDTIF    | Bit 1    | <p>Watchdog timer interrupt flag. Set when watchdog timeout occurs and WDTIE is enabled. Cleared by writing 1 to this bit. If not cleared before next timeout, indicates watchdog event occurred.</p> <p>0 No watchdog interrupt pending</p> <p>1 Watchdog interrupt occurred</p> |
| WDTRF    | Bit 0    | <p>Watchdog timer reset flag. Set when system reset was caused by watchdog timer. Persists across resets until cleared by software. Cleared by writing 1 to this bit.</p> <p>0 Reset not caused by watchdog</p> <p>1 Reset caused by watchdog</p>                                 |

### 15.6.9 WDTVAl Register

Register memory slot: 15, offset: 60 bytes, size: 4 bytes

Watchdog timer value register. Read-only register containing current watchdog counter value. Counter increments on MCLK when watchdog is enabled. Returns 0 when watchdog is disabled.

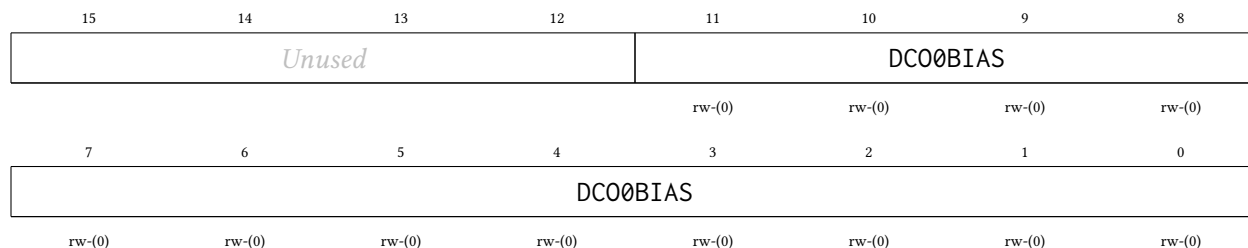
|        |       |       |       |       |       |       |       |
|--------|-------|-------|-------|-------|-------|-------|-------|
| 31     | 30    | 29    | 28    | 27    | 26    | 25    | 24    |
| Unused |       |       |       |       |       |       |       |
| 23     | 22    | 21    | 20    | 19    | 18    | 17    | 16    |
| WDTVAl |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 15     | 14    | 13    | 12    | 11    | 10    | 9     | 8     |
| WDTVAl |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 7      | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| WDTVAl |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |

| Bitfield | Range      | Description                                                                                                                                                  |
|----------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unused   | Bits 31-24 |                                                                                                                                                              |
| WDTVAl   | Bits 23-0  | <p>Watchdog counter value. 24-bit up-counter that increments on MCLK cycles. Watchdog event occurs when bit selected by WDTCDIV transitions from 0 to 1.</p> |

### 15.6.10 DCO0BIAS Register

Register memory slot: 16, offset: 64 bytes, size: 2 bytes

Digitally controlled oscillator 0 bias register. Controls DCO0 output frequency through bias voltage adjustment. Higher bias values generally produce higher frequencies.

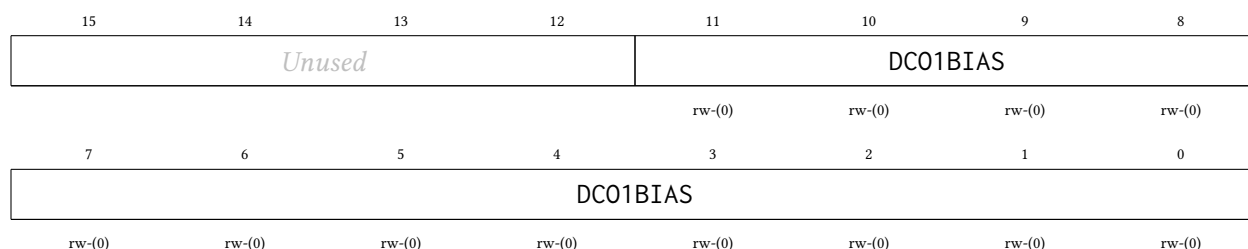


| Bitfield      | Range      | Description                                                                                                              |
|---------------|------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>Unused</i> | Bits 15-12 |                                                                                                                          |
| DCO0BIAS      | Bits 11-0  | DCO0 bias adjustment value. 12-bit bias control for DCO0 frequency tuning. Default value loaded from constants on reset. |

### 15.6.11 DCO1BIAS Register

Register memory slot: 17, offset: 68 bytes, size: 2 bytes

Digitally controlled oscillator 1 bias register. Controls DCO1 output frequency through bias voltage adjustment. Higher bias values generally produce higher frequencies.



| Bitfield      | Range      | Description                                                                                                              |
|---------------|------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>Unused</i> | Bits 15-12 |                                                                                                                          |
| DCO1BIAS      | Bits 11-0  | DCO1 bias adjustment value. 12-bit bias control for DCO1 frequency tuning. Default value loaded from constants on reset. |



## 16 NPU Peripheral

The NPU is a fixed-point neural network processing unit built around a single multiply-accumulate datapath and a configurable sequencer. One engine provides **four datapath modes**, selected per computation by the NPUMODE field (NPUCR bits 22:20): a fully-connected (dense) MLP layer (mode 0, the legacy datapath and the reset default), a one-dimensional convolution (mode 1), a binary XNOR-popcount layer (mode 2), and a general matrix–matrix multiply (mode 3). The peripheral operates on data staged in the shared NPU staging RAM (the 16 KiB SRAM occupying 0xC000–0xFFFF), which is multiplexed between the harts — reaching it through the multi-core arbiter — and the NPU’s own compute port. Software is responsible for staging operands into the staging RAM, configuring the NPU, and reading the results after the computation is complete.

The NPU’s *register bus* is in the shared peripheral page (see Section 4), so any hart may configure the NPU, start a computation, and poll for completion. The *data path* is likewise shared: all operands live in the shared NPU staging RAM at 0xC000–0xFFFF, and any hart may stage operands there or retrieve results through the arbiter. While a computation is running, the in-NPU port multiplexer switches the staging RAM over to the NPU’s compute port for the duration of the THINK, and **no hart is put to sleep**. Because the staging RAM belongs to the NPU during a run, software of any hart **must not read or write 0xC000–0xFFFF while a computation may be active**: poll NPUCR bit 16 (NPUTHINK) and wait for it to read 0 before touching the staging RAM.

### Data Formats

All non-binary NPU operands are fixed-point numbers, each stored in its own 32-bit SRAM word:

- **Inputs** (MLP/convolution inputs, GEMM A elements): signed Q0.24 format (25 bits: 1 sign bit + 24 fractional bits), representing values in  $[-1, 1)$ . Stored in bits 24:0 of a 32-bit SRAM word; bits 31:25 are ignored on read. Bit 24 is the sign bit.
- **Weights** (MLP/convolution weights, GEMM B elements): signed Q7.24 format (32 bits: 1 sign bit + 7 integer bits + 24 fractional bits), representing values in  $[-128, 128)$ .
- **Outputs**: signed Q7.24 format by default; the activation functions produce narrower ranges (see *Activation Functions* below). XNOR mode packs its 1-bit operands 32 per word (see mode 2 below) and emits  $\pm 1.0$  Q7.24 output words.

The accumulator is Q7.24 with **per-step round-to-nearest and saturation at  $\pm 128$** : every multiply-accumulate step rounds and clips individually, in the fixed sequencer order. Sums whose intermediate values exceed  $\pm 128$  clip; this is benign for normalized data but is a real ceiling for long dot products with large coefficients.

### Datapath Modes

The count registers NPUNI and NPUNN (NPUCR bits 15:8 and 7:0, each holding count – 1) and the per-mode configuration words NPUCFG1/NPUCFG2 are reinterpreted by each mode; the start-address registers NPUIVSAR/NPUWVSAR/NPUOVSAR always hold *word indices within the staging RAM* (byte offset from 0xC000 divided by 4).

- **Mode 0 — MLP (legacy, reset default)**. One dense layer: NPUNI + 1 inputs at NPUIVSAR, NPUNN + 1 output neurons. The weight matrix at NPUWVSAR holds one contiguous row per neuron; if bias is enabled (NPUBEN = 1) each row begins with one bias weight (multiplied by an implicit input of 1.0) followed by the NPUNI + 1 synaptic weights. NPUCFG1/NPUCFG2 are unused.

- **Mode 1 — 1-D convolution.** NPUNI = taps  $K - 1$ , NPUNN = filters  $C_{out} - 1$ . NPUCFG1 packs stride, dilation, input length, and input channel count; NPUCFG2 holds the host-computed output length  $L_{out}$  (the sequencer *trusts* it — the ‘valid’/‘same’ padding convention is a firmware decision). Inputs are channel-major at NPUIVSAR; weights are filter-major (bias first per filter when NPUBEN = 1); outputs are written filter-major. See the register descriptions for the exact field packing.
- **Mode 2 — XNOR-popcount (binary).** 1-bit activations and weights packed 32 per word, LSB-first (bit  $i$  of the layer lives at bit  $i \bmod 32$  of word  $i \div 32$ ). NPUNI = packed words per neuron  $- 1$ ; the *exact* bit count  $K \leq 4096$  lives in NPUCFG2 bits 12:0, and hardware masks the unused tail bits of each last packed word out of the popcount. A neuron fires when  $2 \cdot \text{popcount}(\text{XNOR}(a, w)) - K \geq$  the signed threshold in NPUCFG1, emitting  $+1.0$  ( $0x01000000$ ) else  $-1.0$  ( $0xFF000000$ ). NPUBEN and NPUAEN must be 0 in this mode.
- **Mode 3 — GEMM.**  $C = A \times B$  with  $A$  ( $M \times K$ , row-major at NPUIVSAR, Q0.24 elements in  $[-1, 1)$ ),  $B$  ( $K \times N$ , **column-major** at NPUWVSAR, Q7.24), and  $C$  ( $M \times N$ , row-major at NPUOVSAR).  $K - 1$  in NPUNI,  $N - 1$  in NPUNN,  $M - 1$  in NPUCFG1 bits 7:0. The working set  $M \cdot K + K \cdot N + M \cdot N$  must fit the 4096-word staging RAM; larger problems tile by re-pointing the start-address registers between computations (there is **no hardware accumulation across tiles**, so  $K$  must fit in one run). Because  $C$  is row-major, a result matrix chains directly as the next layer’s  $A$  operand with no repacking (activated outputs are valid Q0.24 inputs; see below). NPUBEN must be 0 in this mode.

**The sequencer trusts its configuration:** there are no hardware bounds checks. A working set that exceeds the 4096-word staging RAM, or count/length fields inconsistent with the staged data, wraps the 12-bit word addresses and silently corrupts neighbouring staging-RAM regions — the engine never hangs, but the results (and possibly unrelated staged data) are garbage. Firmware owns operand sizing and tiling.

### Activation Functions

When NPUAEN = 1, each output is passed through the activation selected by NPUACTF (NPUCR bits 25:23) before being written; NPUAEN = 0 writes the raw Q7.24 accumulator regardless of NPUACTF. All activations share the one hardware sigmoid approximator (a piecewise-quadratic logistic function):

- **0 — sigmoid** (reset default):  $\sigma(x)$ , output Q0.24 in  $[0, 1)$ ; saturates for  $|x| \geq 4$ .
- **1 — ReLU:**  $\max(0, x)$ , output Q7.24. Note that a ReLU output at or above 1.0 is *not* a valid Q0.24 next-layer input; chain through normalized weights or use the clamp.
- **2 — tanh** approximation, computed as  $2\sigma(2x) - 1$  through the same sigmoid hardware: output in  $[-1, 1)$ , sign-extended, whose low 25 bits are a valid Q0.24 next-layer input. Saturates for  $|x| \geq 2$  (half the sigmoid’s linear region, a consequence of the input doubling).
- **3 — clamp** (hardtanh): the identity saturated to the Q0.24 rails  $[-1, 1)$ , sign-extended — a bounded linear activation whose output always chains as a Q0.24 input.
- **4 — exponential approximation**  $\widetilde{\exp}(x) = 2\sigma(x)$ , output Q1.24 in  $[0, 2)$ , for the softmax leg: hardware emits the per-element scores only, and **firmware performs the sum and normalization**. Subtracting the maximum logit before the run is recommended; the maximum element then maps to exactly 1.0.
- Codes 5–7 are reserved and behave as 0 (sigmoid).

The activation selection is latched when THINK is set, so a mid-computation write to NPUACTF (or NPUMODE) takes effect at the *next* computation. XNOR mode (mode 2) ignores the activation path entirely — its  $\pm 1.0$  encoding is fixed.

## Operation

1. Stage the operands into the staging RAM per the selected mode's layout and write their word indices to NPUIVSAR and NPUWVSAR.
2. Write the desired output word index to NPUOVSAR.
3. Write the per-mode configuration (NPUCFG1/NPUCFG2) if the mode uses it.
4. Configure NPUCR: NPUNI, NPUNN, NPUMODE, NPUBEN, NPUAEN, and NPUACTF as required by the mode.
5. Set NPUTHINK (NPUCR bit 16) to 1 to start the computation. The staging RAM is now owned by the NPU; no hart may access  $0xC000-0xFFFF$  until the computation completes.
6. Poll NPUCR bit 16 (NPUTHINK) until it reads 0, indicating that the computation is complete and the staging RAM has been released back to the harts — or take the think-done interrupt (below).
7. Read the outputs from the staging RAM at the word index stored in NPUOVSAR.

Run times scale with the mode's loop product (e.g. a GEMM costs on the order of  $5 \cdot M \cdot N \cdot K$  NPU clock cycles; a convolution  $5 \cdot C_{out} \cdot L_{out} \cdot C_{in} \cdot K$ ). Full-scope convolutions can reach hundreds of milliseconds at 24 MHz — size polling budgets (or use the think-done interrupt) accordingly. XNOR mode is the cheapest per output by roughly an order of magnitude (32 synapses per fetched word pair).

## Think-done interrupt (vector 120)

As an alternative to polling, the NPU raises the *think-done* interrupt (interrupt vector 120) when a computation completes. The NPUTHINKDONE flag (NPUSR bit 0) is set once per computation — by the same internal event that self-clears NPUTHINK — and is *sticky*: it remains set until software clears it by writing a 1 to NPUSR bit 0 (write-1-to-clear; writing 0 has no effect, and reading NPUSR never clears it). The interrupt is a level, asserted while both NPUTHINKDONE and the enable bit NPUTDIE (NPUCR bit 19) are set; it is routed, claimed, and completed through the interrupt router like every other peripheral source. NPUTDIE resets to 0, so software that polls is unaffected by the interrupt's existence. Every mode uses the same completion event and the same vector.

A typical interrupt-driven flow arms NPUTDIE, starts the computation, and sleeps; the handler clears NPUTHINKDONE (write-1-to-clear) before completing the interrupt at the router. **The staging-RAM ownership contract is unchanged by the interrupt:** the staging RAM belongs to the NPU while a computation may be active, and no hart may access  $0xC000-0xFFFF$  until NPUCR bit 16 (NPUTHINK) reads 0. Taking the think-done interrupt implies the computation has finished, but any hart *other* than the one servicing the interrupt must still consult NPUTHINK before touching the staging RAM.

## 16.1 Register Description

### 16.1.1 NPUCR Register

Register memory slot: 0, offset: 0 bytes, size: 4 bytes

NPU control register

|        |        |        |        |        |        |        |         |
|--------|--------|--------|--------|--------|--------|--------|---------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24      |
| Unused |        |        |        |        |        | ACTF   |         |
|        |        |        |        |        |        | rw-(0) | rw-(0)  |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16      |
| ACTF   | MODE   |        |        | TDIE   | BEN    | AEN    | THINK   |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw1-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8       |
| NI     |        |        |        |        |        |        |         |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0)  |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0       |
| NN     |        |        |        |        |        |        |         |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0)  |

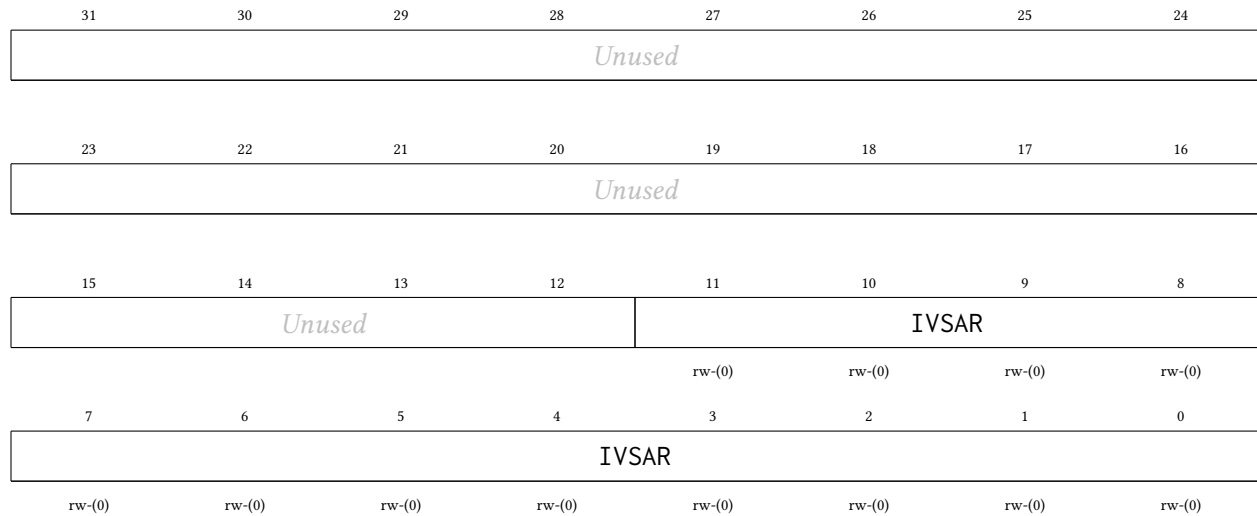
| Bitfield | Range      | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unused   | Bits 31-26 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| ACTF     | Bits 25-23 | Post-accumulator activation function select, effective when NPUAEN = 1 (P4 architecture family; NPUAEN = 0 is always raw passthrough regardless of this field). 0 = logistic sigmoid approximation (the legacy activation; reset default), output Q0.24 in [0,1). 1 = ReLU (max(0, x)), output Q7.24 – note a ReLU output at or above 1.0 is not a valid Q0.24 next-layer input. 2 = tanh approximation computed as 2*sigmoid(2x)-1 through the same sigmoid hardware, output in [-1,1) sign-extended (the low 25 bits are a valid Q0.24 next-layer input); saturates for  x  >= 2. 3 = clamp (hardtanh): the identity saturated to the Q0.24 rails [-1, 1), sign-extended. 4 = exponential approximation 2*sigmoid(x), output Q1.24 in [0,2), for the softmax leg – hardware emits per-element scores only, firmware performs the sum and normalize (subtracting the maximum logit first is recommended, mapping the maximum to exactly 1.0). Codes 5-7 are reserved and behave as 0 (sigmoid). The selection is latched when THINK is set; a mid-THINK write takes effect at the next THINK. |

|       |            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MODE  | Bits 22-20 | Datapath mode select for the NPU architecture family (P4). 0 = multilayer-perceptron mode (the legacy datapath; reset default). 1 = one-dimensional convolution mode. 2 = XNOR-popcount binary mode (1-bit activations/weights packed 32 per word, LSB-first; NPUBEN and NPUAEN must be 0 in this mode). 3 = general matrix-multiply (GEMM) mode: $C = A \times B$ with A ( $M \times K$ , row-major at NPUIVSAR, Q0.24 elements in $[-1,1]$ ), B ( $K \times N$ , column-major at NPUWVSAR, Q7.24), C ( $M \times N$ , row-major at NPUOVSAR, Q7.24 or Q0.24 through the sigmoid when NPUAEN=1); K-1 in NPUNI, N-1 in NPUNN, M-1 in NPUCFG1 bits 7:0; NPUBEN must be 0 in this mode; the working set $M \times K + K \times N + M \times N$ must fit the 4096-word staging RAM (larger problems tile by re-pointing the start-address registers between THINKs — there is no hardware accumulation across tiles). Codes 4-7 are reserved. Reserved and unimplemented codes behave as mode 0. |
| TDIE  | Bit 19     | Think-done interrupt enable. When set, NPUSR.THINKDONE drives the NPU think-done interrupt (vector 120). When cleared (reset default) the NPU is polling-only, exactly as before the interrupt existed.<br>0 Interrupt disabled (polling only)<br>1 Interrupt enabled                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| BEN   | Bit 18     | Bias enable. When set, the first weight of each output neuron's row in the weight matrix is used as a bias term: it is multiplied by an implicit input of 1.0 and accumulated before the synaptic weights.<br>0 Disabled<br>1 Enabled                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| AEN   | Bit 17     | Activation function enable. When set, the logistic sigmoid approximation activation function is applied to the accumulator output. When cleared, the raw accumulator output is used (linear/identity).<br>0 Disabled (linear output)<br>1 Enabled (logistic sigmoid approximation)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| THINK | Bit 16     | NPU computation start and status bit. Write 1 to start the NPU. Self-clears when the computation is complete. Poll this bit to determine when results are ready.<br>0 Idle (computation complete or not started)<br>1 Running (write 1 to start)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| NI    | Bits 15-8  | Number of inputs in the input vector minus 1. The actual number of inputs is NPUNI + 1.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| NN    | Bits 7-0   | Number of output neurons minus 1. The actual number of outputs is NPUNN + 1.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

### 16.1.2 NPUIVSAR Register

Register memory slot: 1, offset: 4 bytes, size: 4 bytes

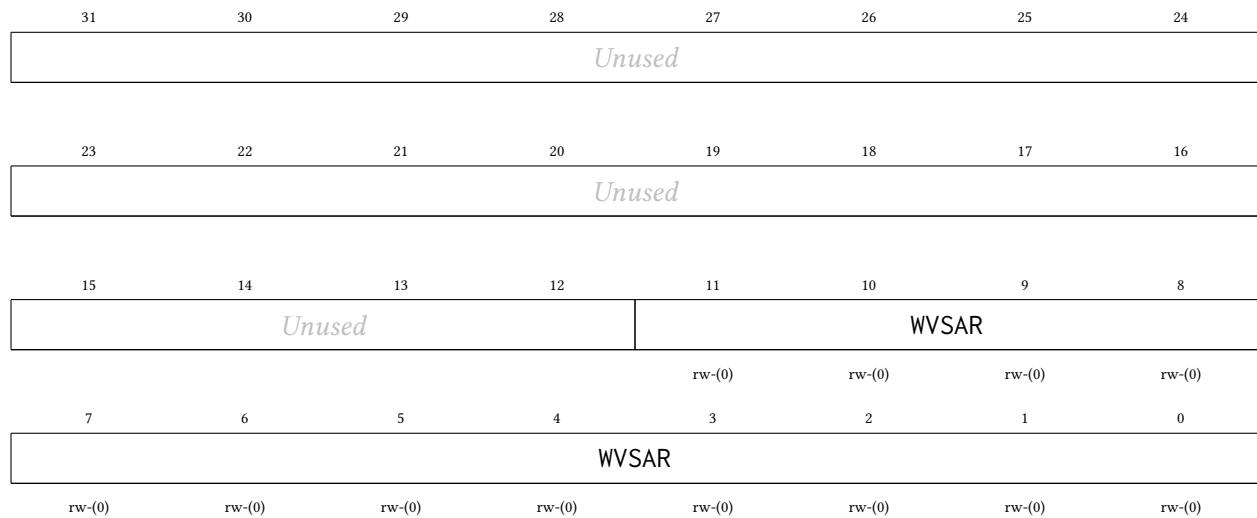
Input vector start word index within the shared NPU staging RAM: the byte offset from the start of the staging RAM (0xC000) divided by 4. For example, an input vector at byte address 0xC100 has word index 0x40. Each input is a signed Q0.24 value stored in bits 24:0 of its 32-bit SRAM word; bits 31:25 are ignored. Bit 24 is the sign bit. The input at index 0 is at word index NPUIVSAR. The input at index 1 is at word index NPUIVSAR + 1. The rest of the inputs follow in consecutive words.



### 16.1.3 NPUWVSAR Register

*Register memory slot: 2, offset: 8 bytes, size: 4 bytes*

Synaptic weight matrix start word index within the shared NPU staging RAM: the byte offset from the start of the staging RAM (0xC000) divided by 4. Each weight is a signed Q7.24 value occupying all 32 bits of its SRAM word; bit 31 is the sign bit. Weights are stored row-major, one per 32-bit word, in the following order: for each output neuron (0 through NPUNN), if bias is enabled (NPUBEN = 1), the first word in the row is the bias weight (multiplied by an implicit input of 1.0), followed by NPUNI + 1 synaptic weights for inputs 0 through NPUNI. If bias is disabled, each row contains NPUNI + 1 synaptic weights only.

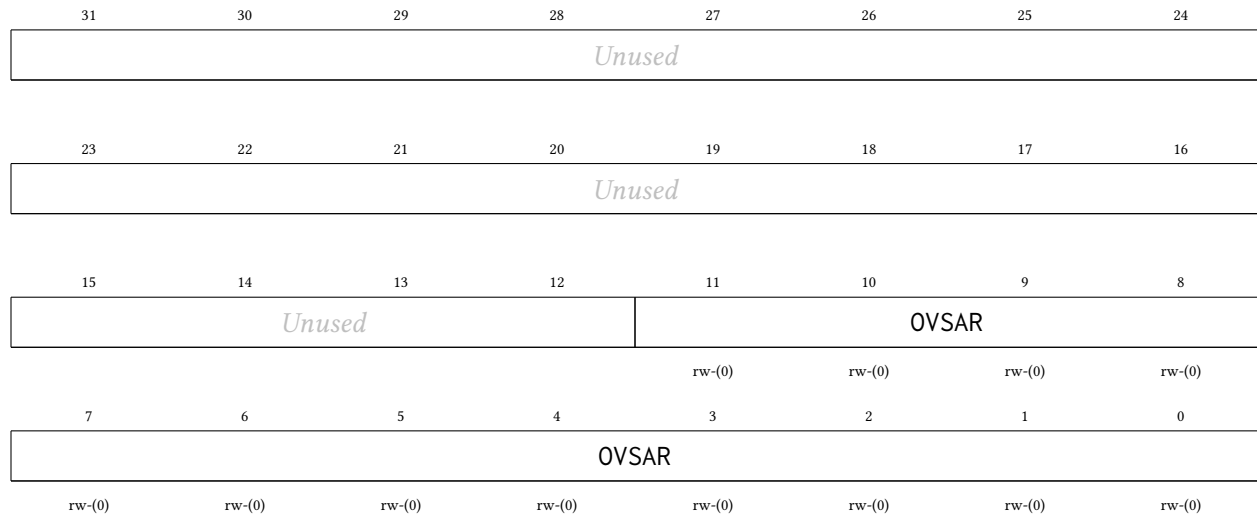


### 16.1.4 NPUOVSAR Register

*Register memory slot: 3, offset: 12 bytes, size: 4 bytes*

Output vector start word index within the shared NPU staging RAM: the byte offset from the start of the staging RAM (0xC000) divided by 4. Each output is a signed Q7.24 value occupying all 32 bits of its SRAM

word; bit 31 is the sign bit. The output at index 0 is written to word index NPUOVSAR. The output at index 1 is at word index NPUOVSAR + 1, and so on.



### 16.1.5 NPUSR Register

Register memory slot: 4, offset: 16 bytes, size: 4 bytes

NPU status register. THINKDONE is write-1-to-clear (write a 1 to bit 0 to clear it; writing 0 leaves it unchanged) and is never cleared by a read. The NPU think-done interrupt (vector 120) is THINKDONE and NPUCR.NPUTDIE. Clearing THINKDONE does not affect NPUCR.NPUTHINK, which self-clears at completion as before; the staging-RAM ownership contract is unchanged (no hart may access 0xC000-0xFFFF until NPUCR bit 16 reads 0 again).



| Bitfield      | Range     | Description |
|---------------|-----------|-------------|
| <i>Unused</i> | Bits 31-1 |             |

|           |       |                                                                                                                                                                                                                                                                                                                                                    |
|-----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| THINKDONE | Bit 0 | Think-done flag. Set once per computation, when the NPU finishes (the same event that self-clears NPUCR.NPUTHINK); sticky. Drives the think-done interrupt (vector 120) when NPUCR.NPUTDIE is set. Write 1 to clear. On a same-cycle collision between a completing computation and a write-1-to-clear, the set wins (a completion is never lost). |
|           | 0     | No completed computation pending                                                                                                                                                                                                                                                                                                                   |
|           | 1     | A computation has completed                                                                                                                                                                                                                                                                                                                        |

### 16.1.6 NPUCFG1 Register

*Register memory slot: 5, offset: 20 bytes, size: 4 bytes*

NPU mode configuration word 1. The interpretation depends on NPUCR.NPUMODE. Multilayer-perceptron mode (0): unused, no effect. One-dimensional convolution mode (1): bits 3:0 = stride S (1-15), bits 7:4 = dilation D (1-15), bits 23:8 = input length L per channel in samples, bits 31:24 = number of input channels minus 1 (the actual channel count  $C_{in}$  is this field + 1). XNOR-popcount mode (2): the full 32-bit signed firing threshold THRESH — a neuron fires (output +1.0) when  $2 * \text{popcount}(\text{XNOR}(\text{activations}, \text{weights})) - K$  is greater than or equal to THRESH, else outputs -1.0. GEMM mode (3): bits 7:0 = M - 1, the number of A rows minus 1 (the actual row count M is this field + 1); bits 31:8 are ignored in this mode.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| CFG1   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| CFG1   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| CFG1   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| CFG1   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

### 16.1.7 NPUCFG2 Register

*Register memory slot: 6, offset: 24 bytes, size: 4 bytes*

NPU mode configuration word 2. The interpretation depends on NPUCR.NPUMODE. Multilayer-perceptron mode (0): unused, no effect. One-dimensional convolution mode (1): bits 15:0 = output length Lout per filter in samples. Lout is computed by the host (the sequencer trusts this value; the valid/same padding convention is a firmware decision) and every referenced input sample index  $j*S + k*D$  must be less than L. XNOR-popcount mode (2): bits 12:0 = K, the exact input bit count (1-4096); NPUNI must hold  $\text{ceil}(K/32) - 1$ , the packed words per neuron, and when  $K \bmod 32$  is nonzero the unused high bits of each last packed word are masked out of the popcount by hardware. GEMM mode (3): unused, no effect. Bits 31:16 are reserved and read 0.



|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| Unused |        |        |        |        |        |        |        |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| Unused |        |        |        |        |        |        |        |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| CFG2   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| CFG2   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

## 17 PWRCTRL Peripheral

The power controller manages the switchable MTCMOS power domains of the tile harts (harts 1–3). Each tile is implemented as its own header-switched power domain: PMOS header switches sit between the always-on supply grid and the tile’s local rail, isolation cells clamp every signal leaving the tile while it is unpowered, and a per-tile hardware sequencer guarantees the controls can only ever be applied in the legal order. Hart 0 — the management hart, which owns the SPI boot path, the console UART, and the CLINT — is permanently powered and cannot be gated. The main features are listed below:

- One independently switchable power domain per tile hart (harts 1–3)
- Single-bit gate/ungate control per tile; a hardware sequencer orders isolation, reset, and the header-switch enable correctly in both directions
- Cold gating: a gated tile consumes no dynamic or switched-leakage power and loses **all** state, including its TCM
- Wake equals boot: an ungated tile cold-boots through the shared boot ROM, parks in WFI, and is reloaded with the standard loader-row + msip launch sequence
- Read-only per-tile sequencer state for software polling
- All domains power up ON at reset — chip boot is unaffected and the controller is inert until software gates a tile

### 17.1 Usage

To gate tile hart  $h$  (1–3), first ensure the tile is parked or otherwise quiesced — typically it is sitting in its boot-ROM WFI park, or software has confirmed it finished its work. Then set the gate bit and, if required, wait for the sequencer to report the domain off:

```
li      t0, PWRCTRL_BASE_ADDR
lw      t1, PWRCR(t0)
ori     t1, t1, (1 << h)      # request gate of tile h
sw      t1, PWRCR(t0)
gate_poll:
lw      t2, PWRSR(t0)
srli    t2, t2, (4 * h)
andi    t2, t2, 0xF
li      t3, 3                 # 3 = OFF
bne     t2, t3, gate_poll
```

To wake the tile, clear the same bit and wait for the state nibble to return to 0 (ON). The tile then re-executes the shared boot ROM from address 0, parks in WFI, and is relaunched exactly like at chip power-on: write the loader row (source, length, entry) and send the tile an msip.

Because gating is *cold*, everything on the tile dies with the rail: the register file, CSRs, and the private TCM contents. Any data the tile must keep across a power cycle has to be staged to shared RAM before gating. The isolation clamps drive every outbound tile signal to its inactive level, which is identical to the tile’s reset state, so the rest of the chip observes a gated tile simply as a silent one.

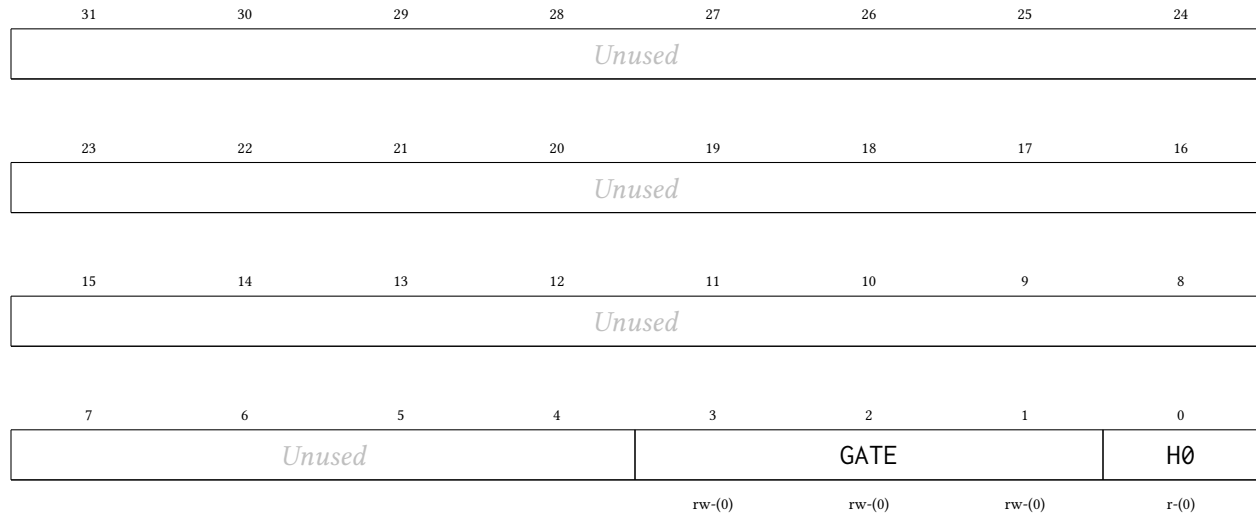
The sequencer never aborts a transition: a gate-bit change made mid-sequence takes effect when the current sequence completes. The rail-settle delay on wake is sized for the header-switch daisy chain of one tile domain; software does not need to add its own delays beyond polling PWRSR.

## 17.2 Register Description

### 17.2.1 PWRCR Register

Register memory slot: 0, offset: 0 bytes, size: 4 bytes

Power gate control. Setting GATE bit h powers tile hart h down (isolation, reset, rail off); clearing it powers the tile back up and cold-boots it. A request made while the sequencer is mid-sequence is honored when the sequence completes (no aborts). Bit 0 (hart 0) is reserved: always-on, reads 0, writes ignored.

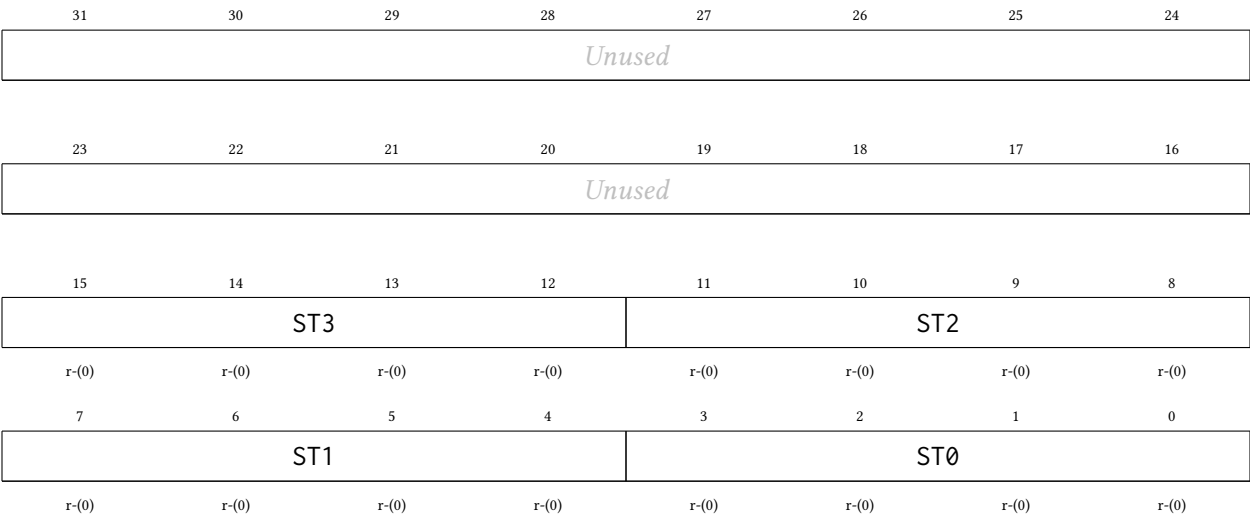


| Bitfield | Range     | Description                                                                                                                                                                 |
|----------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unused   | Bits 31-4 |                                                                                                                                                                             |
| GATE     | Bits 3-1  | Gate request per tile hart: bit h = 1 powers tile hart h down, 0 powers it up (cold boot). Poll PWRSR for sequencer completion.<br>000 PWRGATE_NONE: All tile harts powered |
| H0       | Bit 0     | Hart 0 is always-on: reads 0, writes ignored.                                                                                                                               |

### 17.2.2 PWRSR Register

Register memory slot: 1, offset: 4 bytes, size: 4 bytes

Power sequencer state, one read-only nibble per hart (bits 4h+3:4h). 0 = ON, 1 = ISO (clamps asserting), 2 = RSTOFF (reset held, rail dying), 3 = OFF (gated), 4 = RAIL (waking, rail settling), 5 = UNISO (clamps releasing). Hart 0's nibble always reads 0. A tile is safely gated when its nibble reads 3, and fully awake (booting or parked in the ROM) when it returns to 0.



| Bitfield | Range      | Description                                    |
|----------|------------|------------------------------------------------|
| Unused   | Bits 31-16 |                                                |
| ST3      | Bits 15-12 | Tile hart 3 sequencer state.                   |
| ST2      | Bits 11-8  | Tile hart 2 sequencer state.                   |
| ST1      | Bits 7-4   | Tile hart 1 sequencer state.                   |
| ST0      | Bits 3-0   | Hart 0 state: always 0 (ON, always-on domain). |

## 18 I2Cx Peripheral

I2C serial port interface. The master and slave I2C interfaces are split between two sets of registers.

To use master transmitter mode, first configure the I2C peripheral by setting I2CMEN, clearing I2CSEN, and configuring I2CMDIV with the appropriate clock division factor, noting that the I2C clock source is SMCLK. To send a start condition, set I2CMST, wait for the I2CMSTS flag to be set (if the bus is busy, the I2C peripheral will wait for it to become idle and then send a start condition), and then clear the status register. I2CMCB will now indicate that the I2C peripheral now has control of the bus as its master. Next, write to I2CxMTX the desired slave address in the most significant 7 bits followed by the desired read/write bit (0 for write/master transmitter) in the least significant bit. Then, wait for I2CMXC or I2CMARB to be set. If I2CMARB is set, then the I2C peripheral has lost the bus arbitration contest and has released control of the bus. If I2CMNR is set, then the desired slave has not acknowledged itself. Clear the status register. Next, send the slave a byte of data by writing the desired data to I2CxMTX. When the I2C peripheral is ready for another byte of data to be queued for transmission, the I2CMTXE flag will be set. Again, wait for I2CMXC or I2CMARB, then check I2CMARB and I2CMNR, and finally clear the status register. Once finished sending all of the desired bytes, either send a stop condition to release control of the bus by setting I2CMSP, or send a repeated start condition to retain control of the bus with a new transmission (and possibly a new slave and read/write mode) by setting I2CMST. Once a stop condition is sent, wait for I2CMSTS to be set, indicating that a stop condition has been sent. Clear the status register.

To use master receiver mode, first configure the I2C peripheral by setting I2CMEN, clearing I2CSEN, and configuring I2CMDIV with the appropriate clock division factor, noting that the I2C clock source is SMCLK. To send a start condition, set I2CMST, wait for the I2CMSTS flag to be set (if the bus is busy, the I2C peripheral will wait for it to become idle and then send a start condition), and then clear the status register. I2CMCB will now indicate that the I2C peripheral now has control of the bus as its master. Next, write to I2CxMTX the desired slave address in the most significant 7 bits followed by the desired read/write bit (1 for read/master receiver) in the least significant bit. Then, wait for I2CMXC or I2CMARB to be set. If I2CMARB is set, then the I2C peripheral has lost the bus arbitration contest and has released control of the bus. If I2CMNR is set, then the desired slave has not acknowledged itself. Clear the status register. Next, begin to receive a byte of data from the slave by setting I2CMRB. Wait for I2CMXC to be set. Clear the status register. Read I2CxMRX to get the byte of data received from the slave. To send the slave an ACK and begin to read another byte from the slave, set I2CMRB. Or, to send the slave a NACK and send a stop condition, set I2CMSP. Or, to send the slave a NACK and send a repeated start condition, set I2CMST. Wait for the appropriate flag, then clear the status register.

To use slave receiver mode, first configure the I2C peripheral by setting I2CSEN, clearing I2CSEN, clearing I2CSN, and configuring I2CSCS and I2CGCE to the desired values. Note that if clock stretching is enabled with I2CSCS, the I2C peripheral will seize control of the bus by driving SCL low during every ACK/NACK bit transfer (if the I2C peripheral was addressed) until I2CSC is set, which requires user intervention to prevent indefinite hold-ups of the I2C bus. But, if clock stretching is not enabled, the master will be allowed full control of the rate data is sent over the bus, which opens the possibility that the software running on this MCU does not notice that a byte has been transferred in time before the next is transferred. Note that if I2CGCE is set, the I2C peripheral will be addressed if either its address is received or if the general call is received. Wait for the I2C peripheral to be addressed when I2CSA is set. Check I2CSTM to see if the master has requested slave receiver or slave transmitter mode (0 indicates slave receiver). Clear the status register. If clock stretching is enabled, send an ACK or NACK by clearing or setting I2CSN, and then set I2CSC to release SDA and continue with the transfer. If clock stretching is not enabled, the I2C peripheral will automatically ACK or NACK depending on the value of I2CSN. Next, wait for the slave to receive a data byte from the master when I2CSXC is set. If I2CSOVF is set, then the MCU has failed to read one of the bytes sent by the master in the past. Clear the status register, and then read I2CSRX to get

the data byte sent from the master. If clock stretching is enabled, send an ACK or NACK by clearing or setting I2CSN, and then set I2CSC to release SDA and continue with the transfer. If clock stretching is not enabled, the I2C peripheral will automatically ACK or NACK depending on the value of I2CSN. Next, wait for I2CSXC, I2CSPR, or I2CSTR to be set, indicating the I2C peripheral has received a new byte of data, a stop condition, or a repeated start condition. If a stop or start condition has been received, clear the status register.

To use slave transmitter mode, first configure the I2C peripheral by setting I2CSEN, clearing I2CSEN, clearing I2CSN, and configuring I2CSCS and I2CGCE to the desired values. Note that if clock stretching is enabled with I2CSCS, the I2C peripheral will seize control of the bus by driving SCL low during every ACK/NACK bit transfer (if the I2C peripheral was addressed) until I2CSC is set, which requires user intervention to prevent indefinite hold-ups of the I2C bus. But, if clock stretching is not enabled, the master will be allowed full control of the rate data is sent over the bus, which opens the possibility that the software running on this MCU does not notice that a byte has been transferred in time before the next is transferred. Check I2CSTM to see if the master has requested slave receiver or slave transmitter mode (1 indicates slave transmitter). Clear the status register. Queue the byte of data to transmit to the master by writing the byte to I2CSTX. If clock stretching is enabled, set I2CSC to release SDA and continue with the transfer. If clock stretching is not enabled, the I2C peripheral will automatically ACK or NACK depending on the value of I2CSN. Wait for I2CSTXE to be set, indicating that the I2C peripheral is ready to queue the next byte to send to the master. Clear the status register, and write the next byte to send to the master to I2CxSTX. If clock stretching is enabled, wait for I2CSXC to be set, clear the status register, and set I2CSC. Wait for I2CSTXE, I2CSPR, or I2CSTR to be set, then clear the status register.

## 18.1 Register Description

### 18.1.1 I2CCR Register

Register memory slot: 0, offset: 0 bytes, size: 4 bytes

I2C control register

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| Unused |        |        |        |        |        |        |        |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| Unused |        | MEN    | SEN    | SN     | SCS    | GCE    | MDIV   |
| rw-(0) |        | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| MDIV   |        | SAIE   | STXEIE | SOVFIE | SNRIE  | SXCIE  |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| MSTSIE | MSPSIE | MARBIE | MTXEIE | MNRIE  | MXCIE  | STRIE  | SPRIE  |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range      | Description |
|----------|------------|-------------|
| Unused   | Bits 31-22 |             |

|      |            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MEN  | Bit 21     | <p>I2C master enable. When enabled, this device awaits a command to send a start condition with I2CMST, and then begins acting as a master until commanded to send a stop condition with I2CMSP. If master mode is also enabled on this device, then this device will act as a slave until commanded to send a start condition with I2CMST, whereupon it will begin acting as a master. Once the master transfer is complete, it will resume acting as a slave.</p> <p>0 Disabled</p> <p>1 Enabled</p>                                  |
| SEN  | Bit 20     | <p>I2C slave enable. When enabled, this device behaves as an I2C slave and begins listening for its address. If master mode is also enabled on this device, then this device will act as a slave until commanded to send a start condition with I2CMST, whereupon it will begin acting as a master. Once the master transfer is complete, it will resume acting as a slave.</p> <p>0 Disabled</p> <p>1 Enabled</p>                                                                                                                      |
| SN   | Bit 19     | <p>I2C slave NACK next byte received. When enabled, this slave will reply with a NACK whenever it receives its address or whenever it receives a byte from a master. When disabled, it will send an ACK in those situations. If clock stretching is enabled, this bit can be changed in accordance with the desired ACK/NACK reply before allowing a rising edge of SCL.</p> <p>0 ACK</p> <p>1 NACK</p>                                                                                                                                 |
| SCS  | Bit 18     | <p>I2C slave clock stretching enable. When enabled, this slave will hold the SCL line low during the ACK phase of the transmission to allow this slave more time. Note that the master will be left waiting for the ACK/NACK as long as this bit is set to 1.</p> <p>0 Disabled</p> <p>1 Enabled</p>                                                                                                                                                                                                                                    |
| GCE  | Bit 17     | <p>I2C slave general call enable. When enabled, this slave will be addressed if a global call is issued on the bus.</p> <p>0 Disabled</p> <p>1 Enabled</p>                                                                                                                                                                                                                                                                                                                                                                              |
| MDIV | Bits 16-13 | <p>I2C master mode clock divider. The master mode finite state machine clock source is SMCLK, which is divided by a factor of <math>4 * 2^{**}I2CMDIV</math>.</p> <p>0000 I2CMDIV_1: /1 (no division)</p> <p>0001 I2CMDIV_2: /2</p> <p>0010 I2CMDIV_4: /4</p> <p>0011 I2CMDIV_8: /8</p> <p>0100 I2CMDIV_16: /16</p> <p>0101 I2CMDIV_32: /32</p> <p>0110 I2CMDIV_64: /64</p> <p>0111 I2CMDIV_128: /128</p> <p>1000 I2CMDIV_256: /256</p> <p>1001 I2CMDIV_512: /512</p> <p>1010 I2CMDIV_1024: /1,024</p> <p>1011 I2CMDIV_2048: /2,048</p> |

|        |        |                                                           |                        |
|--------|--------|-----------------------------------------------------------|------------------------|
|        |        | 1100                                                      | I2CMDIV_4096: /4,096   |
|        |        | 1101                                                      | I2CMDIV_8192: /8,192   |
|        |        | 1110                                                      | I2CMDIV_16384: /16,384 |
|        |        | 1111                                                      | I2CMDIV_32768: /32,768 |
| SAIE   | Bit 12 | I2C slave mode addressed interrupt enable                 |                        |
|        |        | 0                                                         | Interrupt disabled     |
|        |        | 1                                                         | Interrupt enabled      |
| STXEIE | Bit 11 | I2C slave transmit register empty interrupt enable        |                        |
|        |        | 0                                                         | Interrupt disabled     |
|        |        | 1                                                         | Interrupt enabled      |
| SOVFIE | Bit 10 | I2C slave receive register overflow interrupt enable      |                        |
|        |        | 0                                                         | Interrupt disabled     |
|        |        | 1                                                         | Interrupt enabled      |
| SNRIE  | Bit 9  | I2C slave mode NACK received from master interrupt enable |                        |
|        |        | 0                                                         | Interrupt disabled     |
|        |        | 1                                                         | Interrupt enabled      |
| SXCIE  | Bit 8  | I2C slave mode transfer complete interrupt enable         |                        |
|        |        | 0                                                         | Interrupt disabled     |
|        |        | 1                                                         | Interrupt enabled      |
| MSTSIE | Bit 7  | I2C master mode start condition sent interrupt enable     |                        |
|        |        | 0                                                         | Interrupt disabled     |
|        |        | 1                                                         | Interrupt enabled      |
| MSPSIE | Bit 6  | I2C master mode stop condition sent interrupt enable      |                        |
|        |        | 0                                                         | Interrupt disabled     |
|        |        | 1                                                         | Interrupt enabled      |
| MARBIE | Bit 5  | I2C master mode arbitration error interrupt enable        |                        |
|        |        | 0                                                         | Interrupt disabled     |
|        |        | 1                                                         | Interrupt enabled      |
| MTXEIE | Bit 4  | I2C master transmit register empty interrupt enable       |                        |
|        |        | 0                                                         | Interrupt disabled     |
|        |        | 1                                                         | Interrupt enabled      |
| MNRIE  | Bit 3  | I2C master mode NACK received interrupt enable            |                        |
|        |        | 0                                                         | Interrupt disabled     |
|        |        | 1                                                         | Interrupt enabled      |
| MXCIE  | Bit 2  | I2C master transfer complete interrupt enable             |                        |
|        |        | 0                                                         | Interrupt disabled     |
|        |        | 1                                                         | Interrupt enabled      |
| STRIE  | Bit 1  | I2C start received interrupt enable                       |                        |
|        |        | 0                                                         | Interrupt disabled     |
|        |        | 1                                                         | Interrupt enabled      |
| SPRIE  | Bit 0  | I2C stop received interrupt enable                        |                        |
|        |        | 0                                                         | Interrupt disabled     |
|        |        | 1                                                         | Interrupt enabled      |

### 18.1.2 I2CFR Register

Register memory slot: 1, offset: 4 bytes, size: 1 byte



I2C flow control register. Writing a 1 to a bit in this register initiates or queues the associated command. Writing a 0 to a bit does nothing. Reading this register always returns the value 0.

|               |   |   |   |        |        |        |        |
|---------------|---|---|---|--------|--------|--------|--------|
| 7             | 6 | 5 | 4 | 3      | 2      | 1      | 0      |
| <i>Unused</i> |   |   |   | SC     | MST    | MSP    | MRB    |
|               |   |   |   | w1-(0) | w1-(0) | w1-(0) | w1-(0) |

| Bitfield      | Range    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|---------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Unused</i> | Bits 7-4 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| SC            | Bit 3    | I2C slave continue command. When clock stretching is enabled in slave mode, set this bit to tell the slave to continue with the ACK/NACK phase of the current byte by releasing SCL. This may only be set if clock stretching is enabled, slave mode is enabled, and the slave transfer complete flag has just been set.<br>0 No effect<br>1 Send a start condition                                                                                                                                                                                                                                                 |
| MST           | Bit 2    | I2C master send start condition command. Set this bit to 1 to send a start condition or a repeated start condition. Once this bit is set, this master is required to send at least one address frame before initiating this command again. If another master has control of the bus when this bit is set, this master will wait until the bus is idle before seizing control of it and sending the start condition. If this master is busy with a transaction when this bit is set, then it will send a restart condition immediately after it finishes the transaction.<br>0 No effect<br>1 Send a start condition |
| MSP           | Bit 1    | I2C master send stop condition command. Set this bit to 1 while this master is in control of the bus to send a stop condition. This master is required to have already sent a start condition and at least one address frame before initiating this command. If this master is busy with a transaction when this bit is set, then it will send the stop condition immediately after it finishes the transaction.<br>0 No effect<br>1 Send a stop condition                                                                                                                                                          |
| MRB           | Bit 0    | I2C master read byte command. Set this bit to 1 while in master receiver mode to read one byte from the slave. This master is required to have already sent the slave address and received an ACK from the slave before initiating this command.<br>0 No effect<br>1 Read next byte                                                                                                                                                                                                                                                                                                                                 |

### 18.1.3 I2CSR Register

Register memory slot: 2, offset: 8 bytes, size: 2 bytes

I2C status register

|         |         |         |         |         |         |         |         |
|---------|---------|---------|---------|---------|---------|---------|---------|
| 15      | 14      | 13      | 12      | 11      | 10      | 9       | 8       |
| BS      | MCB     | STM     | SA      | STXE    | SOVF    | SNR     | SXC     |
| r-(0)   | r-(0)   | r-(0)   | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) |
| 7       | 6       | 5       | 4       | 3       | 2       | 1       | 0       |
| MSTS    | MSPS    | MARB    | MTXE    | MNR     | MXC     | STR     | SPR     |
| rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) | rw1-(0) |

| Bitfield | Range  | Description                                                                                                                                                                                                                                                                                                                                    |
|----------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BS       | Bit 15 | I2C bus state indicator. This bit cannot be cleared by writing to the status register.<br>0 The I2C bus is idle<br>1 The I2C bus is active                                                                                                                                                                                                     |
| MCB      | Bit 14 | I2C master controls bus indicator. This bit cannot be cleared by writing to the status register.<br>0 This master does not control the bus<br>1 This master controls the bus                                                                                                                                                                   |
| STM      | Bit 13 | I2C slave transmitter mode indicator. Indicates for which mode this slave has been addressed. Only valid if I2CSA is 1. This bit cannot be cleared by writing to the status register.<br>0 Slave receiver mode<br>1 Slave transmitter mode                                                                                                     |
| SA       | Bit 12 | I2C slave mode addressed interrupt flag. Indicates that this slave has been addressed by another master. Write a 1 to this bit to clear it.<br>0 No pending interrupt<br>1 Pending interrupt                                                                                                                                                   |
| STXE     | Bit 11 | I2C slave transmit register empty interrupt flag. This bit is set when this slave latches the data stored in the masslaveter transmit register to indicate that the slave transmit register is ready to accept another byte and queue it for transmission. Write a 1 to this bit to clear it.<br>0 No pending interrupt<br>1 Pending interrupt |
| SOVF     | Bit 10 | I2C slave receive register overflow interrupt flag. Indicates that this slave has failed to read one or more bytes from the I2CxSRX register before they were overwritten by another transmission. Write a 1 to this bit to clear it.<br>0 No pending interrupt<br>1 Pending interrupt                                                         |
| SNR      | Bit 9  | I2C slave mode NACK received from master interrupt flag. This bit is set in slave transmitter mode if the master responds with a NACK after this slave sends it a byte of data. This bit is not set if the master responds with an ACK. Write a 1 to this bit to clear it.<br>0 No pending interrupt<br>1 Pending interrupt                    |
| SXC      | Bit 8  | I2C slave mode transfer complete interrupt flag. This bit is set after this slave receives a byte of data from a master, but before this slave sends an ACK/NACK. Write a 1 to this bit to clear it.                                                                                                                                           |

|      |       |                                                                                                                                                                                                                                                                                                                                      |
|------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      |       | 0 No pending interrupt                                                                                                                                                                                                                                                                                                               |
|      |       | 1 Pending interrupt                                                                                                                                                                                                                                                                                                                  |
| MSTS | Bit 7 | I2C master mode start condition sent interrupt flag. This flag is set after this master sends a start condition or repeated start condition. Write a 1 to this bit to clear it.                                                                                                                                                      |
|      |       | 0 No pending interrupt                                                                                                                                                                                                                                                                                                               |
|      |       | 1 Pending interrupt                                                                                                                                                                                                                                                                                                                  |
| MSPS | Bit 6 | I2C master mode stop condition sent interrupt flag. This flag is set after this master sends a stop condition. Write a 1 to this bit to clear it.                                                                                                                                                                                    |
|      |       | 0 No pending interrupt                                                                                                                                                                                                                                                                                                               |
|      |       | 1 Pending interrupt                                                                                                                                                                                                                                                                                                                  |
| MARB | Bit 5 | I2C master mode arbitration loss interrupt flag. This bit is set when this master detects that the value it tried to write to SDA is being overridden by another master. After it detects the arbitration loss, this master releases control of the bus. Write a 1 to this bit to clear it.                                          |
|      |       | 0 No pending interrupt                                                                                                                                                                                                                                                                                                               |
|      |       | 1 Pending interrupt                                                                                                                                                                                                                                                                                                                  |
| MTXE | Bit 4 | I2C master transmit register empty interrupt flag. This bit is set when this master latches the data stored in the master transmit register to indicate that the master transmit register is ready to accept another byte and queue it for transmission. Write a 1 to this bit to clear it.                                          |
|      |       | 0 No pending interrupt                                                                                                                                                                                                                                                                                                               |
|      |       | 1 Pending interrupt                                                                                                                                                                                                                                                                                                                  |
| MNR  | Bit 3 | I2C master mode NACK received interrupt flag. This flag is set in master transmitter mode after ACK/NACK bit is sent if the slave sends a NACK. It is not set if the slave sends an ACK. Write a 1 to this bit to clear it.                                                                                                          |
|      |       | 0 No pending interrupt                                                                                                                                                                                                                                                                                                               |
|      |       | 1 Pending interrupt                                                                                                                                                                                                                                                                                                                  |
| MXC  | Bit 2 | I2C master transfer complete interrupt flag. In master transmitter mode, this flag is set after this master has sent the data byte and the slave has sent an ACK/NACK. In master receiver mode, this flag is set after the slave has sent the data byte and before this master sends an ACK/NACK. Write a 1 to this bit to clear it. |
|      |       | 0 No pending interrupt                                                                                                                                                                                                                                                                                                               |
|      |       | 1 Pending interrupt                                                                                                                                                                                                                                                                                                                  |
| STR  | Bit 1 | I2C start condition received interrupt flag. This flag is set whenever a start condition or repeated start condition is detected on the bus, regardless of if this master or another sent it. Write a 1 to this bit to clear it.                                                                                                     |
|      |       | 0 No pending interrupt                                                                                                                                                                                                                                                                                                               |
|      |       | 1 Pending interrupt                                                                                                                                                                                                                                                                                                                  |
| SPR  | Bit 0 | I2C stop condition received interrupt flag. This flag is set whenever a stop condition condition is detected on the bus, regardless of which device sent it. Write a 1 to this bit to clear it.                                                                                                                                      |
|      |       | 0 No pending interrupt                                                                                                                                                                                                                                                                                                               |
|      |       | 1 Pending interrupt                                                                                                                                                                                                                                                                                                                  |

### 18.1.4 I2CMTX Register

*Register memory slot: 3, offset: 12 bytes, size: 1 byte*

I2C master transmit register. Write the desired slave address and read/write bit to this register after sending a start condition to begin a transmission with a slave. Note that the desired slave address must occupy the upper seven bits and the read/write bit must occupy the least significant bit. If the read bit is 0, the master enters master transmitter mode. If the read bit is 1, the master enters master receiver mode. Write a byte of data to this register after sending the address frame or a data frame to send that byte of data to the slave. If this master is busy with a transmission when this register is written, it will send the byte after it finishes the transmission.



### 18.1.5 I2CMRX Register

*Register memory slot: 4, offset: 16 bytes, size: 1 byte*

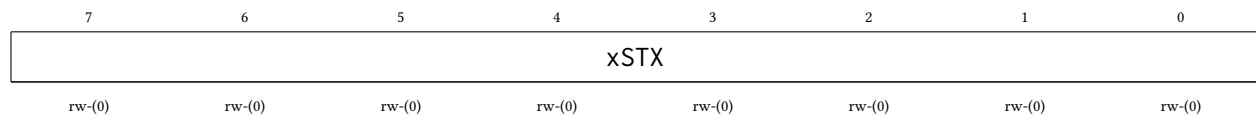
I2C master receive register. When in master receiver mode, read this register after the master transfer complete interrupt flag (I2CMXC) is set to get the received data byte.



### 18.1.6 I2CSTX Register

*Register memory slot: 5, offset: 20 bytes, size: 1 byte*

I2C slave transmit register. When in slave transmitter mode, write to this register after the slave addressed flag (I2CSA) or the slave transaction complete flag (I2CSXC) has been set to queue the next byte to send to the master.



### 18.1.7 I2CSRX Register

*Register memory slot: 6, offset: 24 bytes, size: 1 byte*

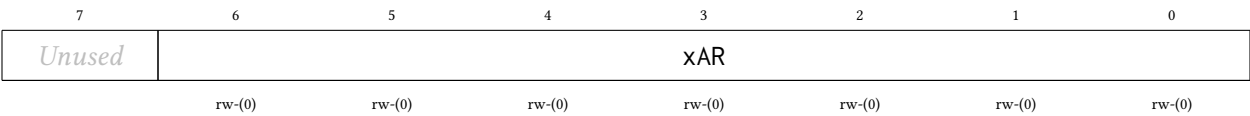
I2C slave receive register. When in slave receiver mode, read this register after the slave transaction complete flag (I2CSXC) has been set to get the data byte sent from the master. Note that if this slave fails to clear the status register before another byte is received, the slave receive overflow flag (I2CSOVF) will be set.



18.1.8 I2CAR Register

Register memory slot: 7, offset: 28 bytes, size: 1 byte

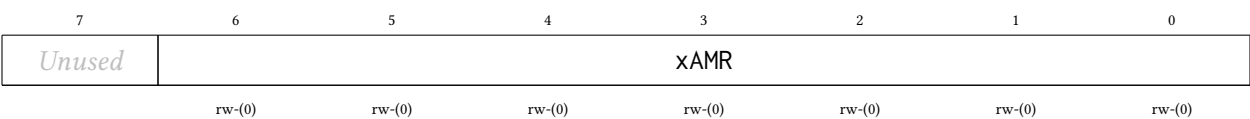
I2C this slave address register. When in slave mode, any master that sends an address frame containing this address will activate this slave.



18.1.9 I2CAMR Register

Register memory slot: 8, offset: 32 bytes, size: 1 byte

I2C this slave address mask register. Any bit set to 1 in this register indicates that the corresponding bit in the slave address register is a wildcard. Only the slave address register bits that correspond to 0s in this register will be compared to the received slave address.



## 19 CLINT Peripheral

The Core-Local Interruptor (CLINT) provides the two interrupt sources that make multi-core software possible: per-hart *software interrupts* (the inter-processor interrupt, or IPI, mechanism) and per-hart *timer interrupts* driven by a shared free-running 64-bit counter. It lives in the shared window behind the multi-core arbiter, so **any hart can raise, clear, or program any hart's interrupts**. The main features are listed below:

- One software interrupt (IPI) register per hart (MSIP0–MSIP3), writable by any hart
- Shared free-running 64-bit MTIME counter, incrementing once per MCLK cycle
- One 64-bit compare register per hart (MTIMECMP0–MTIMECMP3); hart  $h$ 's timer interrupt is asserted while  $\text{MTIME} \geq \text{MTIMECMP}_h$
- Level-sensitive interrupt delivery into interrupt vectors 83 (software) and 84 (timer)
- Wakes harts sleeping via the custom sleep instruction, enabling zero-power hart parking
- Compare registers reset to all-ones, so no timer interrupt fires until one is programmed

The CLINT runs on the free-running MCLK — the same clock as each hart's interrupt handler — so a software interrupt can wake a hart whose gated CPU clock is currently off.

### 19.1 Software Interrupts (IPIs)

Writing 1 to MSIP $_h$  raises the software-interrupt level into hart  $h$  (interrupt vector 83); writing 0 clears it. Because the interrupt is **level-sensitive**, hart  $h$ 's interrupt service routine must write 0 to its own MSIP $_h$  register before returning, or the interrupt immediately re-triggers.

The canonical use is waking a parked hart: the parked hart enables interrupt vector 83 and executes the custom sleep instruction (see Section 15.3); another hart writes 1 to the sleeper's MSIP register. The sleeper's interrupt service routine runs, clears its MSIP, and must execute the custom wake instruction before returning — otherwise the hart goes back to sleep when the service routine returns.

### 19.2 Timer Interrupts

MTIME is a single 64-bit counter shared by all four harts. Each hart has its own 64-bit compare value; the timer interrupt level for hart  $h$  (interrupt vector 84) is asserted while  $\text{MTIME} \geq \text{MTIMECMP}_h$  and cleared by moving MTIMECMP $_h$  into the future (or by wrapping it back to all-ones to disarm it).

Because the registers are 32 bits wide, 64-bit values are accessed as low/high pairs. To read a coherent 64-bit time: read MTIMEH, then MTIMEL, then MTIMEH again, and retry if the two MTIMEH reads differ. To program a compare value without a spurious interrupt: write all-ones to MTIMECMPxH, then the new low half to MTIMECMPxL, then the real high half to MTIMECMPxH.

### 19.3 Addressing Note

The CLINT lives in shared-window page 1 at base address 0x5000. The register layout scales with the hart count: the software-interrupt registers are word-mapped from the base (MSIP $_h$  at 0x5000 + 4 $h$ ), the shared MTIME low/high pair begins at 0x5010, and hart  $h$ 's MTIMECMP $_h$  low/high pair is at 0x5020 + 8 $h$ . The block decodes only the low bits of the address, so its registers alias every 64 bytes throughout 0x5000–0x5FFF. Always use the canonical addresses.

## 19.4 Register Description

### 19.4.1 MSIP0 Register

Register memory slot: 0, offset: 0 bytes, size: 4 bytes

Hart 0 software interrupt (IPI) register. Any hart may write it through the shared window: writing 1 raises the software interrupt level into hart 0 (interrupt vector 83); writing 0 clears it. The interrupt is level-sensitive, so hart 0's service routine must write 0 here before returning.



| Bitfield      | Range     | Description                                |
|---------------|-----------|--------------------------------------------|
| <i>Unused</i> | Bits 31-1 |                                            |
| MSIPH0        | Bit 0     | Software interrupt (IPI) level for hart 0. |
|               |           | 0 No software interrupt pending            |
|               |           | 1 Software interrupt raised                |

### 19.4.2 MSIP1 Register

Register memory slot: 1, offset: 4 bytes, size: 4 bytes

Hart 1 software interrupt (IPI) register. Any hart may write it through the shared window: writing 1 raises the software interrupt level into hart 1 (interrupt vector 83); writing 0 clears it. The interrupt is level-sensitive, so hart 1's service routine must write 0 here before returning.



| Bitfield      | Range     | Description                                                                                                  |
|---------------|-----------|--------------------------------------------------------------------------------------------------------------|
| <i>Unused</i> | Bits 31-1 |                                                                                                              |
| MSIPH1        | Bit 0     | Software interrupt (IPI) level for hart 1.<br>0 No software interrupt pending<br>1 Software interrupt raised |

19.4.3 MSIP2 Register

Register memory slot: 2, offset: 8 bytes, size: 4 bytes

Hart 2 software interrupt (IPI) register. Any hart may write it through the shared window: writing 1 raises the software interrupt level into hart 2 (interrupt vector 83); writing 0 clears it. The interrupt is level-sensitive, so hart 2’s service routine must write 0 here before returning.





| Bitfield      | Range     | Description                                                                                                  |
|---------------|-----------|--------------------------------------------------------------------------------------------------------------|
| <i>Unused</i> | Bits 31-1 |                                                                                                              |
| MSIPH2        | Bit 0     | Software interrupt (IPI) level for hart 2.<br>0 No software interrupt pending<br>1 Software interrupt raised |

#### 19.4.4 MSIP3 Register

Register memory slot: 3, offset: 12 bytes, size: 4 bytes

Hart 3 software interrupt (IPI) register. Any hart may write it through the shared window: writing 1 raises the software interrupt level into hart 3 (interrupt vector 83); writing 0 clears it. The interrupt is level-sensitive, so hart 3's service routine must write 0 here before returning.



| Bitfield      | Range     | Description                                                                                                  |
|---------------|-----------|--------------------------------------------------------------------------------------------------------------|
| <i>Unused</i> | Bits 31-1 |                                                                                                              |
| MSIPH3        | Bit 0     | Software interrupt (IPI) level for hart 3.<br>0 No software interrupt pending<br>1 Software interrupt raised |

#### 19.4.5 MTIMEL Register

Register memory slot: 4, offset: 16 bytes, size: 4 bytes

Machine time counter, lower 32 bits. mtime is a free-running 64-bit counter shared by all four harts that increments once per MCLK cycle. It is writable for initialization; a write merges only the addressed 32-bit half.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| MTIMEL |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| MTIMEL |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| MTIMEL |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| MTIMEL |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description      |
|----------|-----------|------------------|
| MTIMEL   | Bits 31-0 | mtime bits 31:0. |

#### 19.4.6 MTIMEH Register

*Register memory slot: 5, offset: 20 bytes, size: 4 bytes*

Machine time counter, upper 32 bits. Read MTIMEH, then MTIMEL, then MTIMEH again (retry if the two MTIMEH reads differ) to obtain a coherent 64-bit time value.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| MTIMEH |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| MTIMEH |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| MTIMEH |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| MTIMEH |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description       |
|----------|-----------|-------------------|
| MTIMEH   | Bits 31-0 | mtime bits 63:32. |

#### 19.4.7 MTIMECMP0L Register

*Register memory slot: 8, offset: 32 bytes, size: 4 bytes*

Hart 0 timer compare register, lower 32 bits. The timer interrupt level for hart 0 (interrupt vector 84) is raised while  $mtime \geq mtimecmp0$ . Resets to all-ones (no interrupt). To set a new compare value without a spurious interrupt, first write all-ones to MTIMECMP0H, then the new lower half, then the real upper half.

|            |        |        |        |        |        |        |        |
|------------|--------|--------|--------|--------|--------|--------|--------|
| 31         | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| MTIMECMP0L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23         | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| MTIMECMP0L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15         | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| MTIMECMP0L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7          | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| MTIMECMP0L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield   | Range     | Description          |
|------------|-----------|----------------------|
| MTIMECMP0L | Bits 31-0 | mtimecmp0 bits 31:0. |

#### 19.4.8 MTIMECMP0H Register

Register memory slot: 9, offset: 36 bytes, size: 4 bytes

Hart 0 timer compare register, upper 32 bits.

|            |        |        |        |        |        |        |        |
|------------|--------|--------|--------|--------|--------|--------|--------|
| 31         | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| MTIMECMP0H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23         | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| MTIMECMP0H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15         | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| MTIMECMP0H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7          | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| MTIMECMP0H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range | Description |
|----------|-------|-------------|
|----------|-------|-------------|

---

|            |           |                       |
|------------|-----------|-----------------------|
| MTIMECMP0H | Bits 31-0 | mtimecmp0 bits 63:32. |
|------------|-----------|-----------------------|

---

### 19.4.9 MTIMECMP1L Register

*Register memory slot: 10, offset: 40 bytes, size: 4 bytes*

Hart 1 timer compare register, lower 32 bits. The timer interrupt level for hart 1 (interrupt vector 84) is raised while mtime >= mtimecmp1. Resets to all-ones (no interrupt). To set a new compare value without a spurious interrupt, first write all-ones to MTIMECMP1H, then the new lower half, then the real upper half.

|            |        |        |        |        |        |        |        |
|------------|--------|--------|--------|--------|--------|--------|--------|
| 31         | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| MTIMECMP1L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23         | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| MTIMECMP1L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15         | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| MTIMECMP1L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7          | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| MTIMECMP1L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

---

| Bitfield   | Range     | Description          |
|------------|-----------|----------------------|
| MTIMECMP1L | Bits 31-0 | mtimecmp1 bits 31:0. |

---

### 19.4.10 MTIMECMP1H Register

*Register memory slot: 11, offset: 44 bytes, size: 4 bytes*

Hart 1 timer compare register, upper 32 bits.

|            |        |        |        |        |        |        |        |
|------------|--------|--------|--------|--------|--------|--------|--------|
| 31         | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| MTIMECMP1H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23         | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| MTIMECMP1H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15         | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| MTIMECMP1H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7          | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| MTIMECMP1H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield   | Range     | Description           |
|------------|-----------|-----------------------|
| MTIMECMP1H | Bits 31-0 | mtimecmp1 bits 63:32. |

#### 19.4.11 MTIMECMP2L Register

*Register memory slot: 12, offset: 48 bytes, size: 4 bytes*

Hart 2 timer compare register, lower 32 bits. The timer interrupt level for hart 2 (interrupt vector 84) is raised while mtime >= mtimecmp2. Resets to all-ones (no interrupt). To set a new compare value without a spurious interrupt, first write all-ones to MTIMECMP2H, then the new lower half, then the real upper half.

|            |        |        |        |        |        |        |        |
|------------|--------|--------|--------|--------|--------|--------|--------|
| 31         | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| MTIMECMP2L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23         | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| MTIMECMP2L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15         | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| MTIMECMP2L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7          | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| MTIMECMP2L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield   | Range     | Description          |
|------------|-----------|----------------------|
| MTIMECMP2L | Bits 31-0 | mtimecmp2 bits 31:0. |

### 19.4.12 MTIMECMP2H Register

Register memory slot: 13, offset: 52 bytes, size: 4 bytes

Hart 2 timer compare register, upper 32 bits.

|            |        |        |        |        |        |        |        |
|------------|--------|--------|--------|--------|--------|--------|--------|
| 31         | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| MTIMECMP2H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23         | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| MTIMECMP2H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15         | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| MTIMECMP2H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7          | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| MTIMECMP2H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield   | Range     | Description           |
|------------|-----------|-----------------------|
| MTIMECMP2H | Bits 31-0 | mtimecmp2 bits 63:32. |

### 19.4.13 MTIMECMP3L Register

Register memory slot: 14, offset: 56 bytes, size: 4 bytes

Hart 3 timer compare register, lower 32 bits. The timer interrupt level for hart 3 (interrupt vector 84) is raised while mtime >= mtimecmp3. Resets to all-ones (no interrupt). To set a new compare value without a spurious interrupt, first write all-ones to MTIMECMP3H, then the new lower half, then the real upper half.

|            |        |        |        |        |        |        |        |
|------------|--------|--------|--------|--------|--------|--------|--------|
| 31         | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| MTIMECMP3L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23         | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| MTIMECMP3L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15         | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| MTIMECMP3L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7          | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| MTIMECMP3L |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield   | Range     | Description          |
|------------|-----------|----------------------|
| MTIMECMP3L | Bits 31-0 | mtimecmp3 bits 31:0. |

#### 19.4.14 MTIMECMP3H Register

Register memory slot: 15, offset: 60 bytes, size: 4 bytes

Hart 3 timer compare register, upper 32 bits.

|            |        |        |        |        |        |        |        |
|------------|--------|--------|--------|--------|--------|--------|--------|
| 31         | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| MTIMECMP3H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23         | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| MTIMECMP3H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15         | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| MTIMECMP3H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7          | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| MTIMECMP3H |        |        |        |        |        |        |        |
| rw-(0)     | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield   | Range     | Description           |
|------------|-----------|-----------------------|
| MTIMECMP3H | Bits 31-0 | mtimecmp3 bits 63:32. |

## 20 MUTEX Peripheral

The hardware mutex bank provides sixteen word-mapped advisory locks with **single-instruction** cross-hart mutual exclusion — no LR/SC retry loop and no reservation state. Atomicity comes for free from the multi-core arbiter, which serializes whole shared-window transactions. The main features are listed below:

- 16 independent mutexes, one 32-bit word each
- Claim with one load: reading a free mutex returns 0 *and claims it* in the same transaction
- Reading a held mutex returns the owner's marker (hartid + 1) without disturbing it
- Release with one store of 0
- Force-release supported: any hart may write 0, so a supervisor can recover a dead hart's mutex
- Ownership cannot be forged: nonzero writes are ignored
- All mutexes reset to free

### 20.1 Usage

To claim mutex  $i$ , read it once and test the result:

```
lw      t0, (MUTEXi)      # t0 == 0 -> you now hold the mutex
bnez    t0, retry         # t0 == h+1 -> held by hart h, back off and retry
```

To release a mutex you hold:

```
sw      x0, (MUTEXi)      # owner := free
```

Re-reading a mutex you already hold returns your own marker (mhartid + 1) and does not disturb the lock, so an accidental double claim-read is harmless.

When a claim fails, spin with a hart-scaled backoff and a bounded retry count (see the Multi-Core Architecture chapter): four harts re-reading a contested mutex in lockstep waste arbiter bandwidth and can starve useful work.

### 20.2 Rules

- These are **advisory** locks: the hardware does not block a hart that ignores the protocol and accesses the protected resource directly. Correct software claims the mutex before touching the resource it guards.
- **Never** access a mutex address with LR/SC or AMO instructions — only plain lw/sw. A suppressed store-conditional arrives at the bank indistinguishable from a claim-read and will corrupt the ownership protocol.
- Release (write 0) is deliberately *not* qualified by owner so that a supervisory hart can force-release the mutex of a hung or dead hart. Correct software releases only mutexes it holds.

The bank occupies shared-window page 2 at base address 0x6000; the sixteen mutexes are word-mapped, with mutex  $i$  at  $0x6000 + 4i$ . The bank decodes only the low bits of the address, so its words alias throughout the page. Always use the canonical addresses.



## 20.3 Register Description

### 20.3.1 MUTEX0 Register

*Register memory slot: 0, offset: 0 bytes, size: 4 bytes*

Hardware mutex 0. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| OWN0   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| OWN0   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| OWN0   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| OWN0   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN0     | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN0_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN0_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN0_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN0_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN0_H3: Held by hart 3 |

### 20.3.2 MUTEX1 Register

*Register memory slot: 1, offset: 4 bytes, size: 4 bytes*

Hardware mutex 1. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| OWN1   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| OWN1   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| OWN1   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| OWN1   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN1     | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN1_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN1_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN1_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN1_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN1_H3: Held by hart 3 |

### 20.3.3 MUTEX2 Register

Register memory slot: 2, offset: 8 bytes, size: 4 bytes

Hardware mutex 2. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| OWN2   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| OWN2   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| OWN2   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| OWN2   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN2     | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN2_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN2_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN2_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN2_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN2_H3: Held by hart 3 |

### 20.3.4 MUTEX3 Register

Register memory slot: 3, offset: 12 bytes, size: 4 bytes

Hardware mutex 3. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.

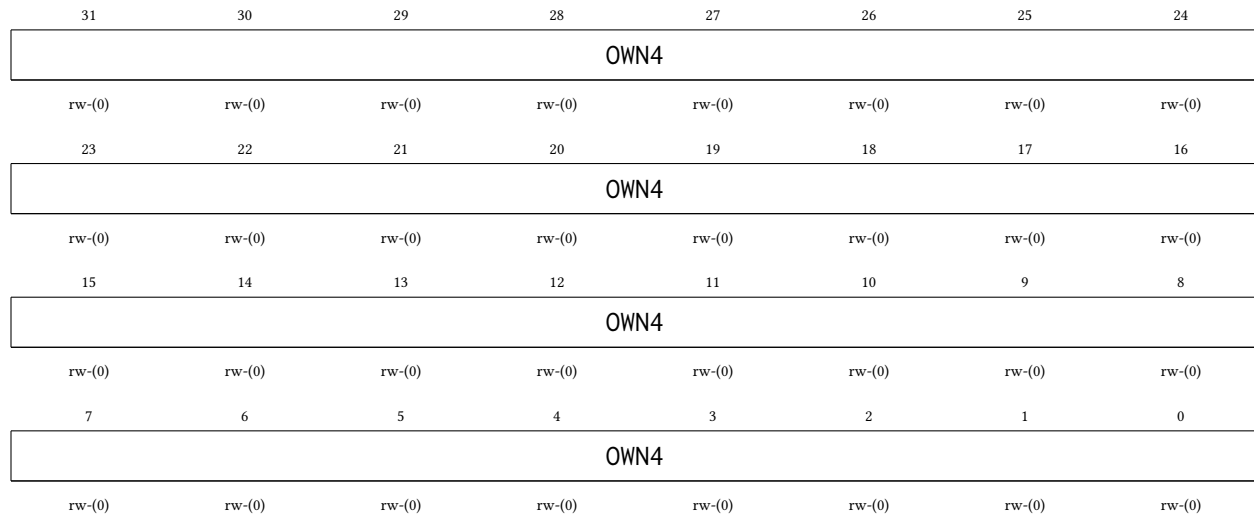
|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| OWN3   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| OWN3   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| OWN3   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| OWN3   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN3     | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN3_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN3_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN3_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN3_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN3_H3: Held by hart 3 |

### 20.3.5 MUTEX4 Register

Register memory slot: 4, offset: 16 bytes, size: 4 bytes

Hardware mutex 4. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN4     | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN4_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN4_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN4_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN4_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN4_H3: Held by hart 3 |

### 20.3.6 MUTEX5 Register

Register memory slot: 5, offset: 20 bytes, size: 4 bytes

Hardware mutex 5. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| OWN5   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| OWN5   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| OWN5   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| OWN5   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN5     | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN5_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN5_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN5_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN5_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN5_H3: Held by hart 3 |

### 20.3.7 MUTEX6 Register

Register memory slot: 6, offset: 24 bytes, size: 4 bytes

Hardware mutex 6. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| OWN6   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| OWN6   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| OWN6   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| OWN6   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN6     | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN6_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN6_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN6_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN6_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN6_H3: Held by hart 3 |

### 20.3.8 MUTEX7 Register

Register memory slot: 7, offset: 28 bytes, size: 4 bytes

Hardware mutex 7. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.

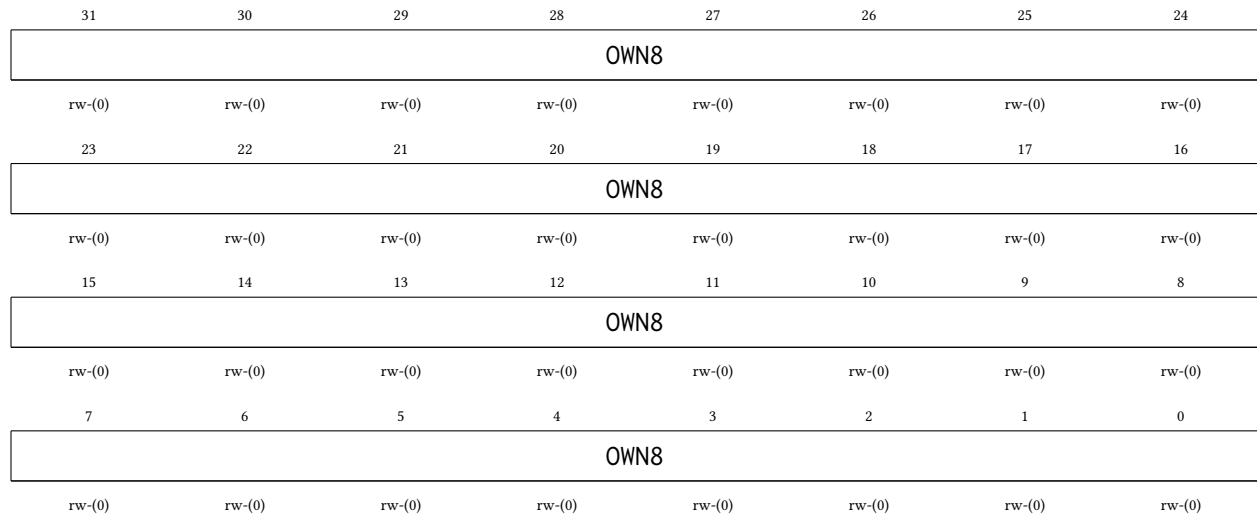
|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| OWN7   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| OWN7   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| OWN7   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| OWN7   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN7     | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN7_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN7_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN7_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN7_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN7_H3: Held by hart 3 |

### 20.3.9 MUTEX8 Register

Register memory slot: 8, offset: 32 bytes, size: 4 bytes

Hardware mutex 8. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN8     | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN8_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN8_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN8_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN8_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN8_H3: Held by hart 3 |

### 20.3.10 MUTEX9 Register

Register memory slot: 9, offset: 36 bytes, size: 4 bytes

Hardware mutex 9. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| OWN9   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| OWN9   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| OWN9   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| OWN9   |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN9     | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN9_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN9_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN9_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN9_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN9_H3: Held by hart 3 |

### 20.3.11 MUTEX10 Register

Register memory slot: 10, offset: 40 bytes, size: 4 bytes

Hardware mutex 10. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| OWN10  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| OWN10  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| OWN10  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| OWN10  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |



| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN10    | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN10_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN10_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN10_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN10_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN10_H3: Held by hart 3 |

### 20.3.12 MUTEX11 Register

Register memory slot: 11, offset: 44 bytes, size: 4 bytes

Hardware mutex 11. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.

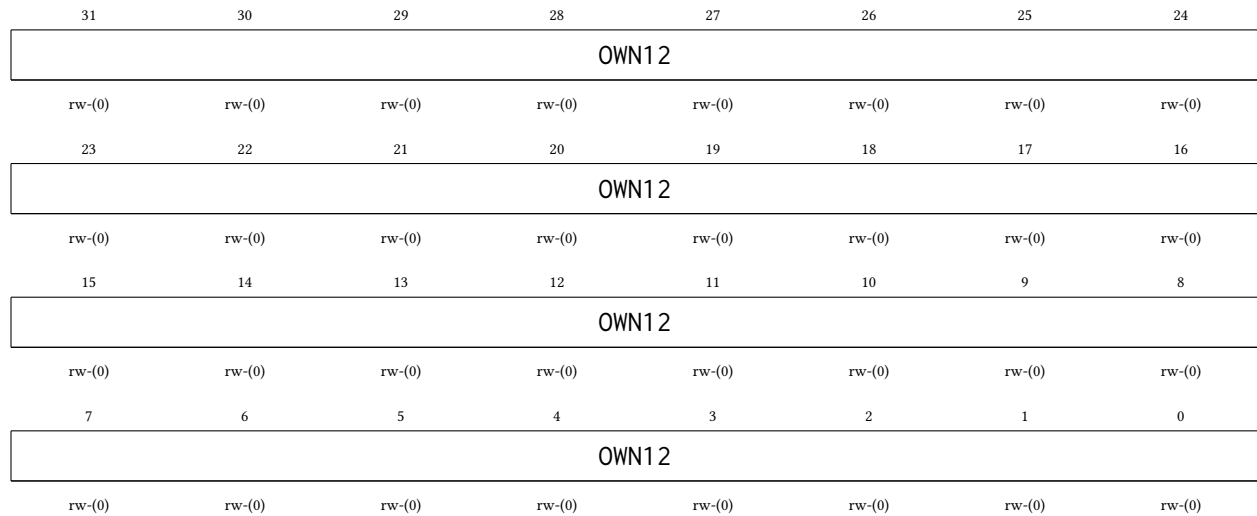
|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| OWN11  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| OWN11  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| OWN11  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| OWN11  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN11    | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN11_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN11_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN11_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN11_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN11_H3: Held by hart 3 |

### 20.3.13 MUTEX12 Register

Register memory slot: 12, offset: 48 bytes, size: 4 bytes

Hardware mutex 12. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.



| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN12    | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN12_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN12_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN12_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN12_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN12_H3: Held by hart 3 |

### 20.3.14 MUTEX13 Register

Register memory slot: 13, offset: 52 bytes, size: 4 bytes

Hardware mutex 13. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| OWN13  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| OWN13  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| OWN13  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| OWN13  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN13    | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN13_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN13_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN13_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN13_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN13_H3: Held by hart 3 |

### 20.3.15 MUTEX14 Register

Register memory slot: 14, offset: 56 bytes, size: 4 bytes

Hardware mutex 14. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| OWN14  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| OWN14  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| OWN14  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| OWN14  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN14    | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN14_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN14_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN14_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN14_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN14_H3: Held by hart 3 |

### 20.3.16 MUTEX15 Register

Register memory slot: 15, offset: 60 bytes, size: 4 bytes

Hardware mutex 15. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| OWN15  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| OWN15  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| OWN15  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| OWN15  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OWN15    | Bits 31-0 | Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h.<br>00000000000000000000000000000000 MTXOWN15_FREE: Mutex free (a read returning this value claims the mutex)<br>00000000000000000000000000000001 MTXOWN15_H0: Held by hart 0<br>00000000000000000000000000000010 MTXOWN15_H1: Held by hart 1<br>00000000000000000000000000000011 MTXOWN15_H2: Held by hart 2<br>00000000000000000000000000000100 MTXOWN15_H3: Held by hart 3 |

## 21 IRQROUTER Peripheral

The interrupt router is the chip's peripheral interrupt controller. Every deglitched peripheral interrupt level terminates here, and delivery to the harts follows the claim/complete model of a RISC-V platform-level interrupt controller (PLIC): the router raises a single external-interrupt wire (meip, interrupt vector 85) to each hart, and the servicing hart reads the CLAIM register to discover — and atomically claim — the pending source. Routing is per-hart and per-vector: software can steer any peripheral's interrupt to whichever hart currently owns that peripheral — for example, a hart that has taken over TIMER1 through the shared peripheral page can also receive TIMER1's interrupts. The main features are listed below:

- One row of three 32-bit routing/enable registers per hart, covering interrupt vectors 0–84
- Claim/complete delivery: one meip wire per hart (vector 85), CLAIM read = atomic claim of the lowest pending enabled vector, CLAIM write = complete
- Exactly-once delivery: a claimed vector is masked from every hart's meip until completed, so a vector routed to several harts is serviced by exactly one of them
- Programmable by *any* hart through the shared window; claims are attributed to the reading hart by the shared-bus arbiter
- Fixed priority: the lowest pending vector number wins
- Read-only PENDx (raw levels) and INSVCx (under-service) status words for debugging and recovery
- Resets fully masked (all zeros): the router is inert until software programs it
- The CLINT vectors (83 and 84) are never delivered through meip — each hart receives its own msip/mtip on dedicated hardwired wires — so their row bits are writable but have no effect
- Runs on the free-running MCLK, so a routed interrupt can wake a hart whose gated CPU clock is off

### 21.1 Delivery Model

Hart  $h$ 's meip wire is asserted while some vector  $v$  is pending (deglitched level high), enabled in hart  $h$ 's row (bit  $v$ ), and not under service. The hart takes vector 85 through its interrupt vector table like any other interrupt; its handler then:

1. Reads CLAIM. The router returns the lowest such  $v$  for the reading hart and marks it under service, hiding it from every hart's meip. A return value of 0xFFFFFFFF means nothing was pending for this hart (for example, another hart claimed the source first) — the handler treats the interrupt as spurious and simply returns.
2. Services the source and *clears the interrupt level at the peripheral* (the usual level-interrupt contract).
3. Writes  $v$  back to CLAIM (complete). The vector becomes deliverable again; if its level is still asserted — a new event — it pends again immediately.

Completion is deliberately not qualified by owner: a supervising hart can complete a vector on behalf of a hart that died mid-service (the INSVCx words show which vectors are stuck). Every hart, hart 0 included, uses this same path — the single-core chip's vectored controller in the SYSTEM peripheral is retired, and row 0 is hart 0's live routing row.

Note that the vector packing of the HxENU registers is contiguous (bit  $b$  of HxENU is vector  $64 + b$ , for  $b = 0 \dots 20$ ), unlike the write-packing quirk of the retired single-core IRQENU register.

A typical hand-off of a peripheral to hart 2 looks like:

1. Hart 2 (or any hart) sets the peripheral's vector bits in H2ENL/H2ENM/H2ENU.
2. The previous owner clears the same bits in its own row.
3. Hart 2 starts operating the peripheral through the shared peripheral page; its vector-85 handler dispatches on the claimed vector number.

The routing rows live at the block base (16 bytes per hart); the CLAIM register and the status words sit at fixed offsets +0x800, +0x810 and +0x820, independent of the hart count.

## 21.2 Register Description

### 21.2.1 H0ENL Register

Register memory slot: 0, offset: 0 bytes, size: 4 bytes

Hart 0 interrupt routing register, vectors 31:0. Each bit enables delivery of the corresponding interrupt vector to hart 0 via its meip wire (vector 85) and the CLAIM/COMPLETE mechanism.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| H0ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H0ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H0ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H0ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                               |
|----------|-----------|-----------------------------------------------------------|
| H0ENL    | Bits 31-0 | Interrupt enable bits for vectors 31:0, routed to hart 0. |

### 21.2.2 H0ENM Register

Register memory slot: 1, offset: 4 bytes, size: 4 bytes

Hart 0 interrupt routing register, vectors 63:32.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| H0ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H0ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H0ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H0ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                |
|----------|-----------|------------------------------------------------------------|
| H0ENM    | Bits 31-0 | Interrupt enable bits for vectors 63:32, routed to hart 0. |

### 21.2.3 H0ENU Register

*Register memory slot: 2, offset: 8 bytes, size: 4 bytes*

Hart 0 interrupt routing register, vectors 95:64. Unlike the retired SYSTEM IRQENU register of the single-core chip, the packing here is contiguous in both directions. Bits 19 and 20 correspond to the CLINT vectors 83 and 84, which are delivered on dedicated hardwired wires and never through meip: these two bits are writable but have no effect.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| H0ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H0ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H0ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H0ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                |
|----------|-----------|------------------------------------------------------------|
| H0ENU    | Bits 31-0 | Interrupt enable bits for vectors 95:64, routed to hart 0. |

### 21.2.4 H0ENX Register

*Register memory slot: 3, offset: 12 bytes, size: 4 bytes*

Hart 0 interrupt routing register, vectors 120:96 (bits 24:0; upper bits read as 0).



|               |        |        |        |        |        |        |        |
|---------------|--------|--------|--------|--------|--------|--------|--------|
| 31            | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| <i>Unused</i> |        |        |        |        |        |        | H0ENX  |
| rw-(0)        |        |        |        |        |        |        |        |
| 23            | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H0ENX         |        |        |        |        |        |        |        |
| rw-(0)        | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15            | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H0ENX         |        |        |        |        |        |        |        |
| rw-(0)        | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7             | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H0ENX         |        |        |        |        |        |        |        |
| rw-(0)        | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield      | Range      | Description                                                 |
|---------------|------------|-------------------------------------------------------------|
| <i>Unused</i> | Bits 31-25 |                                                             |
| H0ENX         | Bits 24-0  | Interrupt enable bits for vectors 120:96, routed to hart 0. |

### 21.2.5 H1ENL Register

Register memory slot: 4, offset: 16 bytes, size: 4 bytes

Hart 1 interrupt routing register, vectors 31:0. Each bit enables delivery of the corresponding interrupt vector to hart 1 via its meip wire (vector 85) and the CLAIM/COMPLETE mechanism.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| H1ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H1ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H1ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H1ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                               |
|----------|-----------|-----------------------------------------------------------|
| H1ENL    | Bits 31-0 | Interrupt enable bits for vectors 31:0, routed to hart 1. |

### 21.2.6 H1ENM Register

Register memory slot: 5, offset: 20 bytes, size: 4 bytes

Hart 1 interrupt routing register, vectors 63:32.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| H1ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H1ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H1ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H1ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                |
|----------|-----------|------------------------------------------------------------|
| H1ENM    | Bits 31-0 | Interrupt enable bits for vectors 63:32, routed to hart 1. |

### 21.2.7 H1ENU Register

Register memory slot: 6, offset: 24 bytes, size: 4 bytes

Hart 1 interrupt routing register, vectors 95:64. Unlike the retired SYSTEM IRQENU register of the single-core chip, the packing here is contiguous in both directions. Bits 19 and 20 correspond to the CLINT vectors 83 and 84, which are delivered on dedicated hardwired wires and never through meip: these two bits are writable but have no effect.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| H1ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H1ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H1ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H1ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                |
|----------|-----------|------------------------------------------------------------|
| H1ENU    | Bits 31-0 | Interrupt enable bits for vectors 95:64, routed to hart 1. |

### 21.2.8 H1ENX Register

Register memory slot: 7, offset: 28 bytes, size: 4 bytes

Hart 1 interrupt routing register, vectors 120:96 (bits 24:0; upper bits read as 0).

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| Unused |        |        |        |        |        |        | H1ENX  |
| rw-(0) |        |        |        |        |        |        |        |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H1ENX  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H1ENX  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H1ENX  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range      | Description                                                 |
|----------|------------|-------------------------------------------------------------|
| Unused   | Bits 31-25 |                                                             |
| H1ENX    | Bits 24-0  | Interrupt enable bits for vectors 120:96, routed to hart 1. |

### 21.2.9 H2ENL Register

Register memory slot: 8, offset: 32 bytes, size: 4 bytes

Hart 2 interrupt routing register, vectors 31:0. Each bit enables delivery of the corresponding interrupt vector to hart 2 via its meip wire (vector 85) and the CLAIM/COMPLETE mechanism.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| H2ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H2ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H2ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H2ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                               |
|----------|-----------|-----------------------------------------------------------|
| H2ENL    | Bits 31:0 | Interrupt enable bits for vectors 31:0, routed to hart 2. |

### 21.2.10 H2ENM Register

Register memory slot: 9, offset: 36 bytes, size: 4 bytes

Hart 2 interrupt routing register, vectors 63:32.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| H2ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H2ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H2ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H2ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                |
|----------|-----------|------------------------------------------------------------|
| H2ENM    | Bits 31:0 | Interrupt enable bits for vectors 63:32, routed to hart 2. |

### 21.2.11 H2ENU Register

Register memory slot: 10, offset: 40 bytes, size: 4 bytes

Hart 2 interrupt routing register, vectors 95:64. Unlike the retired SYSTEM IRQENU register of the single-core chip, the packing here is contiguous in both directions. Bits 19 and 20 correspond to the CLINT vectors 83 and 84, which are delivered on dedicated hardwired wires and never through meip: these two bits are writable but have no effect.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| H2ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H2ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H2ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H2ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                |
|----------|-----------|------------------------------------------------------------|
| H2ENU    | Bits 31-0 | Interrupt enable bits for vectors 95:64, routed to hart 2. |

### 21.2.12 H2ENX Register

Register memory slot: 11, offset: 44 bytes, size: 4 bytes

Hart 2 interrupt routing register, vectors 120:96 (bits 24:0; upper bits read as 0).

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| Unused |        |        |        |        |        |        | H2ENX  |
|        |        |        |        |        |        |        | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H2ENX  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H2ENX  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H2ENX  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range | Description |
|----------|-------|-------------|
|----------|-------|-------------|

|               |                                                                       |
|---------------|-----------------------------------------------------------------------|
| <i>Unused</i> | Bits 31-25                                                            |
| H2ENX         | Bits 24-0 Interrupt enable bits for vectors 120:96, routed to hart 2. |

### 21.2.13 H3ENL Register

Register memory slot: 12, offset: 48 bytes, size: 4 bytes

Hart 3 interrupt routing register, vectors 31:0. Each bit enables delivery of the corresponding interrupt vector to hart 3 via its meip wire (vector 85) and the CLAIM/COMPLETE mechanism.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| H3ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H3ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H3ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H3ENL  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                               |
|----------|-----------|-----------------------------------------------------------|
| H3ENL    | Bits 31-0 | Interrupt enable bits for vectors 31:0, routed to hart 3. |

### 21.2.14 H3ENM Register

Register memory slot: 13, offset: 52 bytes, size: 4 bytes

Hart 3 interrupt routing register, vectors 63:32.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| H3ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H3ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H3ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H3ENM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                |
|----------|-----------|------------------------------------------------------------|
| H3ENM    | Bits 31-0 | Interrupt enable bits for vectors 63:32, routed to hart 3. |

### 21.2.15 H3ENU Register

*Register memory slot: 14, offset: 56 bytes, size: 4 bytes*

Hart 3 interrupt routing register, vectors 95:64. Unlike the retired SYSTEM IRQENU register of the single-core chip, the packing here is contiguous in both directions. Bits 19 and 20 correspond to the CLINT vectors 83 and 84, which are delivered on dedicated hardwired wires and never through meip: these two bits are writable but have no effect.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| H3ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H3ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H3ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H3ENU  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield | Range     | Description                                                |
|----------|-----------|------------------------------------------------------------|
| H3ENU    | Bits 31-0 | Interrupt enable bits for vectors 95:64, routed to hart 3. |

### 21.2.16 H3ENX Register

*Register memory slot: 15, offset: 60 bytes, size: 4 bytes*

Hart 3 interrupt routing register, vectors 120:96 (bits 24:0; upper bits read as 0).

|               |        |        |        |        |        |        |        |
|---------------|--------|--------|--------|--------|--------|--------|--------|
| 31            | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| <i>Unused</i> |        |        |        |        |        |        | H3ENX  |
| rw-(0)        |        |        |        |        |        |        |        |
| 23            | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| H3ENX         |        |        |        |        |        |        |        |
| rw-(0)        | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15            | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| H3ENX         |        |        |        |        |        |        |        |
| rw-(0)        | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7             | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| H3ENX         |        |        |        |        |        |        |        |
| rw-(0)        | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |

| Bitfield      | Range      | Description                                                 |
|---------------|------------|-------------------------------------------------------------|
| <i>Unused</i> | Bits 31-25 |                                                             |
| H3ENX         | Bits 24-0  | Interrupt enable bits for vectors 120:96, routed to hart 3. |

### 21.2.17 CLAIM Register

*Register memory slot: 512, offset: 2048 bytes, size: 4 bytes*

Interrupt claim/complete register (M19). READ = claim: atomically returns the lowest pending interrupt vector that is enabled in the READING hart's routing row and not already under service, and marks it under service (masking it from every hart's meip). Returns 0xFFFFFFFF if nothing is pending for the reader; a handler seeing that value treats the interrupt as spurious and simply returns. WRITE = complete: write the claimed vector number to end its service; the vector becomes deliverable again (and pends immediately if its level is still asserted, so clear the level at the peripheral BEFORE completing). Completion is deliberately not qualified by owner, so a supervisor hart can complete on behalf of a hung hart (recovery); written values that are not valid vector numbers, including a stored 0xFFFFFFFF, are ignored.

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| 31     | 30     | 29     | 28     | 27     | 26     | 25     | 24     |
| CLAIM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 23     | 22     | 21     | 20     | 19     | 18     | 17     | 16     |
| CLAIM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 15     | 14     | 13     | 12     | 11     | 10     | 9      | 8      |
| CLAIM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |
| 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |
| CLAIM  |        |        |        |        |        |        |        |
| rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) | rw-(0) |



| Bitfield | Range     | Description                                                                                              |
|----------|-----------|----------------------------------------------------------------------------------------------------------|
| CLAIM    | Bits 31-0 | Read: claimed vector number (0xFFFFFFFF = none pending for this hart). Write: vector number to complete. |

### 21.2.18 PENDL Register

*Register memory slot: 516, offset: 2064 bytes, size: 4 bytes*

Raw pending interrupt levels, vectors 31:0 (read-only; deglitched peripheral levels before enable/claim masking). Debug and polling aid.

|       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|
| 31    | 30    | 29    | 28    | 27    | 26    | 25    | 24    |
| PENDL |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 23    | 22    | 21    | 20    | 19    | 18    | 17    | 16    |
| PENDL |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 15    | 14    | 13    | 12    | 11    | 10    | 9     | 8     |
| PENDL |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| PENDL |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |

| Bitfield | Range     | Description                                   |
|----------|-----------|-----------------------------------------------|
| PENDL    | Bits 31-0 | Deglitched interrupt levels for vectors 31:0. |

### 21.2.19 PENDM Register

*Register memory slot: 517, offset: 2068 bytes, size: 4 bytes*

Raw pending interrupt levels, vectors 63:32 (read-only).

|       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|
| 31    | 30    | 29    | 28    | 27    | 26    | 25    | 24    |
| PENDM |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 23    | 22    | 21    | 20    | 19    | 18    | 17    | 16    |
| PENDM |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 15    | 14    | 13    | 12    | 11    | 10    | 9     | 8     |
| PENDM |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| PENDM |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |

| Bitfield | Range     | Description                                    |
|----------|-----------|------------------------------------------------|
| PENDM    | Bits 31-0 | Deglitched interrupt levels for vectors 63:32. |

### 21.2.20 PENDU Register

Register memory slot: 518, offset: 2072 bytes, size: 4 bytes

Raw pending interrupt levels, vectors 95:64 (read-only).

|       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|
| 31    | 30    | 29    | 28    | 27    | 26    | 25    | 24    |
| PENDU |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 23    | 22    | 21    | 20    | 19    | 18    | 17    | 16    |
| PENDU |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 15    | 14    | 13    | 12    | 11    | 10    | 9     | 8     |
| PENDU |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| PENDU |       |       |       |       |       |       |       |
| r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |

| Bitfield | Range     | Description                                    |
|----------|-----------|------------------------------------------------|
| PENDU    | Bits 31-0 | Deglitched interrupt levels for vectors 95:64. |

### 21.2.21 PENDX Register

Register memory slot: 519, offset: 2076 bytes, size: 4 bytes

Raw pending interrupt levels, vectors 120:96 (bits 24:0, read-only; upper bits read as 0). Completes the raw-level readback for the fourth enable word (added with the peripheral-library program; before it, sources above 95 were routable and claimable but absent from the debug readback).

|        |       |       |       |       |       |       |       |
|--------|-------|-------|-------|-------|-------|-------|-------|
| 31     | 30    | 29    | 28    | 27    | 26    | 25    | 24    |
| Unused |       |       |       |       |       |       | PENDX |
|        |       |       |       |       |       |       | r-(0) |
| 23     | 22    | 21    | 20    | 19    | 18    | 17    | 16    |
| PENDX  |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 15     | 14    | 13    | 12    | 11    | 10    | 9     | 8     |
| PENDX  |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 7      | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| PENDX  |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |

| Bitfield | Range      | Description                                    |
|----------|------------|------------------------------------------------|
| Unused   | Bits 31-25 |                                                |
| PENDX    | Bits 24-0  | Deglinted interrupt levels for vectors 120:96. |

### 21.2.22 INSVCL Register

Register memory slot: 520, offset: 2080 bytes, size: 4 bytes

Under-service (claimed, not yet completed) flags, vectors 31:0 (read-only). Debug and recovery visibility: a stuck bit here means a hart claimed the vector and never completed it; any hart can recover by writing the vector number to CLAIM.

|        |       |       |       |       |       |       |       |
|--------|-------|-------|-------|-------|-------|-------|-------|
| 31     | 30    | 29    | 28    | 27    | 26    | 25    | 24    |
| INSVCL |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 23     | 22    | 21    | 20    | 19    | 18    | 17    | 16    |
| INSVCL |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 15     | 14    | 13    | 12    | 11    | 10    | 9     | 8     |
| INSVCL |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 7      | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| INSVCL |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |

| Bitfield | Range     | Description                           |
|----------|-----------|---------------------------------------|
| INSVCL   | Bits 31-0 | Under-service flags for vectors 31:0. |

### 21.2.23 INSVCM Register

Register memory slot: 521, offset: 2084 bytes, size: 4 bytes

Under-service flags, vectors 63:32 (read-only).

|        |       |       |       |       |       |       |       |
|--------|-------|-------|-------|-------|-------|-------|-------|
| 31     | 30    | 29    | 28    | 27    | 26    | 25    | 24    |
| INSVCM |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 23     | 22    | 21    | 20    | 19    | 18    | 17    | 16    |
| INSVCM |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 15     | 14    | 13    | 12    | 11    | 10    | 9     | 8     |
| INSVCM |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 7      | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| INSVCM |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |

| Bitfield | Range     | Description                            |
|----------|-----------|----------------------------------------|
| INSVCM   | Bits 31-0 | Under-service flags for vectors 63:32. |

### 21.2.24 INSVCU Register

Register memory slot: 522, offset: 2088 bytes, size: 4 bytes

Under-service flags, vectors 95:64 (read-only).

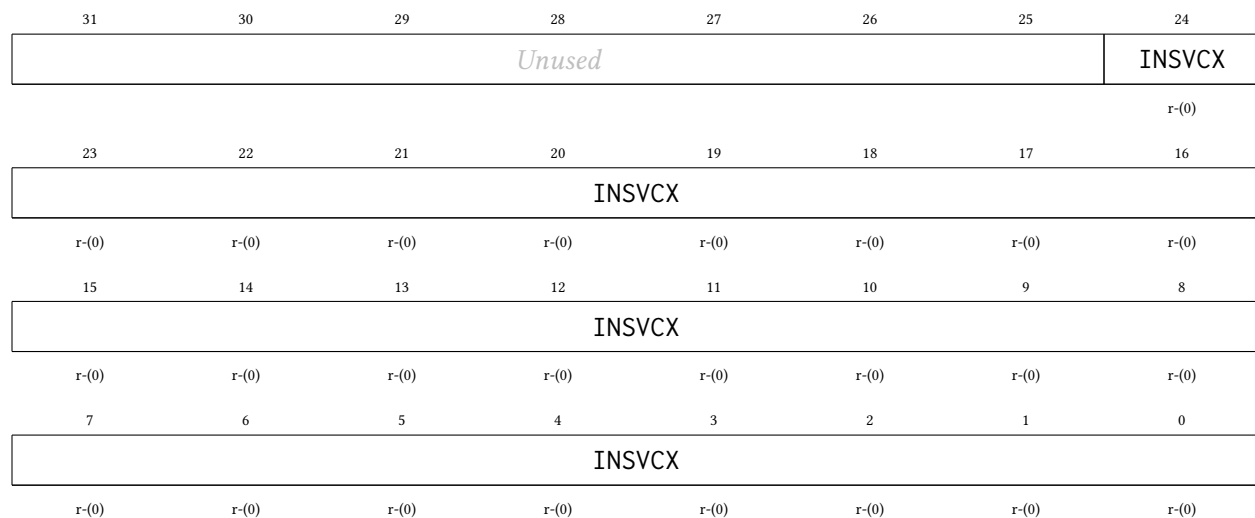
|        |       |       |       |       |       |       |       |
|--------|-------|-------|-------|-------|-------|-------|-------|
| 31     | 30    | 29    | 28    | 27    | 26    | 25    | 24    |
| INSVCU |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 23     | 22    | 21    | 20    | 19    | 18    | 17    | 16    |
| INSVCU |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 15     | 14    | 13    | 12    | 11    | 10    | 9     | 8     |
| INSVCU |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |
| 7      | 6     | 5     | 4     | 3     | 2     | 1     | 0     |
| INSVCU |       |       |       |       |       |       |       |
| r-(0)  | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) | r-(0) |

| Bitfield | Range     | Description                            |
|----------|-----------|----------------------------------------|
| INSVCU   | Bits 31-0 | Under-service flags for vectors 95:64. |

### 21.2.25 INSVCX Register

Register memory slot: 523, offset: 2092 bytes, size: 4 bytes

Under-service flags, vectors 120:96 (bits 24:0, read-only; upper bits read as 0). Completes the under-service readback for the fourth enable word.



| Bitfield | Range      | Description                             |
|----------|------------|-----------------------------------------|
| Unused   | Bits 31-25 |                                         |
| INSVCX   | Bits 24-0  | Under-service flags for vectors 120:96. |

## 22 Register and Peripheral Memory Map

### 22.1 GPIO0 Peripheral

Base address: 0x4000 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| P0IN     | 0x4000  | 0    | 0              | 1            |
| P0OUT    | 0x4004  | 1    | 4              | 1            |
| P0OUTS   | 0x4008  | 2    | 8              | 1            |
| P0OUTC   | 0x400C  | 3    | 12             | 1            |
| P0OUTT   | 0x4010  | 4    | 16             | 4            |
| P0DIR    | 0x4014  | 5    | 20             | 1            |
| P0IF     | 0x4018  | 6    | 24             | 4            |
| P0IES    | 0x401C  | 7    | 28             | 1            |
| P0IE     | 0x4020  | 8    | 32             | 1            |
| P0SEL    | 0x4024  | 9    | 36             | 1            |
| P0REN    | 0x4028  | 10   | 40             | 1            |
| P0AFS    | 0x402C  | 11   | 44             | 4            |

### 22.2 GPIO1 Peripheral

Base address: 0x4100 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| P1IN     | 0x4100  | 0    | 0              | 1            |
| P1OUT    | 0x4104  | 1    | 4              | 1            |
| P1OUTS   | 0x4108  | 2    | 8              | 1            |
| P1OUTC   | 0x410C  | 3    | 12             | 1            |
| P1OUTT   | 0x4110  | 4    | 16             | 4            |
| P1DIR    | 0x4114  | 5    | 20             | 1            |
| P1IF     | 0x4118  | 6    | 24             | 4            |
| P1IES    | 0x411C  | 7    | 28             | 1            |
| P1IE     | 0x4120  | 8    | 32             | 1            |
| P1SEL    | 0x4124  | 9    | 36             | 1            |
| P1REN    | 0x4128  | 10   | 40             | 1            |
| P1AFS    | 0x412C  | 11   | 44             | 4            |

### 22.3 SPI0 Peripheral

Base address: 0x4200 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| SPI0CR   | 0x4200  | 0    | 0              | 4            |
| SPI0SR   | 0x4204  | 1    | 4              | 1            |
| SPI0TX   | 0x4208  | 2    | 8              | 4            |
| SPI0RX   | 0x420C  | 3    | 12             | 4            |
| SPI0FOS  | 0x4210  | 4    | 16             | 4            |

## 22.4 SPI1 Peripheral

Base address: 0x4300 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| SPI1CR   | 0x4300  | 0    | 0              | 4            |
| SPI1SR   | 0x4304  | 1    | 4              | 1            |
| SPI1TX   | 0x4308  | 2    | 8              | 4            |
| SPI1RX   | 0x430C  | 3    | 12             | 4            |
| SPI1FOS  | 0x4310  | 4    | 16             | 4            |

## 22.5 UART0 Peripheral

Base address: 0x4400 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| UART0CR  | 0x4400  | 0    | 0              | 1            |
| UART0SR  | 0x4404  | 1    | 4              | 1            |
| UART0BR  | 0x4408  | 2    | 8              | 2            |
| UART0RX  | 0x440C  | 3    | 12             | 1            |
| UART0TX  | 0x4410  | 4    | 16             | 1            |

## 22.6 UART1 Peripheral

Base address: 0x4500 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| UART1CR  | 0x4500  | 0    | 0              | 1            |
| UART1SR  | 0x4504  | 1    | 4              | 1            |
| UART1BR  | 0x4508  | 2    | 8              | 2            |
| UART1RX  | 0x450C  | 3    | 12             | 1            |
| UART1TX  | 0x4510  | 4    | 16             | 1            |

## 22.7 TIMER0 Peripheral

Base address: 0x4600 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| TIM0CR   | 0x4600  | 0    | 0              | 4            |
| TIM0SR   | 0x4604  | 1    | 4              | 1            |
| TIM0VAL  | 0x4608  | 2    | 8              | 4            |
| TIM0CMP0 | 0x460C  | 3    | 12             | 4            |
| TIM0CMP1 | 0x4610  | 4    | 16             | 4            |
| TIM0CMP2 | 0x4614  | 5    | 20             | 4            |
| TIM0CAP0 | 0x4618  | 6    | 24             | 4            |
| TIM0CAP1 | 0x461C  | 7    | 28             | 4            |

## 22.8 TIMER1 Peripheral

Base address: 0x4700 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| TIM1CR   | 0x4700  | 0    | 0              | 4            |
| TIM1SR   | 0x4704  | 1    | 4              | 1            |
| TIM1VAL  | 0x4708  | 2    | 8              | 4            |
| TIM1CMP0 | 0x470C  | 3    | 12             | 4            |
| TIM1CMP1 | 0x4710  | 4    | 16             | 4            |
| TIM1CMP2 | 0x4714  | 5    | 20             | 4            |
| TIM1CAP0 | 0x4718  | 6    | 24             | 4            |
| TIM1CAP1 | 0x471C  | 7    | 28             | 4            |

## 22.9 GPIO2 Peripheral

Base address: 0x4800 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| P2IN     | 0x4800  | 0    | 0              | 1            |
| P2OUT    | 0x4804  | 1    | 4              | 1            |
| P2OUTS   | 0x4808  | 2    | 8              | 1            |
| P2OUTC   | 0x480C  | 3    | 12             | 1            |
| P2OUTT   | 0x4810  | 4    | 16             | 4            |
| P2DIR    | 0x4814  | 5    | 20             | 1            |
| P2IF     | 0x4818  | 6    | 24             | 4            |
| P2IES    | 0x481C  | 7    | 28             | 1            |
| P2IE     | 0x4820  | 8    | 32             | 1            |
| P2SEL    | 0x4824  | 9    | 36             | 1            |
| P2REN    | 0x4828  | 10   | 40             | 1            |
| P2AFS    | 0x482C  | 11   | 44             | 4            |

## 22.10 SYSTEM Peripheral

Base address: 0x4900 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| SYSCLKCR | 0x4900  | 0    | 0              | 2            |
| CLKDIVCR | 0x4904  | 1    | 4              | 1            |
| BLOCKPWR | 0x4908  | 2    | 8              | 1            |
| CRCDATA  | 0x490C  | 3    | 12             | 1            |
| CRCSTATE | 0x4910  | 4    | 16             | 2            |
| WDTPASS  | 0x4930  | 12   | 48             | 4            |
| WDTCR    | 0x4934  | 13   | 52             | 1            |
| WDTSR    | 0x4938  | 14   | 56             | 1            |
| WDTVAL   | 0x493C  | 15   | 60             | 4            |
| DC00BIAS | 0x4940  | 16   | 64             | 2            |
| DC01BIAS | 0x4944  | 17   | 68             | 2            |



## 22.11 NPU Peripheral

Base address: 0x4A00 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| NPUCR    | 0x4A00  | 0    | 0              | 4            |
| NPUIVSAR | 0x4A04  | 1    | 4              | 4            |
| NPUVVSAR | 0x4A08  | 2    | 8              | 4            |
| NPUOVSAR | 0x4A0C  | 3    | 12             | 4            |
| NPUSR    | 0x4A10  | 4    | 16             | 4            |
| NPUCFG1  | 0x4A14  | 5    | 20             | 4            |
| NPUCFG2  | 0x4A18  | 6    | 24             | 4            |

## 22.12 PWRCTRL Peripheral

Base address: 0x4B00 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| PWRCR    | 0x4B00  | 0    | 0              | 4            |
| PWRSR    | 0x4B04  | 1    | 4              | 4            |

## 22.13 GPI03 Peripheral

Base address: 0x4D00 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| P3IN     | 0x4D00  | 0    | 0              | 1            |
| P3OUT    | 0x4D04  | 1    | 4              | 1            |
| P3OUTS   | 0x4D08  | 2    | 8              | 1            |
| P3OUTC   | 0x4D0C  | 3    | 12             | 1            |
| P3OUTT   | 0x4D10  | 4    | 16             | 4            |
| P3DIR    | 0x4D14  | 5    | 20             | 1            |
| P3IF     | 0x4D18  | 6    | 24             | 4            |
| P3IES    | 0x4D1C  | 7    | 28             | 1            |
| P3IE     | 0x4D20  | 8    | 32             | 1            |
| P3SEL    | 0x4D24  | 9    | 36             | 1            |
| P3REN    | 0x4D28  | 10   | 40             | 1            |
| P3AFS    | 0x4D2C  | 11   | 44             | 4            |

## 22.14 I2C0 Peripheral

Base address: 0x4E00 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| I2C0CR   | 0x4E00  | 0    | 0              | 4            |
| I2C0FCR  | 0x4E04  | 1    | 4              | 1            |
| I2C0SR   | 0x4E08  | 2    | 8              | 2            |
| I2C0MTX  | 0x4E0C  | 3    | 12             | 1            |
| I2C0MRX  | 0x4E10  | 4    | 16             | 1            |

|         |        |   |    |   |
|---------|--------|---|----|---|
| I2C0STX | 0x4E14 | 5 | 20 | 1 |
| I2C0SRX | 0x4E18 | 6 | 24 | 1 |
| I2C0AR  | 0x4E1C | 7 | 28 | 1 |
| I2C0AMR | 0x4E20 | 8 | 32 | 1 |

## 22.15 I2C1 Peripheral

Base address: 0x4F00 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| I2C1CR   | 0x4F00  | 0    | 0              | 4            |
| I2C1FCR  | 0x4F04  | 1    | 4              | 1            |
| I2C1SR   | 0x4F08  | 2    | 8              | 2            |
| I2C1MTX  | 0x4F0C  | 3    | 12             | 1            |
| I2C1MRX  | 0x4F10  | 4    | 16             | 1            |
| I2C1STX  | 0x4F14  | 5    | 20             | 1            |
| I2C1SRX  | 0x4F18  | 6    | 24             | 1            |
| I2C1AR   | 0x4F1C  | 7    | 28             | 1            |
| I2C1AMR  | 0x4F20  | 8    | 32             | 1            |

## 22.16 CLINT Peripheral

Base address: 0x5000 (shared window, arbitrated across all harts)

| Register   | Address | Slot | Offset (bytes) | Size (bytes) |
|------------|---------|------|----------------|--------------|
| MSIP0      | 0x5000  | 0    | 0              | 4            |
| MSIP1      | 0x5004  | 1    | 4              | 4            |
| MSIP2      | 0x5008  | 2    | 8              | 4            |
| MSIP3      | 0x500C  | 3    | 12             | 4            |
| MTIMEL     | 0x5010  | 4    | 16             | 4            |
| MTIMEH     | 0x5014  | 5    | 20             | 4            |
| MTIMECMP0L | 0x5020  | 8    | 32             | 4            |
| MTIMECMP0H | 0x5024  | 9    | 36             | 4            |
| MTIMECMP1L | 0x5028  | 10   | 40             | 4            |
| MTIMECMP1H | 0x502C  | 11   | 44             | 4            |
| MTIMECMP2L | 0x5030  | 12   | 48             | 4            |
| MTIMECMP2H | 0x5034  | 13   | 52             | 4            |
| MTIMECMP3L | 0x5038  | 14   | 56             | 4            |
| MTIMECMP3H | 0x503C  | 15   | 60             | 4            |

## 22.17 MUTEX Peripheral

Base address: 0x6000 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| MUTEX0   | 0x6000  | 0    | 0              | 4            |
| MUTEX1   | 0x6004  | 1    | 4              | 4            |
| MUTEX2   | 0x6008  | 2    | 8              | 4            |
| MUTEX3   | 0x600C  | 3    | 12             | 4            |

|         |        |    |    |   |
|---------|--------|----|----|---|
| MUTEX4  | 0x6010 | 4  | 16 | 4 |
| MUTEX5  | 0x6014 | 5  | 20 | 4 |
| MUTEX6  | 0x6018 | 6  | 24 | 4 |
| MUTEX7  | 0x601C | 7  | 28 | 4 |
| MUTEX8  | 0x6020 | 8  | 32 | 4 |
| MUTEX9  | 0x6024 | 9  | 36 | 4 |
| MUTEX10 | 0x6028 | 10 | 40 | 4 |
| MUTEX11 | 0x602C | 11 | 44 | 4 |
| MUTEX12 | 0x6030 | 12 | 48 | 4 |
| MUTEX13 | 0x6034 | 13 | 52 | 4 |
| MUTEX14 | 0x6038 | 14 | 56 | 4 |
| MUTEX15 | 0x603C | 15 | 60 | 4 |

## 22.18 GPI04 Peripheral

Base address: 0x6300 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| P4IN     | 0x6300  | 0    | 0              | 1            |
| P4OUT    | 0x6304  | 1    | 4              | 1            |
| P4OUTS   | 0x6308  | 2    | 8              | 1            |
| P4OUTC   | 0x630C  | 3    | 12             | 1            |
| P4OUTT   | 0x6310  | 4    | 16             | 4            |
| P4DIR    | 0x6314  | 5    | 20             | 1            |
| P4IF     | 0x6318  | 6    | 24             | 4            |
| P4IES    | 0x631C  | 7    | 28             | 1            |
| P4IE     | 0x6320  | 8    | 32             | 1            |
| P4SEL    | 0x6324  | 9    | 36             | 1            |
| P4REN    | 0x6328  | 10   | 40             | 1            |
| P4AFS    | 0x632C  | 11   | 44             | 4            |

## 22.19 GPI05 Peripheral

Base address: 0x6400 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| P5IN     | 0x6400  | 0    | 0              | 1            |
| P5OUT    | 0x6404  | 1    | 4              | 1            |
| P5OUTS   | 0x6408  | 2    | 8              | 1            |
| P5OUTC   | 0x640C  | 3    | 12             | 1            |
| P5OUTT   | 0x6410  | 4    | 16             | 4            |
| P5DIR    | 0x6414  | 5    | 20             | 1            |
| P5IF     | 0x6418  | 6    | 24             | 4            |
| P5IES    | 0x641C  | 7    | 28             | 1            |
| P5IE     | 0x6420  | 8    | 32             | 1            |
| P5SEL    | 0x6424  | 9    | 36             | 1            |
| P5REN    | 0x6428  | 10   | 40             | 1            |
| P5AFS    | 0x642C  | 11   | 44             | 4            |

## 22.20 IRQROUTER Peripheral

Base address: 0x7000 (shared window, arbitrated across all harts)

| Register | Address | Slot | Offset (bytes) | Size (bytes) |
|----------|---------|------|----------------|--------------|
| H0ENL    | 0x7000  | 0    | 0              | 4            |
| H0ENM    | 0x7004  | 1    | 4              | 4            |
| H0ENU    | 0x7008  | 2    | 8              | 4            |
| H0ENX    | 0x700C  | 3    | 12             | 4            |
| H1ENL    | 0x7010  | 4    | 16             | 4            |
| H1ENM    | 0x7014  | 5    | 20             | 4            |
| H1ENU    | 0x7018  | 6    | 24             | 4            |
| H1ENX    | 0x701C  | 7    | 28             | 4            |
| H2ENL    | 0x7020  | 8    | 32             | 4            |
| H2ENM    | 0x7024  | 9    | 36             | 4            |
| H2ENU    | 0x7028  | 10   | 40             | 4            |
| H2ENX    | 0x702C  | 11   | 44             | 4            |
| H3ENL    | 0x7030  | 12   | 48             | 4            |
| H3ENM    | 0x7034  | 13   | 52             | 4            |
| H3ENU    | 0x7038  | 14   | 56             | 4            |
| H3ENX    | 0x703C  | 15   | 60             | 4            |
| CLAIM    | 0x7800  | 512  | 2048           | 4            |
| PENDL    | 0x7810  | 516  | 2064           | 4            |
| PENDM    | 0x7814  | 517  | 2068           | 4            |
| PENDU    | 0x7818  | 518  | 2072           | 4            |
| PENDX    | 0x781C  | 519  | 2076           | 4            |
| INSVCL   | 0x7820  | 520  | 2080           | 4            |
| INSVCM   | 0x7824  | 521  | 2084           | 4            |
| INSVCU   | 0x7828  | 522  | 2088           | 4            |
| INSVCX   | 0x782C  | 523  | 2092           | 4            |

## 23 Errata

There are no known errata for the Castalia MCU at this time. This section will list hardware issues, their impact, and software workarounds as they are discovered.